
A tensor network formalism for neuro-symbolic AI

Alex Goessmann*, Janina Schütte, Maximilian Fröhlich, and Martin Eigel

Weierstrass Institute of Applied Analysis and Stochastics
Anton-Wilhelm-Amo-Straße 39
Berlin, 10117, Germany

*alex.goessmann@wias-berlin.de

ABSTRACT

The unification of neural and symbolic approaches to artificial intelligence remains a central open challenge. In this work, we introduce a tensor network formalism, which captures sparsity principles originating in the different approaches in tensor decompositions. In particular, we describe a basis encoding scheme for functions and model neural decompositions as tensor decompositions. The proposed formalism can be applied to represent logical formulas and probability distributions as structured tensor decompositions. This unified treatment identifies tensor network contractions as a fundamental inference class and formulates efficiently scaling reasoning algorithms, originating from probability theory and propositional logic, as contraction message passing schemes. The framework enables the definition and training of hybrid logical and probabilistic models, which we call Hybrid Logic Networks. The theoretical concepts are accompanied by the python library `tnreason`, which enables the implementation and practical use of the proposed architectures.

1 Introduction

Modern artificial intelligence is dominated by large-scale neural models that excel at various tasks but mostly remain black-boxes. While these models offer adaptability, the two main concerns when integrating these architectures into safety-critical processes are reliability and explainability. To match these demands, artificial intelligence has followed symbolic paradigms, including probabilistic and logical approaches. However, these paradigms have been mostly neglected due to the success of black-box neural models. The logical tradition of artificial intelligence, historically motivated by the resemblance of human thought to formal logics McCarthy (1959), offers explicit structures and human-readable inference. However, the main problem hindering the success of this classical approach is the inability of classical first-order logic to handle uncertainty or scale to complex real-world data. Probabilistic graphical models Pearl (1988); Koller and Friedman (2009) provide insights based on encoded variable independences and causality Pearl (2009). While probabilistic models and Statistical Relational AI Nickel et al. (2016); Getoor and Taskar (2019) have improved uncertainty handling, bridging these paradigms remains the central goal of *Neuro-Symbolic AI* Hochreiter (2022); Sarker et al. (2022); Colelough and Regli (2024). The field seeks a single, mathematically coherent framework combining structural clarity with neural adaptability. Although progress has been made, for example with Markov Logic Networks Richardson and Domingos (2006), a fully unified substrate that treats logical and probabilistic inference as instances of the same operation is still missing.

In this work, we propose to fill the gap between the probabilistic, neural, and logical paradigms with *tensor networks* in a framework called `tnreason`. Tensor spaces capture both the semantics of logical formulas (by boolean tensors) and probability distributions (by normalized non-negative tensors). This abstraction eliminates the traditional divide between symbolic and neural representations: logical inference, probabilistic computations, and neural inference become different instances of the same underlying operation. As naive tensors are prone to the curse of dimensionality, we turn to distributed representation schemes by tensor networks. We show that fundamental sparsity principles of neural and symbolic AI, such as conditional independence, the existence of sufficient statistics, and neural model decomposition, are equivalent to tensor network decompositions. Moreover, we identify tensor network contraction as the fundamental operation underlying inference tasks, such as computing marginal distributions and deciding entailment. While these contractions are in general computationally hard, efficient schemes to perform inference

are known as message passing schemes. These algorithms have appeared in different communities under names such as belief propagation Pearl (1988); Mézard (2009) and constraint propagation Mackworth (1977). We review these schemes based on our tensor network formalism.

To capture both logical and probabilistic models, and exploit their neural decompositions, we introduce *Computation-Activation Networks (CompActNets)*, an expressive tensor network architecture. The architecture consists of two complementary subarchitectures that serve the purpose of computation and activation, respectively. The *computation network* prepares auxiliary hidden variables with deterministic dependence on the main variables. We interpret the network as a distributed computation scheme of functions describing this dependence, whose decomposition is related to sparsity concepts of the corresponding functions. These auxiliary variables represent logical formulas or more generic statistics in different contexts. The *activation network* then assigns numerical values to the states of those auxiliary variables, and in this way activates the variables to represent factors of a model. Logical models emerge when the activation network is a boolean tensor, probabilistic exponential families when they are elementary positive-valued tensors, and hybrid models in the most general cases.

1.1 Related works

Historically rooted in quantum many-body physics White (1993), tensor networks found their first major success with Matrix Product States (MPS), originally developed to efficiently capture the quantum dynamics and ground states of one-dimensional spin chains Affleck et al. (1987). This format remains a standard tool in the field, with recent contributions refining it for tasks such as large-scale stochastic simulations and variational circuit operations Sander et al. (2025b,a). To address the topological constraints of MPS, the landscape of architectures was subsequently expanded to include Projected Entangled Pair States (PEPS) for two-dimensional lattices and the Multi-scale Entanglement Renormalization Ansatz (MERA), which utilizes a hierarchical geometry to represent scale-invariant critical systems and has recently been adapted for simulating quantum systems Orús (2019); Berezutskii et al. (2025).

Beyond the quantum realm, these formats have been successfully adapted to applied mathematics, particularly for solving high-dimensional parametric PDEs Eigel et al. (2019, 2020); Dolgov and Scheichl (2019); Dolgov et al. (2015); Trunschke et al. (2025), sampling problems and approximation of the Hamilton-Jacobi-Belman equation Gruhlke et al. (2026); Eigel et al. (2023); Dolgov et al. (2023); Cui and Dolgov (2022), modeling complex continuous fields and learning dynamical laws Hagemann et al. (2025); Eigel et al. (2017); Goessmann et al. (2020); Lubich et al. (2013). Furthermore, they exhibit properties helpful for handling these high-dimensional spaces, such as restricted isometry properties Goessmann (2021). Recent advancements have demonstrated the efficacy of these methods in capturing multiscale phenomena in fluid dynamics and turbulence, proving that the tensor network formalism offers a robust alternative to classical numerical schemes Gourianov et al. (2025).

The unification of neural, symbolic, and probabilistic approaches to interpretable model architectures has been a long-standing aim of Neuro-Symbolic AI. A central goal is to achieve *intrinsic explainability*, which, unlike post-hoc interpretations analysing input influence after training Lipton (2018); Barredo Arrieta et al. (2020), aims at explainability of the architecture itself. Early connectionist approaches Towell and Shavlik (1994); Avila Garcez and Zaverucha (1999) towards Neuro-Symbolic AI focus on embedding logical rules into neural connectivity. Further, fruitful relations with statistical relational learning have been identified Marra et al. (2024).

Tensor networks have recently gained interest as a unifying language for AI, framed by Logical Tensor Networks Badreddine et al. (2022) and Tensor Logic Domingos (2025). Furthermore, the MeLoCoToN approach Ali (2025) applies tensor network architectures similar to CompActNets in combinatorial optimization problems. Specifically, tensor networks have emerged as a highly efficient mathematical framework for handling data in high-dimensional spaces, effectively circumventing the "curse of dimensionality" that typically plagues grid-based methods Hackbusch (2012). By decomposing high-order tensors into networks of low-rank components, these structures reduce the storage and computational complexity from exponential to polynomial with respect to the dimension Oseledets (2011); Hackbusch and Kühn (2009); Hitchcock (1927).

1.2 Structure of the paper

The paper is organized as follows. In Section 2 we introduce the basic concepts and notation for categorical variables, tensors, and tensor networks, which are applied in the following sections. Sections 3, 4, and 5 anchor the tensor network formalism in the basic paradigms of artificial intelligence (see Figure 1). The *probabilistic paradigm* is discussed in Section 3, where we in particular show that concepts of independence, graphical models, and sufficient statistics correspond to specific tensor network decompositions. Section 4 is dedicated to the *neural paradigm*, where we show that generic function decompositions have analogous tensor network representations. Section 5 turns to the

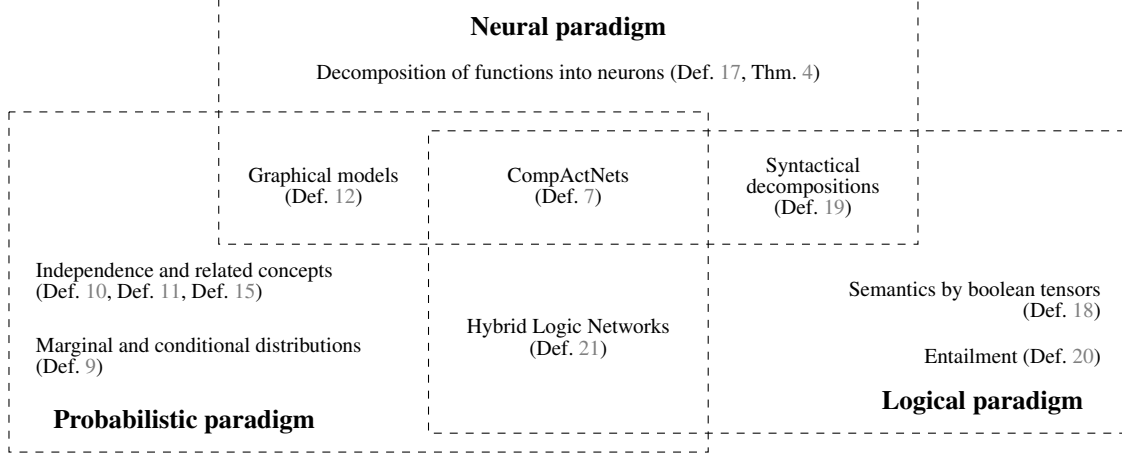


Figure 1: Sketch of the concepts in the neural, probabilistic and logical paradigms, which we define based on tensor network decompositions and contractions.

logical paradigm, where we study tensor equivalents of propositional formulas, knowledge bases, and entailment. In Section 6 we present *Hybrid Logic Networks* as an application of the unified tensor network formalism, combining logical and probabilistic models. We briefly discuss the implementation of these concepts in our open source python package *tnreason* in Section 7, before concluding the paper in Section 8.

2 Foundations

In this section, we introduce the hypergraph-based tensor network formalism and define the most general tensor network architecture of *CompActNets* based on this formalism.

2.1 Tensors

Tensors are multiway arrays and a generalization of vectors and matrices to higher orders. We first provide a formal definition as real maps from index sets enumerating the coordinates of vectors, matrices, and higher-order tensors. To ease the notation, we abbreviate sets as $[d] = \{0, \dots, d-1\}$, tuples of state indices by $x_{[d]} = (x_0, \dots, x_{d-1})$ and tuples of variables by $X_{[d]} = (X_0, \dots, X_{d-1})$.

Definition 1 (Tensor). *For $k \in [d]$, let $m_k \in \mathbb{N}$ and let X_k be categorical variables taking values in $[m_k]$. A tensor $\tau[X_0, \dots, X_{d-1}]$ of order d and leg dimensions $m_0, \dots, m_{d-1}$ is defined through its coordinates*

$$\tau[X_{[d]} = x_{[d]}] = \tau[X_0 = x_0, \dots, X_{d-1} = x_{d-1}] \in \mathbb{R}$$

for index tuples

$$x_{[d]} = (x_0, \dots, x_{d-1}) \in \bigtimes_{k \in [d]} [m_k].$$

Tensors $\tau[X_{[d]}]$ are elements of the tensor space

$$\bigotimes_{k \in [d]} \mathbb{R}^{m_k},$$

which is a linear space, enriched with the operations of coordinate-wise summation and scalar multiplication. We call a tensor $\tau[X_{[d]}]$ *boolean*, when all coordinates are in $\{0, 1\}$, and *positive*, when all coordinates are greater than 0.

We introduced tensors here in a non-canonical way based on categorical variables assigned to their axes. While this may look like syntactic sugar at this point, it allows us to define contractions without further specification of axes, based on comparisons of shared variables. We occasionally also allow for variables X taking values in infinite sets such as $\mathbb{R}$, in which case we denote the set of values to a variable by $\text{val}(X)$.

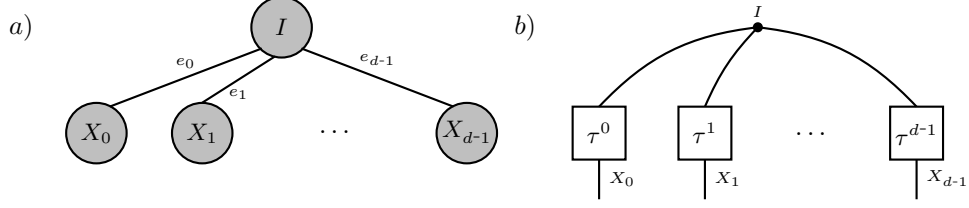


Figure 2: Hypergraph to a CP format (see Example 2). a) Node-centric design. b) Corresponding tensor network on the edges of the hypergraph.

Example 1 (Delta tensor). Given a tuple of variables $X_{[d]} = (X_0, \dots, X_{d-1})$ with identical dimension m , where $d \geq 1$, the delta tensor is the element

$$\delta^{[d],m} [X_{[d]}] \in \bigotimes_{k \in [d]} \mathbb{R}^m$$

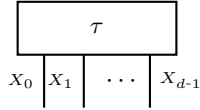
with coordinates

$$\delta^{[d],m} [X_{[d]} = x_{[d]}] = \begin{cases} 1 & \text{if } x_0 = \dots = x_{d-1} \\ 0 & \text{else} \end{cases}.$$

We depict this tensor by black dots, which sometimes appears as auxiliary elements in tensor network diagrams (see e.g. Figure 4). For $d = 1$, the delta tensor is the trivial vector, whose coordinates are constantly 1, which we denote by $\mathbb{I}[X]$.

2.2 Tensor networks and contractions

We use a standard visualization of tensors (dating back to Penrose (1987)) by blocks with lines depicting the axes of the tensor. Additionally, we assign to each axis of the tensor the corresponding variable X_k :



We now associate categorical variables with nodes of a hypergraph and tensors with hyperedges, which are arbitrary subsets of nodes. Based on this association we continue with the definition of tensor networks.

Definition 2 (Tensor network). Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ be a hypergraph, let X_v for $v \in \mathcal{V}$ be categorical variables with dimensions $m_v \in \mathbb{N}$, and let

$$\tau^e [X_e] \in \bigotimes_{v \in e} \mathbb{R}^{m_v}$$

be tensors for $e \in \mathcal{E}$, where we denote by X_e the set of categorical variables X_v with $v \in e$. Then, we call the set

$$\tau^{\mathcal{G}} [X_{\mathcal{V}}] = \{\tau^e [X_e] : e \in \mathcal{E}\}$$

the tensor network of the decorated hypergraph $\mathcal{G}$. The set of tensor networks on $\mathcal{G}$ such that all tensors have non-negative coordinates is denoted by $\mathcal{T}^{\mathcal{G}}$.

As examples we now present the CP and the TT formats in our hypergraph notation.

Example 2 (The CP format). The Candecomp-Parafac (CP) tensor format (see Hitchcock (1927)) corresponds in our notation to a hypergraph (see Figure 2) defined by

- nodes $X_{[d]}$ and a single hidden variable I , decorated by dimensions $m_{[d]}$ and the CP-rank n , respectively
- edges $\{e_k = \{X_k, I\} : k \in [d]\}$ each decorated by a matrix $\tau^{e_k} [X_k, I] \in \mathbb{R}^{m_k \times n}$.

Example 3 (The TT format). The Tensor-Train (TT) format (see Oseledets (2011)) corresponds in our notation to a hypergraph (see Figure 3) defined by

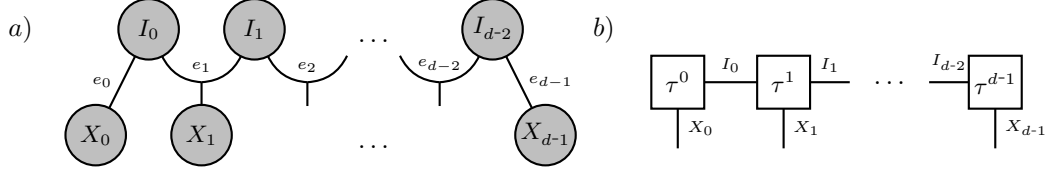


Figure 3: Hypergraph to a TT format (see Example 3). a) Node-centric design. b) Corresponding tensor network on the edges of the hypergraph.

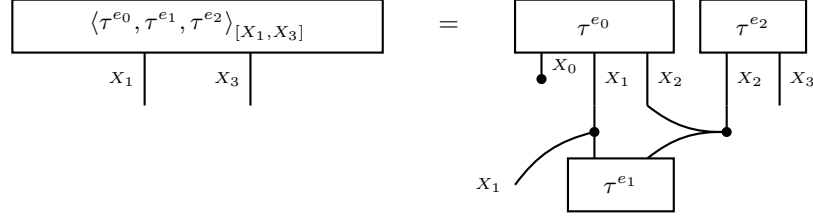


Figure 4: Graphical depiction of a tensor network contraction with the open variables X_1, X_3 . Open variables are depicted by those without a dot at the end of the line.

- nodes $X_{[d]}$ and hidden variables $I_{[d-1]}$, each decorated by a dimension $m_{[d]}$ and $n_{[d-1]}$,
- edges

$$\{e_0 = \{X_0, I_0\}\} \cup \{e_k = \{I_{k-1}, X_k, I_k\} : k \in \{1, \dots, d-2\}\} \cup \{e_{d-1} = \{I_{d-2}, X_{d-1}\}\}$$
 each decorated by a tensor of order 3 (respectively 2 for $k \in \{0, d-1\}$).

2.3 Generic contractions

Let us now exploit our graphical approach to tensor networks in the definition of contractions.

Definition 3. Let $\tau^{\mathcal{G}}$ be a tensor network on a decorated hypergraph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$. For any subset $\mathcal{U} \subset \mathcal{V}$ we define the contraction of $\tau^{\mathcal{G}}$ with open variables $X_{\mathcal{U}}$ to be the tensor (for an example see Figure 4)

$$\langle \tau^{\mathcal{G}} \rangle_{[X_{\mathcal{U}}]} \in \bigotimes_{v \in \mathcal{U}} \mathbb{R}^{m_v}$$

with coordinates at indices $x_{\mathcal{U}} \in \times_{v \in \mathcal{U}} [m_v]$ by

$$\langle \tau^{\mathcal{G}} \rangle_{[X_{\mathcal{U}}=x_{\mathcal{U}}]} = \sum_{x_{\mathcal{V} \setminus \mathcal{U}} \in \times_{v \in \mathcal{V} \setminus \mathcal{U}} [m_v]} \left(\prod_{e \in \mathcal{E}} \tau^e [X_e = x_e] \right).$$

When an open variable X does not appear in any tensor in a contraction, we define the contraction as a tensor product with the trivial tensor $\mathbb{I}[X]$ (see Example 1). To ease notation, we often omit the set notation by brackets $\{\cdot\}$.

Example 4 (Tensor product). The simplest contraction is the tensor product, which maps a pair of two tensors with distinct variables onto a third tensor and has an interpretation by coordinate-wise products. Such a contraction corresponds with a tensor network of two tensors with disjoint variables. Let there be two tensors

$$\tau [X_{[d]}] \in \bigotimes_{k \in [d]} \mathbb{R}^{m_k} \quad \text{and} \quad \tilde{\tau} [Y_{[p]}] \in \bigotimes_{\ell \in [p]} \mathbb{R}^{n_{\ell}}$$

with disjoint tuples of categorical variables assigned to their axes. Then their tensor product is the tensor

$$\langle \tau [X_{[d]}], \tilde{\tau} [Y_{[p]}] \rangle_{[X_{[d]}, Y_{[p]}]} \in \left(\bigotimes_{k \in [d]} \mathbb{R}^{m_k} \right) \otimes \left(\bigotimes_{\ell \in [p]} \mathbb{R}^{n_{\ell}} \right)$$

with coordinates to tuples of $x_{[d]} \in \times_{k \in [d]} [m_k]$ and $y_{[p]} \in \times_{\ell \in [p]} [n_{\ell}]$ as

$$\begin{aligned} \langle \tau [X_{[d]}], \tilde{\tau} [Y_{[p]}] \rangle_{[X_{[d]}=x_{[d]}, Y_{[p]}=y_{[p]}]} \\ := \tau [X_{[d]} = x_{[d]}] \cdot \tilde{\tau} [Y_{[p]} = y_{[p]}]. \end{aligned}$$

2.4 Normalizations

Based on generic contractions, we now introduce the normalization of tensors, which introduces certain constraints on tensors to be depicted by directed hyperedges.

Definition 4. The normalization of a tensor $\tau[X_V]$ on incoming nodes $V^{\text{in}} \subset V$ and outgoing nodes $V^{\text{out}} \subset V \setminus V^{\text{in}}$ is the tensor $\langle \tau[X_V] \rangle_{[X_{V^{\text{out}}} | X_{V^{\text{in}}}]}$ defined for $x_{V^{\text{in}}}$ as

$$\langle \tau[X_V] \rangle_{[X_{V^{\text{out}}} | X_{V^{\text{in}}} = x_{V^{\text{in}}}] = \begin{cases} \frac{\langle \tau \rangle_{[X_{V^{\text{out}}}, X_{V^{\text{in}}} = x_{V^{\text{in}}}]}}{\langle \tau \rangle_{[X_{V^{\text{in}}} = x_{V^{\text{in}}}]}} & \text{if } \langle \tau \rangle_{[X_{V^{\text{in}}} = x_{V^{\text{in}}}] \neq 0 \\ \frac{1}{\prod_{v \in V^{\text{out}}} m_v} \mathbb{I}[X_{V^{\text{out}}}] & \text{else} \end{cases}.$$

We say that $\tau[X_V]$ is normalized with incoming nodes $V^{\text{in}} \subset V$, if

$$\tau[X_V] = \langle \tau[X_V] \rangle_{[X_{V \setminus V^{\text{in}}} | X_{V^{\text{in}}}]}$$

In our graphical tensor notation, we depict normalized tensors by directed hyperedges (a), which are decorated by directed tensors (b), for example when $X_{V^{\text{in}}} = (X_2, X_3)$ and $X_{V \setminus V^{\text{in}}} = (X_0, X_1)$:



2.5 Function encoding and Computation-Activation Networks

Towards presenting the function encoding schemes, we define one-hot encodings mapping the states of variables to basis tensors.

Definition 5 (One-hot encoding). To any variable X taking values in $[m]$, the one-hot encoding of any state $x \in [m]$ is the vector with coordinates

$$\epsilon_x[X = \tilde{x}] := \begin{cases} 1 & \text{if } x = \tilde{x} \\ 0 & \text{else} \end{cases}.$$

To any tuple $X_{[d]}$ of variables taking values in $\times_{k \in [d]} [m_k]$, the one-hot encoding of a state tuple $x_{[d]}$ is the tensor product

$$\epsilon_{x_{[d]}}[X_{[d]}] := \bigotimes_{k \in [d]} \epsilon_{x_k}[X_k].$$

We now use one-hot encodings to encode functions between state sets.

Definition 6 (Basis encoding of maps between state sets). Let there be two sets of variables $X_{[d]}$ and $Y_{[p]}$, and let there be a map

$$q : \times_{k \in [d]} [m_k] \rightarrow \times_{\ell \in [p]} [n_\ell]$$

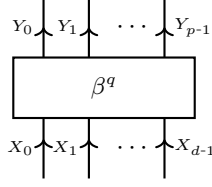
between their state sets. Then, the basis encoding of q is a tensor

$$\beta^q[Y_{[p]}, X_{[d]}] \in \left(\bigotimes_{\ell \in [p]} \mathbb{R}^{n_\ell} \right) \otimes \left(\bigotimes_{k \in [d]} \mathbb{R}^{m_k} \right)$$

defined by

$$\beta^q[Y_{[p]}, X_{[d]}] = \sum_{x_{[d]} \in \times_{k \in [d]} [m_k]} \epsilon_{q(x_{[d]})}[Y_{[p]}] \otimes \epsilon_{x_{[d]}}[X_{[d]}].$$

Basis encodings are normalized tensors and are thus depicted as decorations of directed edges in hypergraphs:



We further generalize basis encodings to arbitrary functions between finite sets by the use of bijective image enumeration maps. Given an arbitrary set $\mathcal{U}$, we say a map

$$I : \prod_{k \in [d]} [m_k] \rightarrow \mathcal{U}$$

is an enumeration map of $\mathcal{U}$ by d variables X_k , taking values in m_k . Given a function $q : \mathcal{U}^{\text{in}} \rightarrow \mathcal{U}^{\text{out}}$ between arbitrary sets and enumerating maps I_{in} and I_{out} for both sets, we define the basis encoding of q as

$$\beta^q [Y_{[p]}, X_{[d]}] = \sum_{u \in \mathcal{U}^{\text{in}}} \epsilon_{I_{\text{out}}^{-1}(q(u))} [Y_{[p]}] \otimes \epsilon_{I_{\text{in}}^{-1}(u)} [X_{[d]}],$$

where $X_{[d]}, Y_{[p]}$ are variables taking values in $[[\mathcal{U}^{\text{in}}]]$ and $[[\mathcal{U}^{\text{out}}]]$. In Example 13 we present index enumeration maps for summations in m -adic integer representations. Based on these concepts, we define the most general tensor network architecture to be applied in the rest of this work.

Definition 7 (Computation-Activation Network (CompActNets)). *Let there be a function $t : \prod_{k \in [d]} [m_k] \rightarrow \mathbb{R}^p$ with basis encoding $\beta^t [Y_{[p]}, X_{[d]}]$, where $Y_{[p]}$ is a tuple of variables to an enumeration map of the image of t . Let there further be a hypergraph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ with nodes $\mathcal{V}$ containing $[p]$. We define the by t computable and by $\mathcal{G}$ activated family of distributions by*

$$\Lambda^{t, \mathcal{G}} = \left\{ \left\langle \beta^t [Y_{[p]}, X_{[d]}], \langle \xi \rangle_{[Y_{[p]}]} \right\rangle_{[X_{[d]} | \emptyset]} : \xi [Y_{\mathcal{V}}] \in \mathcal{T}^{\mathcal{G}} \right\}.$$

We refer to any member $\mathbb{P} [X_{[d]}] \in \Lambda^{t, \mathcal{G}}$ as a Computation-Activation Network (or shorter as a CompActNet). We call $\beta^t [Y_{[p]}, X_{[d]}]$ (and any decomposition of it) the computation network and $\xi [Y_{\mathcal{V}}]$ the activation network.

The elementary activated networks are representable by an elementary activation tensor with respect to the graph

$$\text{EL} = (\mathcal{V}, \{\{v\} : v \in \mathcal{V}\})$$

and we denote such networks by $\Lambda^{t, \text{EL}}$. Any CompActNet is representable with respect to the maximal hypergraph

$$\text{MAX} = (\mathcal{V}, \{\mathcal{V}\}).$$

We therefore have for any graph that $\Lambda^{t, \mathcal{G}} \subset \Lambda^{t, \text{MAX}}$.

3 The probabilistic paradigm

In the following we investigate tensor network decomposition mechanisms of probability distributions. After introducing probability distributions as tensors and independencies as decomposition schemes, we derive tensor network decompositions based on conditional independencies (applying a classical theorem of Hammersley-Clifford, see Clifford and Hammersley (1971)) to motivate graphical models. Furthermore, we present the Fisher-Neyman Factorization Theorem as providing decompositions in the presence of sufficient statistics.

3.1 Basic concepts

As defined next, distributions $\mathbb{P}$ over a discrete state space can be represented by tensors, where each entry corresponds to the probability of a corresponding state.

Definition 8 (Joint probability distribution). *Let there be for each $k \in [d]$ a categorical variable X_k taking values in $[m_k]$. A joint probability distribution of these categorical variables is a tensor*

$$\mathbb{P} [X_{[d]}] \in \bigotimes_{k \in [d]} \mathbb{R}^{m_k}$$

which coordinates are non-negative, that is for any $x_{[d]} \in \times_{k \in [d]} [m_k]$ it holds

$$\mathbb{P}[X_{[d]} = x_{[d]}] \geq 0,$$

and which is normalized with no incoming variables, that is

$$\langle \mathbb{P}[X_{[d]}] \rangle_{[\emptyset]} = 1.$$

Let Z be another variable taking values in a possibly infinite set $\text{val}(Z)$. Then, a tensor $\mathbb{P}[X_{[d]}|Z]$ is a family of joint probability distributions if, for any $z \in \text{val}(Z)$, the slice $\mathbb{P}[X_{[d]}|Z = z]$ is a joint probability distribution.

Example 5 (Family of independent coin tosses). Consider tossing a coin with head probability $z \in [0, 1]$ and repeating the experiment independently $d \in \mathbb{N}$ times. We define a variable Z taking values in $\text{val}(Z) = [0, 1]$ and denote by $X_{[d]}$ d boolean variables. Then, the family of coin toss distributions is the tensor $\mathbb{P}[X_{[d]}|Z]$ with coordinates $x_{[d]} \in \times_{k \in [d]} [2]$ and $z \in [0, 1]$ defined by

$$\mathbb{P}[X_{[d]} = x_{[d]}|Z = z] = \prod_{k \in [d]} z^{x_k} (1 - z)^{1 - x_k} = z^{\sum_{k \in [d]} x_k} (1 - z)^{d - \sum_{k \in [d]} x_k}.$$

Note that by the binomial theorem we have $\langle \mathbb{P}[X_{[d]}, Z = z] \rangle_{[\emptyset]} = 1$ for each slice with respect to $z \in [0, 1]$. Therefore, $\mathbb{P}[X_{[d]}, Z]$ is indeed a family of probability distributions. For $d = 2$ we have more explicitly for any $z \in [0, 1]$ that

$$\mathbb{P}[X_{[2]}|Z = z] = \begin{array}{c} \begin{array}{ccc} & X_1 & \\ & \text{-----} & \\ 0 & & 1 \end{array} \\ x_0 \downarrow \begin{array}{cc} 0 & \\ \downarrow 1 & \end{array} \left[\begin{array}{cc} (1 - z)^2 & z \cdot (1 - z) \\ z \cdot (1 - z) & z^2 \end{array} \right] \end{array}.$$

A basic inference operation on probability distributions is the computation of marginal and conditional distributions.

Definition 9. For any distribution $\mathbb{P}[X_0, X_1]$ the marginal distribution is the contraction (see Def. 3)

$$\mathbb{P}[X_0] := \langle \mathbb{P}[X_0, X_1] \rangle_{[X_1]},$$

which is depicted by the diagram

$$\begin{array}{c} \boxed{\mathbb{P}[X_0]} \\ x_0 \downarrow \end{array} = \begin{array}{c} \boxed{\mathbb{P}[X_0, X_1]} \\ x_0 \downarrow \quad x_1 \downarrow \bullet \end{array}$$

The conditional distribution of X_0 on X_1 is the normalization (see Def. 4)

$$\mathbb{P}[X_0|X_1] := \langle \mathbb{P}[X_0, X_1] \rangle_{[X_0|X_1]}.$$

For $x_1 \in [m_1]$ with $\langle \mathbb{P}[X_0, X_1 = x_1] \rangle_{[\emptyset]}$ we depict the normalization by

$$\begin{array}{c} \boxed{\mathbb{P}[X_0|X_1 = x_1]} \\ x_0 \downarrow \end{array} := \frac{\begin{array}{c} \boxed{\mathbb{P}[X_0, X_1]} \\ x_0 \downarrow \quad x_1 \downarrow \end{array} \quad \begin{array}{c} \epsilon_{x_1} \end{array}}{\begin{array}{c} \boxed{\mathbb{P}[X_0, X_1]} \\ x_0 \downarrow \quad x_1 \downarrow \bullet \end{array} \quad \begin{array}{c} \epsilon_{x_1} \end{array}} =: \begin{array}{c} \boxed{\mathbb{P}[X_0|X_1]} \\ x_0 \downarrow \quad x_1 \uparrow \end{array} \quad \begin{array}{c} \epsilon_{x_1} \end{array}$$

3.2 Factorization into graphical models

The number of coordinates in a tensor representation of probability distributions is the product

$$\prod_{k \in [d]} m_k .$$

It therefore scales exponentially in the number of coordinates. To find efficient representation schemes of probability distributions by tensor networks, we need to exploit additional properties of the distribution. Independence leads to severe sparsifications of conditional probabilities and is hence the key assumption to gain sparse decompositions of probability distributions.

Definition 10 (Independence). *We say that X_0 is independent of X_1 with respect to a distribution $\mathbb{P}[X_0, X_1]$ if the distribution is the tensor product of the marginal distributions, that is*

$$\mathbb{P}[X_0, X_1] = \mathbb{P}[X_0] \otimes \mathbb{P}[X_1] .$$

In this case we write $(X_0 \perp X_1)$.

Thus, independence appears directly as a tensor-product decomposition of probability distribution. Using tensor network diagrams, we depict this property by

The diagram illustrates the decomposition of a joint distribution tensor $\mathbb{P}[X_0, X_1]$ into a tensor product of two marginal distribution tensors. On the left, a box labeled $\mathbb{P}[X_0, X_1]$ has two vertical legs labeled x_0 and x_1 with downward arrows. This is followed by an equals sign, then a tensor product symbol $\otimes$. To the right of the symbol is another box labeled $\mathbb{P}[X_0, X_1]$ with legs x_0 and x_1 . This is followed by another equals sign, then a tensor product symbol $\otimes$. Finally, there are two separate boxes: one labeled $\mathbb{P}[X_0]$ with leg x_0 , and another labeled $\mathbb{P}[X_1]$ with leg x_1 .

Note that the assumption of independence reduces the degrees of freedom from $(m_0 \cdot m_1) - 1$ to $(m_0 - 1) + (m_1 - 1)$. The decomposition into marginal distributions furthermore exploits this reduced freedom and provides an efficient storage. Having a joint distribution of multiple variables whose disjoint subsets are independent, we can iteratively apply the decomposition scheme. As a result, we can reduce the scaling of the degrees of freedom from exponential to linear by the assumption of independence.

As we observed, independence is a strong assumption, which is often too restrictive. Less demanding is conditional independence, which still implies efficient tensor network decomposition schemes. We introduce conditional independence as independence of variables with respect to conditional distributions.

Definition 11 (Conditional independence). *Assume a joint distribution of variables X_0, X_1 and X_2 . We say that X_0 is independent of X_1 conditioned on X_2 if*

$$\mathbb{P}[X_0, X_1 | X_2] = \langle \mathbb{P}[X_0 | X_2], \mathbb{P}[X_1 | X_2] \rangle_{[X_0, X_1, X_2]} .$$

In this case we write $(X_0 \perp X_1) | X_2$.

Conditional independence stated in Def. 11 has a close connection with independence stated in Def. 10. To be more precise, X_0 is independent of X_1 conditioned on X_2 if and only if X_0 is independent of X_1 with respect to any slice $\mathbb{P}[X_0, X_1 | X_2 = x_2]$ of the conditional distribution $\mathbb{P}[X_0, X_1 | X_2]$.

We can further exploit conditional independence to find tensor network decompositions of probabilities as we show in the next corollary.

Corollary 1. *Let $\mathbb{P}[X_0, X_1, X_2]$ be a joint distribution. If and only if X_0 is independent of X_1 conditioned on X_2 , the distribution satisfies*

$$\mathbb{P}[X_0, X_1, X_2] = \langle \mathbb{P}[X_0 | X_2], \mathbb{P}[X_1 | X_2], \mathbb{P}[X_2] \rangle_{[X_0, X_1, X_2]} .$$

In a diagrammatic notation, this is depicted by

The diagram illustrates the decomposition of a joint distribution tensor $\mathbb{P}[X_0, X_1, X_2]$ into a tensor product of three conditional distribution tensors. On the left, a box labeled $\mathbb{P}[X_0, X_1, X_2]$ has three vertical legs labeled x_0 , x_1 , and x_2 with downward arrows. This is followed by an equals sign, then a tensor product symbol $\otimes$. To the right of the symbol are three boxes: $\mathbb{P}[X_0 | X_2]$ with legs x_0 and x_2 , $\mathbb{P}[X_2]$ with leg x_2 , and $\mathbb{P}[X_1 | X_2]$ with legs x_2 and x_1 . The legs x_2 of the three boxes are connected by a curved line at the bottom, indicating a contraction over the x_2 index.

This conditional independence pattern is the basic local building block that is generalized in Markov networks, which we define in the following.

Definition 12 (Markov network). *Let $\tau^{\mathcal{G}}$ be a tensor network of non-negative tensors decorating a hypergraph $\mathcal{G}$. Then the Markov network $\mathbb{P}^{\mathcal{G}}$ to $\tau^{\mathcal{G}}$ is the probability distribution of $X_{\mathcal{V}}$ defined by the tensor*

$$\mathbb{P}^{\mathcal{G}}[X_{\mathcal{V}}] = \frac{\langle \{\tau^e : e \in \mathcal{E}\} \rangle_{[X_{\mathcal{V}}]}}{\langle \{\tau^e : e \in \mathcal{E}\} \rangle_{[\emptyset]}} = \langle \tau^{\mathcal{G}} \rangle_{[X_{\mathcal{V}}|\emptyset]}.$$

We call the denominator

$$\mathcal{Z}(\tau^{\mathcal{G}}) = \langle \{\tau^e : e \in \mathcal{E}\} \rangle_{[\emptyset]}$$

the partition function of the tensor network $\tau^{\mathcal{G}}$.

We define graphical models based on hypergraphs to establish a direct connection with tensor networks decorating the hypergraph. In a more canonical way, Markov networks are instead defined by graphs, where instead of the edges the cliques are decorated by factor tensors (see for example Koller and Friedman (2009)). Following this alternative description, the graphs of the tensor networks are dual to the graphs of the graphical models Robeva and Seigal (2019); Glasser et al. (2019).

We can interpret the factors $\tau[X_{[d]}]$ as activation cores placed on the hyperedges e of the graph. The global activation tensor (and hence the joint distribution) is obtained by contracting this activation network and normalizing by its partition function.

While so far we have defined Markov networks as decomposed probability distributions, we now want to derive assumptions on a distribution, assuring that such decompositions exist. The sets of conditional independencies encoded by a hypergraph are captured by its separation properties, as we define next.

Definition 13 (Separation of hypergraph). *A path in a hypergraph is a sequence of nodes v_k for $k \in [d]$, such that for any $k \in [d-1]$ we find a hyperedge $e \in \mathcal{E}$ such that $(v_k, v_{k+1}) \subset e$. Given disjoint subsets A, B, C of nodes in a hypergraph $\mathcal{G}$, we say that C separates A and B with respect to $\mathcal{G}$ when any path starting at a node in A and ending in a node in B contains a node in C .*

To characterize Markov networks in terms of conditional independencies, we need to further define the property of clique-capturing. This property establishes a correspondence of hyperedges with maximal cliques in the more canonical graph-based definition of Markov networks Koller and Friedman (2009).

Definition 14 (Clique-capturing hypergraph). *We call a hypergraph $\mathcal{G}$ clique-capturing, if the following holds: Each subset $\mathcal{U} \subset \mathcal{V}$, which fulfills that for any $a, b \in \mathcal{U}$ with $a \neq b$, there is a hyperedge $e \in \mathcal{E}$ with $a, b \in e$, is contained in a hyperedge.*

We are now ready to state the Hammersley-Clifford theorem characterizing the sets of Markov networks on a hypergraph by conditional independence.

Theorem 1 (Hammersley-Clifford factorization theorem). *Let there be a positive probability distribution $\mathbb{P}[X_{\mathcal{V}}]$ and a clique-capturing hypergraph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$. Then the following are equivalent:*

- i *The distribution $\mathbb{P}[X_{\mathcal{V}}]$ is representable by a Markov network on $\mathcal{G}$, that is for each edge $e \in \mathcal{E}$ there is a tensor $\tau^e[X_e]$ such that*

$$\mathbb{P}[X_{\mathcal{V}}] = \langle \{\tau^e[X_e] : e \in \mathcal{E}\} \rangle_{[X_{\mathcal{V}}|\emptyset]}.$$

- ii *For any subsets $A, B, C \subset \mathcal{V}$ such that C separates A from B , we have*

$$(X_A \perp X_B) \mid X_C.$$

Proof. This is shown in Appendix Sect. A. □

By Thm. 1 the conditional independence structure of $\mathbb{P}[X_{\mathcal{V}}]$ determines a global tensor network decomposition of $\mathbb{P}[X_{\mathcal{V}}]$. Note that the assumption of a positive distribution is required (i.e. for all $x_{[d]}$ we have $\mathbb{P}[X_{[d]} = x_{[d]}] > 0$). The assumption of positivity is however not required in our characterization of independencies and conditional independencies by the existence of corresponding tensor decompositions (see Def. 10 and Def. 11).

Example 6 (Independent boolean variables). *Let there be d boolean variables $X_{[d]}$, which are i.i.d. drawn from a positive distribution $\mathbb{P}[X]$. From the pairwise independencies of X_k it follows with the Hammersley-Clifford Factorization Thm. 1 that the distribution is representable by an elementary tensor network, that is*

$$\mathbb{P}[X_{[d]}] = \bigotimes_{k \in [d]} \mathbb{P}^k[X_k].$$

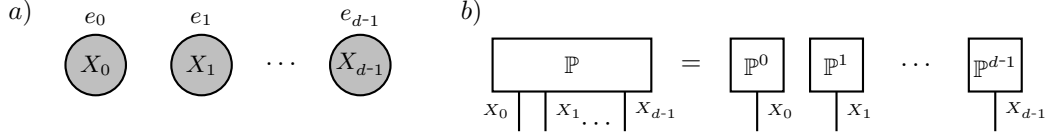


Figure 5: Decomposition of a probability distribution with independent variables (see Example 6). The independencies are captured by the elementary hypergraph a), whose edges contain single nodes. The corresponding tensor $\mathbb{P}[X_{[d]}]$ is then represented by a Markov network on the elementary hypergraph, where each factor is the marginal distribution of the corresponding variable as visualized in b).

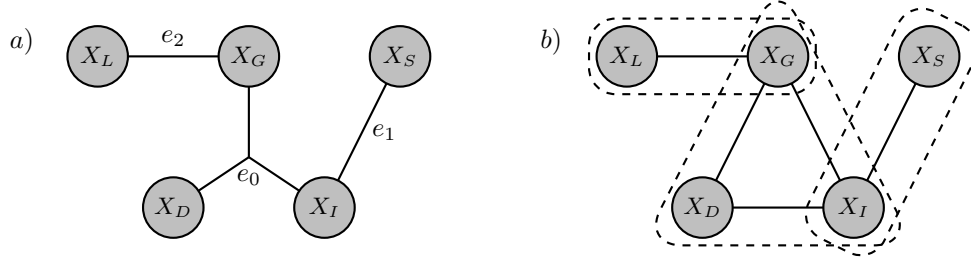


Figure 6: Hypergraph a) capturing the conditional independencies of the student example. The cliques of the node adjacency graph are highlighted in b) and coincide with hyperedges of the hypergraph. The hypergraph is therefore clique-capturing (see Def. 14).

The corresponding hypergraph is the elementary graph, with respect to which any two disjoint subsets of nodes are separated (see Figure 5).

Example 7. We consider a classical example of a graphical model (see (Koller and Friedman, 2009, Example 4.3)): A student of intelligence (X_I) and SAT score (X_S) is assigned a test of difficulty (X_D), for which he gets a grade (X_G) depending on which he gets a recommendation letter (X_L) by its teacher. We make the following modelling assumptions:

- "The SAT score depends only on the students intelligence": $(X_S \perp X_{\{D,G,L\}}) \mid X_I$.
- "The recommendation letter depends only on the grade": $(X_L \perp X_{\{D,I,S\}}) \mid X_G$.

The associated hypergraph capturing these conditional independencies is drawn in Figure 6 a).

3.3 Factorization based on sufficient statistics

Let us now introduce sufficient statistics towards studying further tensor network decompositions of probability distributions.

Definition 15. Let $\mathbb{P}[X, Z]$ be a joint distribution of the m -dimensional variable X and the n -dimensional variable Z and let

$$t : [m] \rightarrow [n]$$

be a statistic. We are interested in the distribution $\mathbb{P}[X, Z, Y_t] = \langle \mathbb{P}[X, Z], \beta^t[Y_t, X] \rangle_{[X, Z, Y_t]}$. We say that t is a sufficient statistic for Z if X is independent of Z conditioned on Y_t .

Example 8 (Sufficient statistics for the probability). Let Z be the value $\mathbb{P}[X_{[d]} = x_{[d]}]$ when drawing $X_{[d]}$ from $\mathbb{P}[X_{[d]}]$. Then t is a sufficient statistic for $Z = \mathbb{P}[X_{[d]}]$ if for all y in the image of t we have

$$\mathbb{P}[X_{[d]} = x_{[d]} | Y_t = y] = \begin{cases} \frac{1}{|\{x_{[d]} : t(x_{[d]}) = y\}|} & \text{if } t(x_{[d]}) = y \\ 0 & \text{else} \end{cases}.$$

When knowing the value $t(x_{[d]})$ of the sufficient statistic at a given index $x_{[d]}$, we then also know the probability $\mathbb{P}[X_{[d]} = x_{[d]}]$. The function t is thus a sufficient statistic for $Z = \mathbb{P}[X_{[d]}]$ if and only if there is a tensor $\xi[Y_{[p]}]$ with

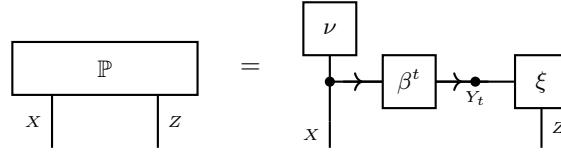
$$\mathbb{P}[X_{[d]}] = \langle \beta^t[Y_{[p]}, X_{[d]}], \xi[Y_{[p]}] \rangle_{[X_{[d]}]} .$$

Example 8 hints at a connection between sufficient statistics and decompositions into CompActNets. More generally, such decompositions are provided by the Fisher-Neyman Factorization Theorem.

Theorem 2 (Fisher-Neyman Factorization Theorem). *Let $\mathbb{P}$ be a joint distribution of variables X, Z with values $\text{val}(X), \text{val}(Z)$. Let there further be a finite set $\text{val}(Y_t)$, which is enumerated with a variable Y_t . Then $t : \text{val}(X) \rightarrow \text{val}(Y_t)$ is a sufficient statistic for Z if and only if there are tensors $\nu[X]$ and $\xi[Y_t, Z]$ such that*

$$\mathbb{P}[X, Z] = \langle \xi[Y_t, Z], \beta^t[Y_t, X], \nu[X] \rangle_{[X, Z]} .$$

We depict this equation diagrammatically by



Proof. Shown in the appendix, see Thm. 11. □

Note that the definition of sufficient statistic does not make use of the marginal distribution $\mathbb{P}[Z]$. We therefore can define sufficient statistics also for families of distributions $\mathbb{P}[X|Z]$ with respect to arbitrary non-degenerate marginal distributions $\mathbb{P}[Z]$. We then use Thm. 2 to embed such families in CompActNets.

Corollary 2. *Let $\mathbb{P}[X_{[d]}|Z]$ be an arbitrary family of distributions of $X_{[d]}$ and t a sufficient statistic for Z . Then, there is a tensor $\nu[X_{[d]}]$ and an activation tensor $\xi[Y_{[p]}, Z]$ such that for any $z \in \text{val}(Z)$ we have*

$$\mathbb{P}[X_{[d]}|Z = z] \in \Lambda^{t, \text{MAX}, \xi} .$$

The Factorization Theorem of Fisher-Neyman provides the fundamental motivation for the CompActNets architecture. Any decomposition of $\beta^t[Y_{[p]}, X_{[d]}]$ is called a *computation network* and common to all members of a family with sufficient statistic t . The activation tensor $\xi[Y_{[p]}, Z = z]$, whose decomposition is called the *activation network*, is specific to each member of the family. We now show in two examples how families of distributions can be represented in CompActNets by sufficient statistics.

Example 9 (Order statistic for boolean variables). *Let there be d boolean variables $X_{[d]}$ and a family $\mathbb{P}[X_{[d]}|Z]$ of distributions. The order statistic assigns the ordered tuple to each tuple $x_{[d]}$. The ordered tuple effectively counts the number of 1 coordinates in the tuple $x_{[d]}$, that is the statistic*

$$t^+ : \bigtimes_{k \in [d]} [m_k] \rightarrow [p], \quad t^+(x_{[d]}) = |\{k : x_k = 1\}| .$$

When the order statistic is sufficient for Z , the detailed order of the outcomes is uninformative about the member z from which the random variables have been drawn. By the Fisher-Neyman Factorization Thm. 2, t^+ is a sufficient statistic if and only if there are tensors $\nu[X_{[d]}]$ and $\xi[Y_+, \Theta]$ such that for each $\theta \in \Theta$

$$\mathbb{P}[X_{[d]}|Z = z] = \langle \xi[Y_+, Z = z], \beta^{t^+}[Y_+, X_{[d]}], \nu[X_{[d]}] \rangle_{[X_{[d]}]} .$$

For each member z of the family, the probability of each sequence $x_{[d]}$ is thus the product of a base measure factor $\nu[X_{[d]} = x_{[d]}]$ and a factor $\xi[Y_+ = +(x_{[d]}), Z = z]$ depending only on the count $+(x_{[d]})$ of 1 coordinates in $x_{[d]}$. We later continue this example in Example 11, where further interpretations to the case of i.i.d. variables are provided.

Example 10 (Graphical models as a special case of CompActNets). *For graphical models we take the identity statistic*

$$\delta(x_{[d]}) = x_{[d]}$$

so that the image coordinates coincide with the variables and there are no non-trivial computation cores. The associated basis encoding is just the identity tensor

$$\beta^\delta [Y_{[d]}, X_{[d]}] = \delta [X_{[d]}, Y_{[d]}] .$$

Therefore, for any activation tensor $\xi [Y_{[p]}]$ we obtain

$$\mathbb{P} [X_{[d]}] = \langle \xi [Y_{[p]}], \beta^\delta [Y_{[d]}, X_{[d]}] \rangle_{[X_{[d]}|\emptyset]} = \langle \xi [X_{[d]}] \rangle_{[X_{[d]}|\emptyset]} .$$

Put differently, in the graphical-model case the activation tensor coincides with the joint distribution tensor. In this setting, structural properties of the distribution such as (conditional) independences can be read off as algebraic factorization patterns of the activation (and hence joint) tensor.

3.4 Exponential families

We now show that exponential families are specific instances of CompActNets, whose activation tensors have elementary decompositions. The importance of exponential families in statistics stems from their universal properties. A classical theorem by Pitman, Koopman and Darmois (see Pitman (1936)) states, that whenever a family exhibits constant support and a finite sufficient statistic for arbitrary large data sets, then it is in an exponential family. For a discussion of further universal properties of exponential families such as the existence of priors and entropy maximizers, see Murphy (2022).

Definition 16 (Exponential family). *Given a base measure ν and a statistic*

$$t : \bigtimes_{k \in [d]} [m_k] \rightarrow \mathbb{R}^p$$

we enumerate for each coordinate $\ell \in [p]$ the image $\text{im}(t_\ell)$ by an interpretation map

$$I_\ell : [|\text{im}(t_\ell)|] \rightarrow \text{im}(t_\ell) .$$

For any canonical parameter vector $\theta [L] \in \mathbb{R}^p$, we build the activation cores $\alpha^{\ell, \theta} [Y_\ell]$ for each coordinate $y_\ell \in [|\text{im}(t_\ell)|]$ by

$$\alpha^{\ell, \theta} [Y_\ell = y_\ell] = \exp [\theta [L = \ell] \cdot I_\ell(y_\ell)]$$

and define the distribution

$$\mathbb{P}^{(t, \theta, \nu)} [X_{[d]}] = \langle \{ \nu [X_{[d]}] \} \cup \{ \beta^{t_\ell} [Y_\ell, X_{[d]}] : \ell \in [p] \} \cup \{ \alpha^{\ell, \theta} [Y_\ell] : \ell \in [p] \} \rangle_{[X_{[d]}|\emptyset]} .$$

We then call the tensor $\mathbb{P}^{t, \nu} [X_{[d]}|\Theta]$ with $\text{val}(\Theta) = \mathbb{R}^p$ and slices for $\theta \in \mathbb{R}^p$ given by

$$\mathbb{P}^{t, \nu} [X_{[d]}|\Theta = \theta] = \mathbb{P}^{(t, \theta, \nu)} [X_{[d]}]$$

the exponential family to the statistic t and the base measure ν .

To see that Def. 16 is consistent with the typical definition of exponential families (see Brown (1987)), note that for each $\theta \in \mathbb{R}^p$ the normalization amounts to the division by

$$\mathcal{Z}(\theta) = \langle \{ \nu [X_{[d]}] \} \cup \{ \beta^{t_\ell} [Y_\ell, X_{[d]}] : \ell \in [p] \} \cup \{ \alpha^{\ell, \theta} [Y_\ell] : \ell \in [p] \} \rangle_{[\emptyset]} ,$$

a quantity which is referred to as the partition function. Then, we have for each coordinate $x_{[d]} \in \bigtimes_{k \in [d]} [m_k]$ that

$$\begin{aligned} \mathbb{P}^{(t, \theta, \nu)} [X_{[d]} = x_{[d]}] &= \frac{1}{\mathcal{Z}(\theta)} \langle \{ \nu [X_{[d]}] \} \cup \{ \beta^{t_\ell} [Y_\ell, X_{[d]}] : \ell \in [p] \} \cup \{ \alpha^{\ell, \theta} [Y_\ell] : \ell \in [p] \} \rangle_{[X_{[d]}=x_{[d]}]} \\ &= \frac{1}{\mathcal{Z}(\theta)} \cdot \nu [X_{[d]} = x_{[d]}] \cdot \exp \left[\sum_{\ell \in [p]} \theta [L = \ell] \cdot t_\ell [X_{[d]} = x_{[d]}] \right] . \end{aligned}$$

Note that by construction each member of an exponential family is an element in a CompActNet with elementary activation cores, that is

$$\mathbb{P}^{t, \nu} [X_{[d]}|\Theta = \theta] \in \Lambda^{t, \text{EL}, \nu} .$$

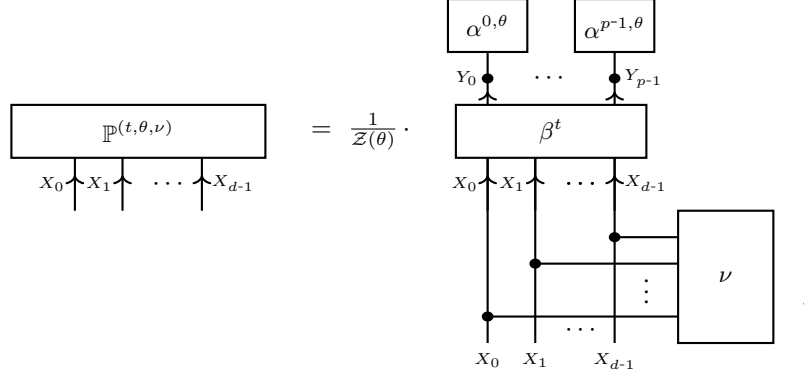


Figure 7: Tensor Network diagram of a member of an exponential family $\mathbb{P}^{t, \nu} [X_{[d]} | \Theta = \theta]$ before normalization as an CompActNet with elementary activation, that is an element in $\Lambda^{t, \text{EL}, \nu}$.

Example 11 (Exponential family of coin tosses). Recall the family of distributions of boolean $X_{[d]}$ from Example 9, which has the order statistic t^+ as a sufficient statistic. We now in addition assume that the variables $X_{[d]}$ are i.i.d. with respect to any member of the family (see Example 9). For the variables to be i.i.d., we need $\nu [X_{[d]}] = \mathbb{I} [X_{[d]}]$ and can thus choose a representation such that for $z \in \text{val}(Z)$

$$\mathbb{P}[X_{[d]} | Z = z] = \left\langle \beta^{t^+} [Y_+, X_{[d]}], \xi [Y_+, Z = z] \right\rangle_{[X_{[d]}]}$$

where for each $k \in [d + 1]$

$$\xi [Y_+ = k, Z = z] = (1 - z)^{d-k} \cdot z^k.$$

The marginal distribution $\mathbb{P}[Y_+ | Z = z]$ is then the binominal distribution $B(d, z)$. When excluding the case of $z \in \{0, 1\}$, the family is a subset of the exponential family with the head count statistic, where each member is reparametrized by

$$\theta := \ln \left[\frac{z}{1 - z} \right].$$

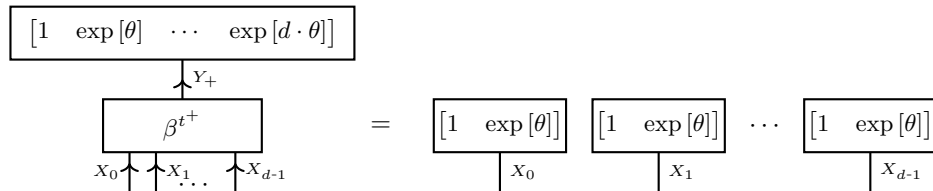
To see that this is true, we observe that the coordinate $y_+ \in [d + 1]$ of the activation tensor of $\mathbb{P}^{(t^+, \theta, \mathbb{I})}$ is

$$\alpha^\theta [Y_+ = y_+] = \exp [y_+ \cdot \theta] = \frac{z^{y_+}}{(1 - z)^{y_+}}.$$

Now, with $Z(\theta) = \frac{1}{(1-z)^d}$ we have for any $x_{[d]}$ with $\sum_{k \in [d]} x_k = y_+$ that

$$\frac{1}{Z(\theta)} \cdot \left\langle \beta^{t^+} [Y_+, X_{[d]} = x_{[d]}], \alpha^\theta [Y_+] \right\rangle_{[\emptyset]} = (1 - z)^d \cdot \frac{z^{y_+}}{(1 - z)^{y_+}} = z^{y_+} \cdot (1 - z)^{d - y_+}.$$

Comparing with the activation tensor $\xi [Y_+]$ above, we note that $Z(\theta)$ is the partition function of the exponential family and $\mathbb{P}^{(t^+, \theta, \mathbb{I})} [X_{[d]}]$ coincides with the member $\mathbb{P}[X_{[d]} | Z = z]$. We further observe that since the statistic t^+ decomposes as a sum of terms depending on single variables only, we have a decomposition of the corresponding CompActNet by



This reproduces the fact that distributions of independent variables are representable by elementary tensors (see Example 6).

3.5 Efficient contractions by message passing

Contractions of tensor networks are generally hard to solve. Here, we investigate message passing algorithms, which decompose global contractions into a sequence of local contractions, whose results are passed as messages through the tensor network. The resulting algorithm is called the Tree Belief Propagation (see Algorithm 1). While various scheduling strategies for the message passing exist, we focus on the case of tree hypergraphs, for which exactness and efficiency can be shown. We denote $\mathcal{E}^{\rightarrow}$ to be all tuples (e_0, e_1) of hyperedges $e_0, e_1 \in \mathcal{E}$ such that $e_0 \neq e_1$ and $e_0 \cap e_1 \neq \emptyset$. For our purposes we call a hypergraph $\mathcal{G}$ a tree when the graph with nodes by the hyperedges $\mathcal{E}$ and edges by $\mathcal{E}^{\rightarrow}$ is a tree (for an example see Figure 8).

Algorithm 1 Tree Belief Propagation

Require: Tensor network $\tau^{\mathcal{G}}$ on a hypergraph $\mathcal{G}$

Ensure: Messages $\{\chi_{e_0 \rightarrow e_1} [X_{e_0 \cap e_1}] : (e_0, e_1) \in \mathcal{E}^{\rightarrow}\}$

Initialize a message scheduler $S = \{(e_0, e_1) \in \mathcal{E}^{\rightarrow} : e_0 \text{ a leaf in the tree } (\mathcal{E}, \mathcal{E}^{\rightarrow})\}$

while S not empty **do**

 Pop a (e_0, e_1) pair from S

 Compute the message

$$\chi_{e_0 \rightarrow e_1} [X_{e_0 \cap e_1}] = \langle \{\tau^{e_0} [X_{e_0}]\} \cup \{\chi_{e_2 \rightarrow e_0} [X_{e_2 \cap e_0}] : (e_2, e_0) \in \mathcal{E}^{\rightarrow}, e_2 \neq e_1\} \rangle_{[X_{e_0 \cap e_1}]}$$

 Update S by all messages (e_1, e_3) which have not yet been sent, if all messages (e_2, e_1) with $e_2 \neq e_3$ have been sent.

end while

return Messages $\{\chi_{e_0 \rightarrow e_1} [X_{e_0 \cap e_1}] : (e_0, e_1) \in \mathcal{E}^{\rightarrow}\}$

For an implementation of Algorithm 1 in the python package `tnreason`, see Sect. B.1.

The following theorem states that the contraction of a whole tensor network can be replaced by local contractions with messages. Since contracting the whole network can be infeasible, this shows that calculating the messages with the Algorithm 1 can be advantageous.

Theorem 3. *Let $\tau^{\mathcal{G}}$ be a tensor network on a tree hypergraph $\mathcal{G}$ (i.e. the graph $(\mathcal{E}, \mathcal{E}^{\rightarrow})$ is a tree). The messages in the tree belief propagation Algorithm 1 are contracted to local marginals, meaning that for each $e_0 \in \mathcal{E}$ we have*

$$\langle \tau^{\mathcal{G}} \rangle_{[X_{e_0}]} = \langle \{\tau^{e_0} [X_{e_0}]\} \cup \{\chi_{e_2 \rightarrow e_0} [X_{e_2 \cap e_0}] : (e_2, e_0) \in \mathcal{E}^{\rightarrow}\} \rangle_{[X_{e_0}]}.$$

We show Thm. 3 based on the following lemma. We denote for each pair (e_0, e_1) the subset $\mathcal{E}^{\rightarrow(e_0, e_1)} \subset \mathcal{E}$ as the subset of edges $e \in \mathcal{E}$, for which each path in $(\mathcal{E}, \mathcal{E}^{\rightarrow})$ to e_1 passes through e_0 . The tree hypergraph property makes this definition equivalent to an existing path through e_0 , which is used in the proof of the following lemma. Note that by construction $e_0 \in \mathcal{E}^{\rightarrow(e_0, e_1)}$.

Lemma 1. *For any tensor network on a tree hypergraph, Algorithm 1 terminates in the tree-based implementation and returns final messages*

$$\chi_{e_0 \rightarrow e_1} [X_{e_0 \cap e_1}] = \left\langle \{\tau^e [X_e] : e \in \mathcal{E}^{\rightarrow(e_0, e_1)}\} \right\rangle_{[X_{e_0 \cap e_1}]}.$$

Proof. We show this property by induction over the size of the edge sets $\mathcal{E}^{\rightarrow(e_0, e_1)}$ for pairs $(e_0, e_1) \in \mathcal{E}^{\rightarrow}$, such that $|\mathcal{E}^{\rightarrow(e_0, e_1)}| \leq n$. Note that since always $e_0 \in \mathcal{E}^{\rightarrow(e_0, e_1)}$ we have that $n \geq 1$.

$n = 1$: In this case we have $\mathcal{E}^{\rightarrow(e_0, e_1)} = \{e_0\}$ and e_0 is a leaf of the tree-hypergraph $\mathcal{G}$. The claimed message property holds thus by definition.

$n \rightarrow n + 1$: Assume that the message obeys the claimed property for edge sets with cardinality up to n . If there is no edge set with cardinality $n + 1$, the property holds also for those with cardinality up to $n + 1$. If there is an edge set $\mathcal{E}^{\rightarrow(e_0, e_1)}$ with size $n + 1$, we have

$$\mathcal{E}^{\rightarrow(e_0, e_1)} = \{e_0\} \cup \left(\bigcup_{e_2 \in \mathcal{E}^{\rightarrow}} \mathcal{E}^{\rightarrow(e_2, e_0)} \right).$$

The message $\chi_{e_0 \rightarrow e_1}$ is sent once all messages $\chi_{e_2 \rightarrow e_0}$ to $(e_2, e_0) \in \mathcal{E}^{\rightarrow(e_0, e_1)}$ arrived. By definition we have that

$$\chi_{e_0 \rightarrow e_1} [X_{e_0 \cap e_1}] = \langle \{\tau^{e_0} [X_{e_0}]\} \cup \{\chi_{e_2 \rightarrow e_0} [X_{e_2 \cap e_0}] : (e_2, e_0) \in \mathcal{E}^{\rightarrow}, e_2 \neq e_1\} \rangle_{[X_{e_0 \cap e_1}]} .$$

Now we use the induction assumption on $\mathcal{E}^{\rightarrow(e_2, e_0)}$ (since its cardinality is at most n) and get

$$\begin{aligned} \chi_{e_0 \rightarrow e_1} [X_{e_0 \cap e_1}] &= \left\langle \{\tau^{e_0} [X_{e_0}]\} \cup \left(\bigcup_{(e_2, e_0) \in \mathcal{E}^{\rightarrow}, e_2 \neq e_1} \left\langle \{\tau^{e_3} [X_{e_3}] : e_3 \in \mathcal{E}^{\rightarrow(e_2, e_0)}\} \right\rangle_{[X_{e_2 \cap e_0}]} \right) \right\rangle_{[X_{e_0 \cap e_1}]} \\ &= \left\langle \{\tau^{e_0} [X_{e_0}]\} \cup \left(\bigcup_{(e_2, e_0) \in \mathcal{E}^{\rightarrow}, e_2 \neq e_1} \{\tau^{e_3} [X_{e_3}] : e_3 \in \mathcal{E}^{\rightarrow(e_2, e_0)}\} \right) \right\rangle_{[X_{e_0 \cap e_1}]} \\ &= \left\langle \{\tau^e [X_e] : e \in \mathcal{E}^{\rightarrow(e_0, e_1)}\} \right\rangle_{[X_{e_0 \cap e_1}]} . \end{aligned}$$

Here, we used the commutation of contraction property in the second equation, which is justified by the assumed tree property of the hypergraph. Thus, the message property holds also for any edge sets of size $n + 1$.

By induction, the claimed message property therefore holds for all final messages. $\square$

Proof of Thm. 3. Since the hypergraph is by assumption a tree, we can partition $\mathcal{E}$ into disjoint subsets $\{e_0\}$ and $\mathcal{E}^{\rightarrow(e_2, e_0)}$ for $(e_2, e_0) \in \mathcal{E}^{\rightarrow}$. We then have

$$\begin{aligned} \langle \tau^G \rangle_{[X_{e_0}]} &= \left\langle \{\tau^{e_0} [X_{e_0}]\} \cup \left\{ \left\langle \tau^e [X_e] : e \in \mathcal{E}^{\rightarrow(e_2, e_0)} \right\rangle_{[X_{e \cap e_2}]} : (e_2, e_0) \in \mathcal{E}^{\rightarrow} \right\} \right\rangle_{[X_{e_0}]} \\ &= \langle \{\tau^{e_0} [X_{e_0}]\} \cup \{\chi_{e_2 \rightarrow e_0} [X_{e_2 \cap e_0}] : (e_2, e_0) \in \mathcal{E}^{\rightarrow}\} \rangle_{[X_{e_0}]} , \end{aligned}$$

where we used Lem. 1 in the second equation. $\square$

We illustrate the usage of Algorithm 1 on the Markov network of Example 7.

Example 12 (Continuation of Example 7). *We exemplify the Belief Propagation Algorithm 1 on the Markov network in the student example (see Example 7). The directions of the messages result from the hyperedge overlaps (see Figure 8 a) and the resulting directions $\mathcal{E}^{\rightarrow}$ are sketched in Figure 8 b). The messages to $\{(e_2, e_0), (e_0, e_2)\}$ are vectors of X_G and the messages $\{(e_0, e_1), (e_1, e_0)\}$ are vectors of X_I .*

Since the hyperedges are minimally connected, we can implement Algorithm 1 by a tree scheduler S :

- *The scheduler is initialized with messages from leafs, in our example $\{(e_2, e_0), (e_1, e_0)\}$.*
- *Each message is placed exactly once on S , when at a hyperedge all but the reverse message have been received. In our example, after execution of (e_2, e_0) the message (e_0, e_1) is placed on S and after execution of (e_1, e_0) the message (e_0, e_2) .*

In this implementation, Algorithm 1 terminates after $|\mathcal{E}^{\rightarrow}| = 4$ iterations of the while loop. The exact marginals of the edge variables are then

$$\begin{aligned} \mathbb{P} [X_L, X_G] &= \langle \tau^{e_2} [X_L, X_G], \chi_{e_0 \rightarrow e_2} [X_G] \rangle_{[X_L, X_G | \emptyset]} , \\ \mathbb{P} [X_G, X_D, X_I] &= \langle \tau^{e_0} [X_G, X_D, X_I], \chi_{e_2 \rightarrow e_0} [X_G], \chi_{e_1 \rightarrow e_0} [X_I] \rangle_{[X_G, X_D, X_I | \emptyset]} , \\ \mathbb{P} [X_I, X_S] &= \langle \tau^{e_1} [X_I, X_S], \chi_{e_0 \rightarrow e_1} [X_I] \rangle_{[X_I, X_S | \emptyset]} . \end{aligned}$$

4 The neural paradigm

The neural paradigm of artificial intelligence exploits the decomposition of functions into neurons, which are aligned in a directed acyclic graph. We show in this section how functions decomposable into neurons can be represented by tensor networks. To this end, we formalize discrete neural models by decomposition graphs and formally prove the corresponding decomposition of their basis encodings.

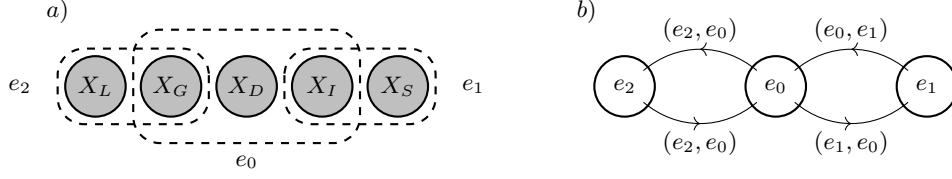


Figure 8: a) Sketch of the overlap of the edges, resulting in the message directions b) $\mathcal{E}^{\rightarrow} = \{(e_2, e_0), (e_0, e_2), (e_0, e_1), (e_1, e_0)\}$.

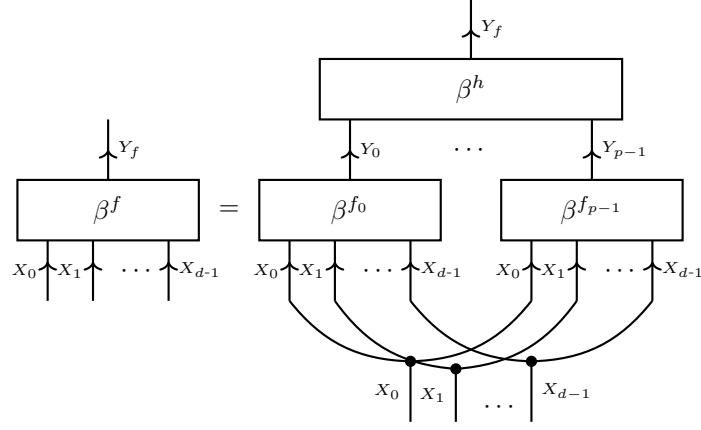


Figure 9: Tensor network decomposition of the basis encoding of a function f , which is the composition of the functions $f_0, \dots, f_{p-1}$ with a function h .

4.1 Function decomposition

As a main principle of tensor decompositions, we now show that basis encodings of composition functions are contractions of the basis encodings of their components.

Lemma 2. *Let $f[X_{[d]}]$ be a composition of a p -ary connective function h and functions $f_\ell[X_{[d]}]$, where $\ell \in [p]$, that is for $x_{[d]} \in \times_{k \in [d]}[2]$ we have*

$$f(x_{[d]}) = h(f_0[x_{[d]}], \dots, f_{p-1}[x_{[d]}]) .$$

Then, we have (see Figure 9)

$$\beta^f[Y_f, X_{[d]}] = \langle \{\beta^h[Y_f, Y_{[p]}]\} \cup \{\beta^{f_\ell}[Y_\ell, X_{[d]}] : \ell \in [p]\} \rangle_{[Y_f, X_{[d]}]} .$$

Proof. For any $x_{[d]} \in \times_{k \in [d]}[m_k]$ we have

$$\begin{aligned} & \langle \{\beta^h[Y_f, Y_{[p]}]\} \cup \{\beta^{f_\ell}[Y_\ell, X_{[d]}] : \ell \in [p]\} \rangle_{[Y_f, X_{[d]}=x_{[d]}]} \\ &= \langle \{\beta^h[Y_f, Y_{[p]}]\} \cup \{\beta^{f_\ell}[Y_\ell, X_{[d]}=x_{[d]}] : \ell \in [p]\} \rangle_{[Y_f]} \\ &= \langle \{\beta^h[Y_f, Y_{[p]}]\} \cup \{\epsilon_{f_\ell(x_{[d]})}[Y_\ell] : \ell \in [p]\} \rangle_{[Y_f]} \\ &= \epsilon_{f(x_{[d]})}[Y_f] \\ &= \beta^f[Y_f, X_{[d]}=x_{[d]}] . \end{aligned}$$

Thus, the tensors on both sides of the equation coincide in all slices to $X_{[d]}$ and are thus equal. $\square$

We now define a more generic decomposition of discrete functions.

Definition 17. A decomposition hypergraph is a directed acyclic hypergraph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ such that the following holds.

- Each node $v \in \mathcal{V}$ is decorated by a set $\mathcal{U}^v$ of finite cardinality m_v , a variable X_v , and an index interpretation function

$$I_v : [m_v] \rightarrow \mathcal{U}^v.$$

- Each directed hyperedge $(e^{\text{in}}, e^{\text{out}})$ has at least one outgoing node, that is $e^{\text{out}} \neq \emptyset$, and is decorated by an activation function

$$g^e : \bigtimes_{v \in e^{\text{in}}} \mathcal{U}^v \rightarrow \bigtimes_{v \in e^{\text{out}}} \mathcal{U}^v.$$

- Each node $v \in \mathcal{V}$ appears at most once as an outgoing node.
- The nodes not appearing as an outgoing node are enumerated by $v_{[d]}^{\text{in}}$. We abbreviate the corresponding variables by $X_{v_{[d]}^{\text{in}}} = X_{[d]}$.
- The nodes not appearing as an incoming node are enumerated by $v_{[p]}^{\text{out}}$. We abbreviate the variables by $X_{v_{[p]}^{\text{out}}} = Y_{[p]}$.

We assign for each $k \in [d]$ restriction functions

$$q_{v_k^{\text{in}}} : \bigtimes_{\tilde{k} \in [d]} \mathcal{U}^{v_k^{\text{in}}} \rightarrow \mathcal{U}^{v_k^{\text{in}}}, \quad x_{[d]}|_k = x_k$$

to the nodes $v_{[d]}^{\text{in}}$ and recursively assign to each further node v a node function

$$q_v : \bigtimes_{k \in [d]} \mathcal{U}^{v_k^{\text{in}}} \rightarrow \mathcal{U}^v, \quad q_v(x_{[d]}) = g^{e_v} \left(\bigtimes_{\tilde{v} \in e^{\text{in}}} q_{\tilde{v}}(x_{[d]}) \right)|_v,$$

where e_v is to each $v \in e^{\text{in}}$ the unique hyperedge with outgoing nodes $\{v\}$. We then call the function

$$q_{\mathcal{G}} : \bigtimes_{k \in [d]} \mathcal{U}^{v_k^{\text{in}}} \rightarrow \bigtimes_{\ell \in [p]} \mathcal{U}^{v_\ell^{\text{out}}}, \quad q_{\mathcal{G}} = \bigtimes_{\ell \in [p]} q_{v_\ell^{\text{out}}}$$

the composition function to the decomposition hypergraph $\mathcal{G}$.

The neural paradigm in AI can be modeled by the existence of decomposition hypergraphs for functions on large sets. We now show how decomposition hypergraphs enable the sparse representation of composition functions by tensor networks.

Theorem 4. For any decomposition hypergraph $\mathcal{G}$ with composition formula $q_{\mathcal{G}}$, we have

$$\beta^{q_{\mathcal{G}}} [Y_{[p]}, X_{[d]}] = \left\langle \left\{ \beta^{g^e} [X_{e^{\text{out}}}, X_{e^{\text{in}}}] : e = (e^{\text{in}}, e^{\text{out}}) \in \mathcal{E} \right\} \right\rangle_{[Y_{[p]}, X_{[d]}]}.$$

Proof. We show by induction over the number of edges in $\mathcal{G}$ that, for any $x_{[d]} \in \bigtimes_{k \in [d]} [m_k]$ and $v \in \mathcal{V}$, we have

$$\left\langle \left\{ \beta^{g^e} [X_{e^{\text{out}}}, X_{e^{\text{in}}}] : e = (e^{\text{in}}, e^{\text{out}}) \in \mathcal{E} \right\} \right\rangle_{[X_{\mathcal{V} \setminus v_{[d]}^{\text{in}}}, X_{[d]} = x_{[d]}]} = \bigotimes_{v \in \mathcal{V} \setminus v_{[d]}^{\text{in}}} \epsilon_{q_v(x_{[d]})} [X_v]. \quad (1)$$

$|\mathcal{E}| = 1$: If there is a single edge $e = (e^{\text{in}}, e^{\text{out}})$ in $\mathcal{E}$, we have $X_{[d]} = X_{e^{\text{in}}}$ and $\mathcal{V} \setminus v_{[d]}^{\text{in}} = e^{\text{out}}$. In this case (1) holds since

$$\beta^{g^e} [X_{e^{\text{out}}}, X_{e^{\text{in}}} = x_{e^{\text{in}}}] = \bigotimes_{v \in \mathcal{V} \setminus v_{[d]}^{\text{in}}} \epsilon_{q_v(x_{[d]})} [X_v].$$

$(|\mathcal{E}| = n) \Rightarrow (|\mathcal{E}| = n + 1)$: Let us now assume that (1) holds for all graphs with $|\mathcal{E}| \leq n$ and let $\mathcal{G}$ be a graph with $|\mathcal{E}| = n + 1$. We find an edge $\tilde{e} = (e^{\text{in}}, e^{\text{out}})$ such that all nodes in e^{in} are only appearing as outgoing nodes in other edges.

$$\begin{aligned}
& \left\langle \left\{ \beta^{g^e} [X_e] : e \in \mathcal{E} \right\} \right\rangle_{[X_{\mathcal{V} \setminus v_{[d]}^{\text{in}}}, X_{[d]} = x_{[d]}]} \\
&= \left\langle \beta^{\tilde{e}} [X_{e^{\text{out}}}, X_{e^{\text{in}}}], \left\langle \left\{ \beta^{g^e} [X_e] : e \in \mathcal{E} \setminus \{\tilde{e}\} \right\} \right\rangle_{[X_{\mathcal{V} \setminus \{v_{[d]}^{\text{in}} \cup e^{\text{out}}\}}, X_{[d]} = x_{[d]}]} \right\rangle_{[X_{\mathcal{V} \setminus v_{[d]}^{\text{in}}}, X_{[d]} = x_{[d]}]} \\
&= \left\langle \beta^{\tilde{e}} [X_{e^{\text{out}}}, X_{e^{\text{in}}}], \bigotimes_{v \in \mathcal{V} \setminus \{v_{[d]}^{\text{in}} \cup e^{\text{out}}\}} \epsilon_{q_v(x_{[d]})} [X_v] \right\rangle_{[X_{\mathcal{V} \setminus v_{[d]}^{\text{in}}}, X_{[d]} = x_{[d]}]} \\
&= \bigotimes_{v \in \mathcal{V} \setminus v_{[d]}^{\text{in}}} \epsilon_{q_v(x_{[d]})} [X_v] .
\end{aligned}$$

Here, in the second equation we used the induction hypothesis on the subgraph $(\mathcal{V}, \mathcal{E} \setminus \{\tilde{e}\})$ with $|\mathcal{E} \setminus \{\tilde{e}\}| = n$ and a generic contraction property of basis encodings in the third equation.

Thus, (1) holds always and we have for any $x_{[d]} \in \times_{k \in [d]} [m_k]$ that

$$\begin{aligned}
\left\langle \left\{ \beta^{g^e} [X_{e^{\text{out}}}, X_{e^{\text{in}}}] : e = (e^{\text{in}}, e^{\text{out}}) \in \mathcal{E} \right\} \right\rangle_{[Y_{[p]}, X_{[d]} = x_{[d]}]} &= \left\langle \bigotimes_{v \in \mathcal{V} \setminus v_{[d]}^{\text{in}}} \epsilon_{q_v(x_{[d]})} [X_v] \right\rangle_{[Y_{[p]}, X_{[d]}]} \\
&= \bigotimes_{v \in v_{[p]}^{\text{out}}} \epsilon_{q_v(x_{[d]})} [X_v] \\
&= \epsilon_{q_{\mathcal{G}}} [Y_{[p]}] \\
&= \beta^{q_{\mathcal{G}}} [Y_{[p]}, X_{[d]} = x_{[d]}] .
\end{aligned}$$

Keeping $X_{[d]}$ open, the claim is established. $\square$

When neurons have tunable parameters, we can discretize those by sets $\mathcal{U}^k$ and understand them as additional input variables.

Example 13 (Sum of integers in m -adic representation). *We develop a tensor network representation of integer summations on the set $[m^d] = \{0, \dots, m^d - 1\}$, where $m, d \in \mathbb{N}$,*

$$+ : [m^d] \times [m^d] \rightarrow [m^{d+1}] \quad , \quad +(i, \tilde{i}) = i + \tilde{i} ,$$

which have a m -adic representation of length d . We define an index interpretation map

$$I : \times_{k \in [d]} [m] \rightarrow [m^d] \quad , \quad I(x_{[d]}) = \sum_{k \in [d]} x_k \cdot m^k ,$$

which enables the parameterization of $[m^d]$ as the states of d categorical variables $X_{[d]}$ of dimension m . We analogously represent a second set $[m^d]$ by variables $\tilde{X}_{[d]}$ and the set $[m^{d+1}]$ of possible sums by $Y_{[d+1]}$. The basis encoding of the sum is then

$$\beta^+ [Y_{[d+1]}, X_{[d]}, Y_{[d]}] = \sum_{x_{[d]}, \tilde{x}_{[d]}} \epsilon_{I^{-1}(I(x_{[d]}) + I(\tilde{x}_{[d]}))} [Y_{[d+1]}] \otimes \epsilon_{x_{[d]}} [X_{[d]}] \otimes \epsilon_{\tilde{x}_{[d]}} [\tilde{X}_{[d]}] .$$

Note that the tensor space of β^+ is of dimension $m^{3 \cdot d + 1}$ increasing exponentially in d . Feasible representation of this tensor for large d therefore requires tensor network decompositions, which we now provide based on a decomposition hypergraph. The targeted function to be decomposed is the representation of the integer sum by

$$+^m : \left(\times_{k \in [d]} [m] \right) \times \left(\times_{k \in [d]} [m] \right) \rightarrow \times_{k \in [d+1]} [m] \quad , \quad +^m(x_{[d]}, \tilde{x}_{[d]}) = I^{-1}(I(x_{[d]}) + I(\tilde{x}_{[d]})) .$$

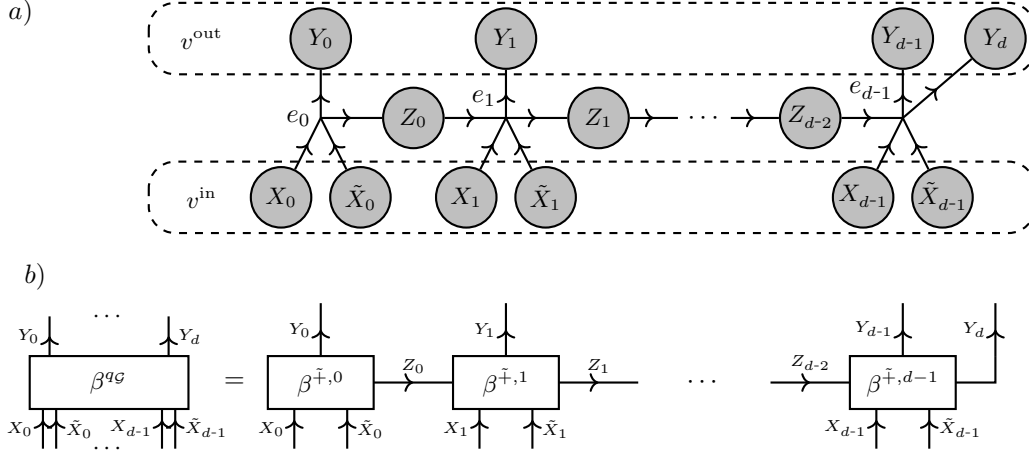


Figure 10: Example of a decomposition hypergraph to the sum of integers (see Example 13). a) Hypergraph of directed edges e_k for $k \in [d]$, each decorated by an integer summation $+$ preparing an index Y_k of the resulting sum. b) Corresponding tensor network decomposition of the basis encoded composition function, which is the sum of integers in m -adic representation.

We build a decomposition hypergraph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ (see Def. 17) consisting of $4 \cdot d$ nodes (see Figure 10a). The nodes carry the $(3 \cdot d + 1)$ variables $X_{[d]}, Y_{[d]}, Y_{[d+1]}$ of dimension m constructed above and $d - 1$ auxiliary variables $Z_{[d-1]}$ of dimension 2 representing carry bits. The directed hyperedges of $\mathcal{G}$ are

$$\mathcal{E} = \left\{ (\{X_0, \tilde{X}_0\}, \{Y_0, Z_0\}) \right\} \cup \left\{ (\{Z_{k-1}, X_k, \tilde{X}_k\}, \{Y_k, Z_k\}) : k \in \{1, \dots, d-2\} \right\} \\ \cup \left\{ (\{Z_{d-2}, X_{d-1}, \tilde{X}_{d-1}\}, \{Y_{d-1}, Y_d\}) \right\}$$

and are decorated by local summation functions

$$\tilde{+} : [2] \times [m] \times [m] \rightarrow [m] \times [2] \quad , \quad \tilde{+}(z, x, \tilde{x}) = \left((z + x + \tilde{x}) \bmod m, \left\lfloor \frac{z + x + \tilde{x}}{m} \right\rfloor \right) .$$

Since to the first hyperedge we do not have a carry bit, the decorating function is the restriction of the first argument to 0.

It is known that the composition of the local summations $\tilde{+}$ is the global summation $+^m$ of integers in m -adic representation. Thus, the composition function $q_{\mathcal{G}}$ is $+^m$. By Thm. 4 we have a decomposition of the basis encoding to $q_{\mathcal{G}}$ (see Figure 10b) as

$$\beta^{+^m} [Y_{[d+1]}, X_{[d]}, \tilde{X}_{[d]}] = \langle \{ \beta^{\tilde{+},0} [Y_0, Z_0, X_0, \tilde{X}_0] \} \cup \\ \{ \beta^{\tilde{+},k} [Y_k, Z_k, X_k, \tilde{X}_k, Z_{k-1}] : k \in \{1, \dots, d-2\} \} \cup \\ \{ \beta^{\tilde{+},d-2} [Y_{d-1}, Y_d, X_{d-1}, \tilde{X}_{d-1}, Z_{d-2}] \} \rangle [Y_{[d+1]}, X_{[d]}, \tilde{X}_{[d]}] .$$

4.2 Function evaluation by message passing

We are now concerned with an efficient inference algorithm based on tensor network contractions. To evaluate a function given as a tensor network decomposition of its basis encoding, the whole network has to be contracted. As this can be infeasible for large networks, a message passing algorithm based on local contractions can be applied, compare Algorithm 1 for a message passing algorithm for tensor networks on a on tree hypergraph.

We apply the Directed Belief Propagation, Algorithm Algorithm 2, on a decomposition hypergraph, where we add hyperedges to each leaf node and assign one-hot encodings of input states. We then show that the messages are the one-hot encodings to the evaluations of the node functions.

Algorithm 2 Directed Belief Propagation

Require: Tensor network $\tau^{\mathcal{G}}$ on a directed hypergraph $\mathcal{G}$ **Ensure:** Messages $\{\chi_{e_0 \rightarrow e_1} [X_{e_0 \cap e_1}] : (e_0, e_1) \in \mathcal{E}^{\rightarrow}\}$

Prepare directed message directions

$$\mathcal{E}^{\rightarrow} = \{((e_0^{\text{in}}, e_0^{\text{out}}), (e_1^{\text{in}}, e_1^{\text{out}})) : e_0^{\text{in}} \cap (e_1^{\text{in}}, e_1^{\text{out}}) = \emptyset, e_1^{\text{out}} \cap (e_0^{\text{in}}, e_0^{\text{out}}) = \emptyset, e_0^{\text{out}} \cap e_1^{\text{in}} \neq \emptyset\}$$

Initialize a message queue $S = \{(e_2, e_0) : e_2 \text{ has empty incoming nodes}\}$ **while** S not empty **do**Pop a (e_0, e_1) pair from S

Update the message

$$\chi_{e_0 \rightarrow e_1} [X_{e_0 \cap e_1}] = \langle \{\tau^{e_0} [X_{e_0}]\} \cup \{\chi_{e_2 \rightarrow e_0} [X_{e_2 \cap e_0}] : (e_2, e_0) \in \mathcal{E}^{\rightarrow}, e_2 \neq e_1\} \rangle_{[X_{e_0 \cap e_1}]}$$

Update S by all messages (e_1, e_3) which have not yet been sent, if all messages (e_2, e_1) have been sent.**end while****return** Messages $\{\chi_{e_0 \rightarrow e_1} [X_{e_0 \cap e_1}] : (e_2, e_0) \in \mathcal{E}^{\rightarrow}\}$

Theorem 5. Let $\mathcal{G}$ be a decomposition graph and let us add hyperedges containing single input nodes, which are decorated by one-hot encodings. Then the messages computed in Algorithm 2 are characterized by

$$\chi_{e_0 \rightarrow e_1} [X_{e_0 \cap e_1}] = \bigotimes_{v \in e_0 \cap e_1} \epsilon_{q_v(x_{[d]})} [X_v] .$$

Proof. We show the theorem inductively over the messages computed in Algorithm 2. The first message is sent from an input edge $\{[k]\}$ to an edge e of the decomposition graph and is by assumption the one-hot encoding of an input state $\epsilon_{x_k} [X_k]$.

We now assume that all previous messages satisfy the claimed equation at an arbitrary stage of the algorithm. The message computed in the while loop is then a contraction of one-hot encodings with basis encodings and

$$\begin{aligned} \chi_{e_0 \rightarrow e_1} [X_{e_0 \cap e_1}] &= \left\langle \{\beta^{g^{e_0}} [X_{e^{\text{out}}}, X_{e^{\text{in}}}] \} \cup \{\chi_{e_2 \rightarrow e_0} [X_{e_2 \cap e_0}] : (e_2, e_0) \in \mathcal{E}^{\rightarrow}\} \right\rangle_{[X_{e_0 \cap e_1}]} \\ &= \left\langle \{\beta^{g^{e_0}} [X_{e^{\text{out}}}, X_{e^{\text{in}}}] \} \cup \{\epsilon_{q_v(x_{[d]})} [X_v] : v \in e^{\text{in}}\} \right\rangle_{[X_{e_0 \cap e_1}]} \\ &= \bigotimes_{v \in e_0 \cap e_1} \epsilon_{q_v(x_{[d]})} [X_v] . \end{aligned}$$

Thus, the new message is also the tensor product of the one-hot encodings of the evaluated node functions. By induction, the property is therefore true for all messages. $\square$

We notice that we can interpret any directed acyclic hypergraph for which each node appears exactly once as an outgoing node and which is decorated by boolean and directed tensors $\tau^{\mathcal{G}}$. Edges with empty incoming sets are carrying one-hot encodings of input states and all further edges carry functions.

Example 14 (Continuation of Example 13). We now show how Algorithm 2 can be exploited to compute an efficient message passing algorithm for the digits of the m -adic sum. Given two numbers in m -adic representation by the tuples $x_{[d]}$ and $\tilde{x}_{[d]}$, we add the hyperedges with empty incoming nodes and single outgoing node

$$\bigcup_{k \in [d]} \{(\emptyset, \{X_k\}), (\emptyset, \{\tilde{X}_k\})\}$$

to the hypergraph of Example 13 and decorate them by the digit one-hot encodings $\epsilon_{x_k} [X_k]$ and $\epsilon_{\tilde{x}_k} [\tilde{X}_k]$ (see Figure 11). We then apply the Directed Belief Propagation Algorithm 2. The initial messages queue then consists of the messages from the digit encoding. As sketched in Figure 11, to each digit there are three messages (with the exception of the first being two), which can be scheduled in consecutive epochs $\chi^{(k, [3])}$. We then have by Thm. 5 for $k \in [d-1]$ that

$$\left\langle \beta^{\tilde{\cdot}, k} [Y_k, Z_k, X_k, \tilde{X}_k, Z_{k-1}] , \chi^{(k-1, 2)} [Z_{k-1}] , \chi^{(k, 0)} [X_k] , \chi^{(k, 1)} [\tilde{X}_k] \right\rangle_{[Z_k]} = \epsilon_{z_k} [Z_k] ,$$

In the state set $\times_{k \in [d]} [2] = \{0, 1\} \times \{0, 1\} \times \{0, 1\}$ we have three models of the formula by the positions of the non-zero entries in the tensor, that is $f[X_{[3]} = x_{[3]}] = 1$ if and only if

$$x_{[3]} \in \{(1, 0, 0), (0, 1, 0), (1, 1, 0)\}.$$

The formula f is therefore satisfiable.

Model counts by contraction Each coordinate of a propositional formula is either 1 or 0, indicating whether the indexed state is a model of the formula or not. In this way, the contraction $\langle f \rangle_{[\emptyset]}$ counts the number of models of the propositional formula f . One can therefore decide the satisfiability of a formula by testing for $\langle f \rangle_{[\emptyset]} > 0$.

CP decomposition We can decompose a formula into the sum of the one-hot encodings of its models:

$$\begin{array}{c} \boxed{f} \\ \begin{array}{c} X_0 \quad X_1 \quad \dots \quad X_{d-1} \end{array} \end{array} = \sum_{\substack{x_{[d]} \in \times_{k \in [d]} [m_k] \\ f[X_{[d]} = x_{[d]}] = 1}} \begin{array}{c} \boxed{\epsilon_{x_0}} \\ \downarrow X_0 \end{array} \dots \begin{array}{c} \boxed{\epsilon_{x_{d-1}}} \\ \downarrow X_{d-1} \end{array} = \begin{array}{c} \begin{array}{c} I \\ \swarrow \quad \downarrow \quad \searrow \end{array} \\ \begin{array}{c} \boxed{\tau^0} \quad \boxed{\tau^1} \quad \dots \quad \boxed{\tau^{d-1}} \\ \begin{array}{c} X_0 \quad X_1 \quad \dots \quad X_{d-1} \end{array} \end{array}$$

As already depicted, one can exploit this summation to find a CP decomposition of the formula. To this end, we enumerate the models $x_{[d]}^i$ of f by a decomposition variable I with values $i \in [\langle f \rangle_{[\emptyset]}]$ and define, for $k \in [d]$, cores with slices

$$\tau^k[X_k, I = i] = \epsilon_{x_k^i}[X_k].$$

Example 16. For the formula described in Example 15, we have

$$\begin{aligned} f[X_{[3]}] &= (\epsilon_1[X_0] \otimes \epsilon_0[X_1] \otimes \epsilon_0[X_2]) + (\epsilon_0[X_0] \otimes \epsilon_1[X_1] \otimes \epsilon_0[X_2]) \\ &\quad + (\epsilon_1[X_0] \otimes \epsilon_1[X_1] \otimes \epsilon_0[X_2]). \end{aligned}$$

Note that we have $\langle f[X_{[d]}] \rangle_{[\emptyset]} = 3$ and we can interpret this sum as a CP decomposition of f with rank 3. We use the decomposition to evaluate the formula f at $x_{[3]} = (1, 1, 0)$ and get

$$\begin{aligned} f[X_{[3]} = x_{[3]}] &= (\epsilon_1[X_0 = 1] \otimes \epsilon_0[X_1 = 1] \otimes \epsilon_0[X_2 = 0]) \\ &\quad + (\epsilon_0[X_0 = 1] \otimes \epsilon_1[X_1 = 1] \otimes \epsilon_0[X_2 = 0]) \\ &\quad + (\epsilon_1[X_0 = 1] \otimes \epsilon_1[X_1 = 1] \otimes \epsilon_0[X_2 = 0]) \\ &= 1 \cdot 0 \cdot 1 + 0 \cdot 1 \cdot 1 + 1 \cdot 1 \cdot 1 = 1, \end{aligned}$$

which verifies that $x_{[3]} = (1, 1, 0)$ is a model of the formula f .

Basis encoding Representing booleans by elements in $\{0, 1\}$ leads to the problem that the negation is an affine transformation and cannot be represented by multilinear tensors. To be able to express different kinds of connectives by contractions, booleans are encoded by one-hot encodings as defined in Def. 5. Propositional formulas f can then be expressed by their basis encodings

$$\beta^f[Y_f = y_f, X_{[d]} = x_{[d]}] = \begin{cases} 1 & \text{if } f[X_{[d]} = x_{[d]}] = y_f \\ 0 & \text{else} \end{cases}.$$

This basis encoding $\beta^f[Y_f, X_{[d]}] \in \{0, 1\}^{2 \times 2^d}$ encodes the formula itself and its negation in its slices, since

$$\beta^f[Y_f, X_{[d]}] = \epsilon_1[Y_f] \otimes f[X_{[d]}] + \epsilon_0[Y_f] \otimes \neg f[X_{[d]}].$$

In our graphical notation this property is visualized by

$$\begin{array}{c} \begin{array}{c} Y_f \\ \uparrow \\ \boxed{\beta^f} \\ \begin{array}{c} X_0 \quad X_1 \quad \dots \quad X_{d-1} \end{array} \end{array} = \sum_{x_{[d]} \in \times_{k \in [d]} [2]} \begin{array}{c} \begin{array}{c} Y_f \\ \uparrow \\ \boxed{\epsilon_f[X_{[d]} = x_{[d]}]} \\ \downarrow \end{array} \\ \begin{array}{c} \boxed{\epsilon_{x_{[d]}}} \\ \begin{array}{c} X_0 \quad X_1 \quad \dots \quad X_{d-1} \end{array} \end{array} = \begin{array}{c} \begin{array}{c} Y_f \\ \uparrow \\ \boxed{\epsilon_0} \\ \downarrow \end{array} \\ \begin{array}{c} \boxed{\neg f} \\ \begin{array}{c} X_0 \quad X_1 \quad \dots \quad X_{d-1} \end{array} \end{array} + \begin{array}{c} \begin{array}{c} Y_f \\ \uparrow \\ \boxed{\epsilon_1} \\ \downarrow \end{array} \\ \begin{array}{c} \boxed{f} \\ \begin{array}{c} X_0 \quad X_1 \quad \dots \quad X_{d-1} \end{array} \end{array} \end{array}$$

We further provide a more detailed example in coordinate sensitive notation in the following.

Example 17 (Logical negation and conjunction). *The basis encodings of the negation $\neg : [2] \rightarrow [2]$ is the matrix*

$$\beta^\neg [Y_\neg, X] = \begin{matrix} & \begin{matrix} \xrightarrow{Y_\neg} \\ 0 \quad 1 \end{matrix} \\ \begin{matrix} \downarrow \\ x_0 \end{matrix} \quad \begin{matrix} \downarrow \\ x_1 \end{matrix} \end{matrix} \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

The 2-ary conjunctions $\wedge : [2] \times [2] \rightarrow [2]$ is encoded by the order-3 tensor

$$\beta^\wedge [Y_\wedge, X_0, X_1] = \begin{matrix} & \begin{matrix} \xrightarrow{X_1} \\ 0 \quad 1 \end{matrix} \\ \begin{matrix} \downarrow \\ y_\wedge \end{matrix} \quad \begin{matrix} \downarrow \\ x_0 \end{matrix} \quad \begin{matrix} \downarrow \\ x_1 \end{matrix} \end{matrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} \otimes \begin{matrix} & \begin{matrix} \xrightarrow{X_1} \\ 0 \quad 1 \end{matrix} \\ \begin{matrix} \downarrow \\ x_0 \end{matrix} \quad \begin{matrix} \downarrow \\ x_1 \end{matrix} \end{matrix} \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix} + \begin{matrix} & \begin{matrix} \xrightarrow{X_1} \\ 0 \quad 1 \end{matrix} \\ \begin{matrix} \downarrow \\ y_\wedge \end{matrix} \quad \begin{matrix} \downarrow \\ x_0 \end{matrix} \quad \begin{matrix} \downarrow \\ x_1 \end{matrix} \end{matrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} \otimes \begin{matrix} & \begin{matrix} \xrightarrow{X_1} \\ 0 \quad 1 \end{matrix} \\ \begin{matrix} \downarrow \\ x_0 \end{matrix} \quad \begin{matrix} \downarrow \\ x_1 \end{matrix} \end{matrix} \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix} = \begin{matrix} & \begin{matrix} \xrightarrow{X_1} \\ 0 \quad 1 \end{matrix} \\ \begin{matrix} \downarrow \\ x_0 \end{matrix} \quad \begin{matrix} \downarrow \\ x_1 \end{matrix} \end{matrix} \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix} \begin{matrix} \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix} \\ \xrightarrow{Y_\wedge} \\ 0 \quad 1 \end{matrix}$$

Furthermore, the 2-ary disjunction $\vee : [2] \times [2] \rightarrow [2]$ is encoded by the order-3 tensor

$$\beta^\vee [Y_\vee, X_0, X_1] = \begin{matrix} & \begin{matrix} \xrightarrow{X_1} \\ 0 \quad 1 \end{matrix} \\ \begin{matrix} \downarrow \\ y_\vee \end{matrix} \quad \begin{matrix} \downarrow \\ x_0 \end{matrix} \quad \begin{matrix} \downarrow \\ x_1 \end{matrix} \end{matrix} \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix} \begin{matrix} \begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix} \\ \xrightarrow{Y_\vee} \\ 0 \quad 1 \end{matrix}$$

Interpretation as CompActNets The propositional formula and its negation can be represented by this tensor via

$$f [X_{[d]}] = \langle \epsilon_1 [Y_f], \beta^f [Y_f, X_{[d]}] \rangle_{[X_{[d]}]} \quad \text{and} \quad \neg f [X_{[d]}] = \langle \epsilon_0 [Y_f], \beta^f [Y_f, X_{[d]}] \rangle_{[X_{[d]}]}.$$

Both f and $\neg f$ are thus Computation-Activation Networks to the statistic $\{f\}$ and the hard activation tensor $\epsilon_1 [Y_f]$, respectively $\epsilon_0 [Y_f]$.

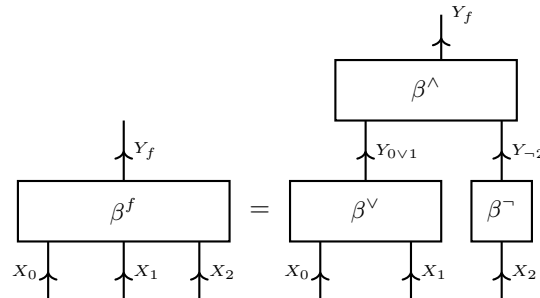
5.2 Syntactic decomposition of propositional formulas

Propositional formulas of concern often have a syntactic specification as composed functions. We can therefore apply the neural paradigm to find efficient representations of them.

Definition 19 (Syntactic decompositions). *A syntactic decomposition of a propositional formula f is a decomposition hypergraph (see Def. 17) such that all nodes are decorated with the dimension $m_v = 2$ and the composition function f .*

We thus have a tensor network representation of any propositional formula based on its syntactic decomposition, where the hypergraph of the syntactic decomposition equals the hypergraph of the representing tensor network.

Example 18. *For the formula $f [X_{[3]}] = (X_0 \vee X_1) \wedge \neg X_2$ from Example 15, we have the following syntactical decomposition of its basis encoding:*



5.3 Entailment decision by contractions

We have already seen that the contraction of a propositional formula counts its models. This allows to define entailment between two propositional formulas as defined in the following. To generalize the treatment, we no longer demand that the variables of a formula are of dimension 2. We further use $\neg f[X_{[d]}] = \mathbb{I}[X_{[d]}] - f[X_{[d]}]$.

Definition 20 (Entailment of propositional formulas). *Given two propositional formulas $\mathcal{KB}$ and f , we say that $\mathcal{KB}$ entails f , denoted by $\mathcal{KB} \models f$, if any model of $\mathcal{KB}$ is also a model of f , that is*

$$\langle \mathcal{KB}[X_{[d]}], \neg f[X_{[d]}] \rangle_{[\emptyset]} = 0.$$

If $\mathcal{KB} \models \neg f$ holds (i.e. $\langle \mathcal{KB}[X_{[d]}], f[X_{[d]}] \rangle_{[\emptyset]} = 0$), we say that $\mathcal{KB}$ contradicts f .

Classically (see e.g. Russell and Norvig (2021)) entailment in propositional logics is defined as the unsatisfiability of $\mathcal{KB} \wedge \neg f$. This is equivalent to Def. 20 due to the equivalence of $\langle \mathcal{KB}[X_{[d]}], \neg f[X_{[d]}] \rangle_{[\emptyset]} = 0$ and $\langle (\mathcal{KB} \wedge (\neg f))[X_{[d]}] \rangle_{[\emptyset]} = 0$, which is the unsatisfiability of $\mathcal{KB} \wedge \neg f$.

Example 19 ($n^2 \times n^2$ Sudoku). We index the rows and the columns by tuples $(r0, r1)$ and $(c0, c1)$, where $r0, r1, c0, c1 \in [n]$. The first index indicates the block and the second counts the row or column inside that block. For each $r0, r1, c0, c1 \in [n]$ and $i \in [n^2]$ we then define an atomic variable $X_{r0, r1, c0, c1, i} \in \{0, 1\}$ indicating whether in the row $(r0, r1)$ and column $(c0, c1)$ the number i is written. The Sudoku rules then amount to the formula

$$\mathcal{KB}^n := \left(\bigwedge_{r0, r1, c0, c1 \in [n]} \left(\bigoplus_{i \in [n^2]}^{(1)} X_{r0, r1, c0, c1, i} \right) \right) \wedge \left(\bigwedge_{r0, r1 \in [n], i \in [n^2]} \left(\bigoplus_{c0, c1 \in [n]}^{(1)} X_{r0, r1, c0, c1, i} \right) \right) \wedge \left(\bigwedge_{c0, c1 \in [n], i \in [n^2]} \left(\bigoplus_{r0, r1 \in [n]}^{(1)} X_{r0, r1, c0, c1, i} \right) \right) \wedge \left(\bigwedge_{r0, c0 \in [n], i \in [n^2]} \left(\bigoplus_{r1, c1 \in [n]}^{(1)} X_{r0, r1, c0, c1, i} \right) \right),$$

where $\bigoplus^{(1)}$ is the n^2 -ary exclusive or connective (that is 1 if and only if exactly one of the arguments is 1). The four outer brackets in $\mathcal{KB}$ mark the constraints that at each position exactly one number is assigned, further that in each row each number is assigned once, and similar for the columns and the squares of the board. When solving a specific Sudoku instance, one typically knows from an initial board assignment E^{start} a collection of atomic variables, which hold, and needs to find further atomic variables, which are entailed. This means, we need to decide for each $(r0, r1, c0, c1, i) \notin E^{\text{start}}$ whether the Sudoku rules and the initial board imply that the atomic variable $X_{r0, r1, c0, c1, i}$ (i.e. assignment to the board) is true

$$\mathcal{KB}^n \wedge \left(\bigwedge_{(r0, r1, c0, c1, i) \in E^{\text{start}}} X_{r0, r1, c0, c1, i} \right) \models X_{r0, r1, c0, c1, i}$$

or false

$$\mathcal{KB} \wedge \left(\bigwedge_{(r0, r1, c0, c1, i) \in E^{\text{start}}} X_{r0, r1, c0, c1, i} \right) \models \neg X_{r0, r1, c0, c1, i}.$$

If and only if the Sudoku has a unique solution given the initial board assignment E^{start} , exactly one of these entailment statements holds for each $(r0, r1, c0, c1, i) \notin E^{\text{start}}$. Deciding which is equivalent to solving the Sudoku.

For a more concrete example, let $n = 2$ and

$$E^{\text{start}} = \{(0, 0, 0, 0, 0), (0, 0, 1, 0, 2), (0, 0, 1, 1, 1), (0, 1, 0, 1, 1), (1, 0, 1, 0, 3), (1, 1, 0, 0, 3), (1, 1, 0, 1, 2)\}.$$

We visualize this evidence by writing $i + 1$ in a grid cell $(r0, r1, c0, c1)$ to indicate that $(r0, r1, c0, c1, i) \in E^{\text{start}}$:

1		3	2
	2		
		4	
4	3		

After deriving a sparse tensor network representations in Example 21, we demonstrate a solution algorithm to solve this instance in Example 22.

5.4 Efficient representation of knowledge bases

We now investigate the representation of propositional knowledge bases $\mathcal{KB} = \{f_\ell : \ell \in [p]\}$, which are sets of propositional formulas f_ℓ . The conjunction of these formulas is the knowledge base formula

$$\mathcal{KB} [X_{[d]}] = \bigwedge_{\ell \in [p]} f_\ell [X_{[d]}] .$$

To show efficient representations, we use the following identities.

Lemma 3 (Computation Network Symmetries). *For the d -ary $\wedge$ -connective (where $d \in \mathbb{N}$) and the unary $\neg$ -connective it holds that*

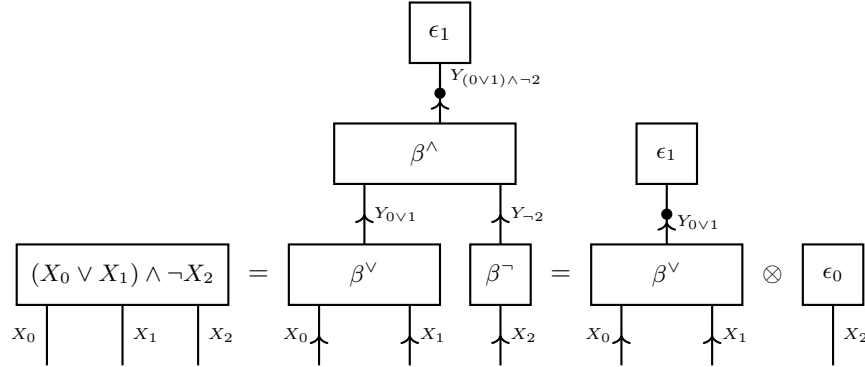
$$\langle \epsilon_1 [Y], \beta^\wedge [Y, X_{[d]}] \rangle_{[X_{[d]}]} = \bigotimes_{k \in [d]} \epsilon_1 [X_k] \quad \text{and} \quad \langle \epsilon_1 [Y], \beta^\neg [Y, X] \rangle_{[X]} = \epsilon_0 [X] .$$

Proof. Follows directly from the definitions of the basis encodings and the connectives. $\square$

Example 20. *For the propositional formula from Example 15*

$$f [X_{[3]}] = (X_0 \vee X_1) \wedge \neg X_2 ,$$

we can write the formula in terms of a Computation-Activation Network with activation tensor ϵ_1 and computation network decomposed by the basis encodings as depicted below. First, it is written with one activation vector. Second, we see that it can also be interpreted with multiple features.



We use this to decompose knowledge bases into their individual formulas as follows.

Theorem 6. *For any knowledge base $\mathcal{KB} [X_{[d]}] = \bigwedge_{\ell \in [p]} f_\ell [X_{[d]}]$ it holds that*

$$\mathcal{KB} [X_{[d]}] = \langle \{f_\ell [X_{[d]}] : \ell \in [p]\} \rangle_{[X_{[d]}]} .$$

Proof. With Lem. 3 we have

$$\begin{aligned} \mathcal{KB} [X_{[d]}] &= \langle \{ \epsilon_1 [Y_\wedge], \beta^\wedge [Y_\wedge, Y_{[p]}] \} \cup \{ \beta^{f_\ell} [Y_\ell, X_{[d]}] : \ell \in [p] \} \rangle_{[X_{[d]}]} \\ &= \left\langle \bigcup_{\ell \in [p]} \{ \epsilon_1 [Y_\ell], \beta^{f_\ell} [Y_\ell, X_{[d]}] : \ell \in [p] \} \right\rangle_{[X_{[d]}]} \\ &= \langle \{f_\ell [X_{[d]}] : \ell \in [p]\} \rangle_{[X_{[d]}]} . \end{aligned}$$

$\square$

Example 21 (Sparse representation of Sudoku rule knowledge base). *We now exploit Thm. 6 to find efficient tensor network representation of the Sudoku knowledge base from Example 19. We directly get, that the knowledge base $\mathcal{KB}^n$ of Sudoku rules is a tensor network of the $4 \cdot n^4$ constraint formulas using the n^2 -ary connective $\bigoplus^{(1)}$, and the evidence E^{start} can be encoded by vectors $\epsilon_1 [X_{(r_0, r_1, c_0, c_1, i)}]$. To get a representation by matrices instead of tensors*

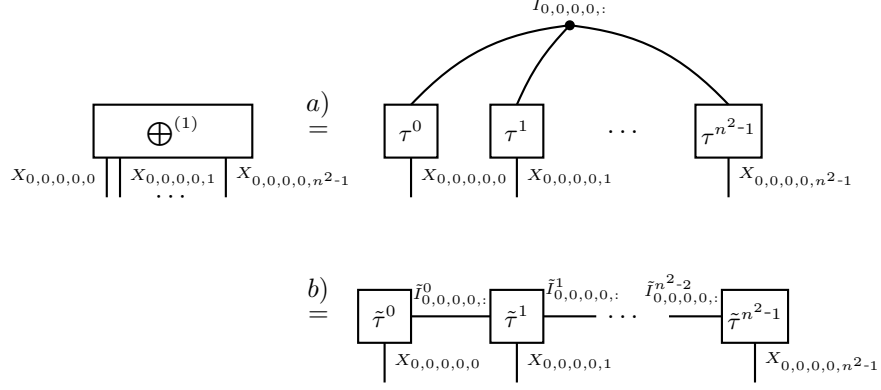


Figure 12: Decomposition of the position constraint $\bigoplus^{(1)}$ at position $(r_0, r_1, c_0, c_1) = (0, 0, 0, 0)$ into a) a CP decomposition with hidden variable $I_{0,0,0,0,:}$ and b) a TT decomposition with $d - 1$ hidden variables $I_{0,0,0,0,:}^k, k \in [d - 1]$.

of order n^2 , we introduce a hidden variable I taking values in $[n^2]$ for each of the constraints. With the usage of matrices

$$\tau^k [X_k, I] = \epsilon_0 [X_k] \otimes \mathbb{I} [I] + (\epsilon_1 [X_k] - \epsilon_0 [X_k]) \otimes \epsilon_k [I]$$

we have the decomposition

$$\bigoplus^{(1)} [X_{[n^2]}] = \langle \{ \tau^k [X_k, I] : k \in [n^2] \} \rangle_{[X_{[n^2]}]},$$

which is a CP decomposition (see Example 2) depicted in Figure 12 a).

Alternatively there is a TT decomposition (see Example 3) of the constraint $\bigoplus^{(1)}$, which we depict in Figure 12 b). We introduce for $k \in [d - 1]$ hidden variables I^k of dimension 2, which are interpreted as the indicator, whether one of the variables $X_{[k]}$ is true. Following this interpretation we introduce TT cores

$$\begin{aligned} \tilde{\tau}^0 [X_0, \tilde{I}^0] &= \epsilon_1 [X_0] \otimes \epsilon_1 [\tilde{I}^0] + \epsilon_0 [X_0] \otimes \epsilon_0 [\tilde{I}^0] \\ \tilde{\tau}^{d-1} [\tilde{I}^{d-2}, X_{d-1}] &= \epsilon_0 [\tilde{I}^{d-2}] \otimes \epsilon_1 [X_{d-1}] + \epsilon_1 [\tilde{I}^{d-2}] \otimes \epsilon_0 [X_{d-1}] \end{aligned}$$

and for $k \in \{1, \dots, d - 2\}$

$$\tilde{\tau}^k [\tilde{I}^{k-1}, X_k, \tilde{I}^k] = \delta [\tilde{I}^{k-1}, \tilde{I}^k] \otimes \epsilon_0 [X_k] + \epsilon_0 [\tilde{I}^{k-1}] \otimes \epsilon_1 [X_k] \otimes \epsilon_1 [\tilde{I}^{k-1}].$$

We notice, that the TT decomposition of the constraint $\bigoplus^{(1)}$ introduces $d - 1$ many hidden variables of dimension 2, whereas the CP decomposition introduces a single hidden variable of dimension d . However, in the following we will further apply the CP decomposition.

Given evidence E^{start} we denote the Sudoku knowledge base $\mathcal{KB}^{n, E^{\text{start}}}$. We model the Sudoku knowledge base $\mathcal{KB}^{n, E^{\text{start}}}$ as a tensor network on a hypergraph $\mathcal{G}^{\text{Sudoku}, n}$ consistent in

- $n^6 + 4 \cdot n^4$ nodes by n^6 categorical variables $X_{(r_0, r_1, c_0, c_1, i)}$ and by $4 \cdot n^4$ decomposition variables to the constraints
- $5 \cdot n^6$ edges

$$\begin{aligned} \mathcal{E} = \bigcup_{r_0, r_1, c_0, c_1 \in [n]} & \{ \{ X_{(r_0, r_1, c_0, c_1, i)} \}, \{ X_{(r_0, r_1, c_0, c_1, i)}, I_{r_0, r_1, c_0, c_1, :} \}, \{ X_{(r_0, r_1, c_0, c_1, i)}, I_{r_0, r_1, :, :, i} \}, \\ & \{ X_{(r_0, r_1, c_0, c_1, i)}, I_{:, :, c_0, c_1, i} \}, \{ X_{(r_0, r_1, c_0, c_1, i)}, I_{r_0, :, c_0, :, i} \} \} \end{aligned}$$

We denote the decomposition variables to the position, row, column and square constraints by $I_{r_0, r_1, c_0, c_1, :}$, $I_{r_0, r_1, :, :, i}$, $I_{:, :, c_0, c_1, i}$ and $I_{r_0, :, c_0, :, i}$.

Each edge containing a decomposition variable is decorated by a matrix $\tau^k [X, I]$ corresponding to a core in the CP decomposition of a constraint. Here k is determined by the tuple $(r0, r1, c0, c1, i)$ and the type of the constraint (for example, for the variable $X_{(0,1,1,2,1)}$ and the row constraint $I_{(0,1,1,2,1)}$ we have $k = 1 \cdot n + 2$). We further assign to each edge containing a single variable $\{X_{(r0,r1,c0,c1,i)}\}$ either the vector $\epsilon_1 [X_{(r0,r1,c0,c1,i)}]$ if $(r0, r1, c0, c1, i) \in E^{\text{start}}$ or the trivial vector $\mathbb{I} [X_{(r0,r1,c0,c1,i)}]$.

5.5 Entailment decision by message passing

Since contracting the whole tensor network is often infeasible, local contractions can be considered to decide entailment in some cases. Here, a local contraction describes the calculation of contractions along few closely connected tensors in the network. Before presenting the resulting Constraint Propagation algorithm, we first show two important properties of local entailment motivating the procedure.

Theorem 7 (Monotonicity of propositional logics). *If $\tilde{\mathcal{KB}} \subset \mathcal{KB}$ and $\tilde{\mathcal{KB}} \models f$ then also $\mathcal{KB} \models f$.*

Proof. Since $\tilde{\mathcal{KB}} \models f$ it holds that $\langle \tilde{\mathcal{KB}}[X_{[d]}], \neg f[X_{[d]}] \rangle_{[\emptyset]} = 0$ and thus

$$\langle \tilde{\mathcal{KB}}[X_{[d]}], \neg f[X_{[d]}] \rangle_{[X_{[d]}]} = 0 [X_{[d]}] .$$

Denoting by $\mathcal{KB}/\tilde{\mathcal{KB}}$ the conjunctions of formulas in $\mathcal{KB}$ not in $\tilde{\mathcal{KB}}$, we have

$$\begin{aligned} \langle \mathcal{KB} [X_{[d]}], \neg f [X_{[d]}] \rangle_{[\emptyset]} &= \langle (\mathcal{KB}/\tilde{\mathcal{KB}})[X_{[d]}], \tilde{\mathcal{KB}}[X_{[d]}], \neg f [X_{[d]}] \rangle_{[\emptyset]} \\ &= \left\langle (\mathcal{KB}/\tilde{\mathcal{KB}})[X_{[d]}], \left\langle \tilde{\mathcal{KB}}[X_{[d]}], \neg f [X_{[d]}] \right\rangle_{[X_{[d]}]} \right\rangle_{[\emptyset]} \\ &= \langle (\mathcal{KB}/\tilde{\mathcal{KB}})[X_{[d]}], 0 [X_{[d]}] \rangle_{[\emptyset]} \\ &= 0 . \end{aligned}$$

□

To decide entailment, we can therefore investigate entailment on smaller parts of the knowledge base. This is sound by the above theorem but not complete since it can happen that no smaller part of the knowledge base entails the formula while the whole knowledge base does. We can furthermore add entailed formulas to the knowledge base without changing it as is shown next.

Theorem 8 (Invariance of adding entailed formulas). *If and only if $\mathcal{KB} \models f$ we have that*

$$\mathcal{KB} [X_{[d]}] = \langle \mathcal{KB} [X_{[d]}], f [X_{[d]}] \rangle_{[X_{[d]}]} .$$

Proof. We use that $f [X_{[d]}] + \neg f [X_{[d]}] = \mathbb{I} [X_{[d]}]$ and thus

$$\begin{aligned} \mathcal{KB} [X_{[d]}] &= \langle \mathcal{KB} [X_{[d]}], (f [X_{[d]}] + \neg f [X_{[d]}]) \rangle_{[X_{[d]}]} \\ &= \langle \mathcal{KB} [X_{[d]}], f [X_{[d]}] \rangle_{[X_{[d]}]} + \langle \mathcal{KB} [X_{[d]}], \neg f [X_{[d]}] \rangle_{[X_{[d]}]} . \end{aligned}$$

Since $\langle \mathcal{KB} [X_{[d]}], \neg f [X_{[d]}] \rangle_{[X_{[d]}]}$ is boolean, we have that

$$\mathcal{KB} [X_{[d]}] = \langle \mathcal{KB} [X_{[d]}], f [X_{[d]}] \rangle_{[X_{[d]}]}$$

if and only if $\langle \mathcal{KB} [X_{[d]}], \neg f [X_{[d]}] \rangle_{[\emptyset]} = 0$, that is $\mathcal{KB} \models f$. □

The mechanism of Thm. 8 provides us with a means to store entailment information in small-order auxiliary tensors. One way to exploit this accessibility of local entailment information are message passing schemes similar to Algorithm 1 propagating the information. This approach decides local entailment by iteratively adding entailed formulas to the knowledge base and checking further entailment on neighboring tensors of the knowledge base. Since

for entailment decisions the support of the contractions is sufficient, we can apply non-zero indicators before sending contraction messages. We then schedule new messages in the direction (e_0, e_1) once the support of a message received at e_0 has been changed. Note that such a scheduling system is guaranteed to converge since there can only be a finite number of message changes. We further directly reduce the computation of messages to their support and call the resulting Constraint Propagation (Algorithm 3).

Algorithm 3 Constraint Propagation

Require: Tensor network $\tau^{\mathcal{G}}$ on a hypergraph $\mathcal{G}$

Ensure: Messages $\{\chi_{e_0 \rightarrow e_1} [X_{e_0 \cap e_1}] : (e_0, e_1) \in \mathcal{E}^{\rightarrow}\}$ containing entailment statements

Initialize a queue $S = \mathcal{E}^{\rightarrow}$ of message directions

Initialize messages $\chi_{e_0 \rightarrow e_1} [X_{e_0 \cap e_1}] = \mathbb{I} [X_{e_0 \cap e_1}]$ for $(e_0, e_1) \in \mathcal{E}^{\rightarrow}$

while S not empty **do**

 Pop a (e_0, e_1) pair from S

 Update the message

$$\chi_{e_0 \rightarrow e_1} [X_{e_0 \cap e_1}] = \mathbb{I}_{\neq 0} \left(\langle \{\tau^{e_0} [X_{e_0}]\} \cup \{\chi_{e_2 \rightarrow e_0} [X_{e_2 \cap e_0}] : (e_2, e_0) \in \mathcal{E}^{\rightarrow}, e_2 \neq e_1\} \rangle_{[X_{e_0 \cap e_1}]} \right)$$

if $\tau [X_{e_0 \cap e_1}] \neq \chi_{e_0 \rightarrow e_1} [X_{e_0 \cap e_1}]$ **then**

 Update the message: $\chi_{e_0 \rightarrow e_1} [X_{e_0 \cap e_1}] := \tau [X_{e_0 \cap e_1}]$

 Add $S = S \cup \{(e_1, e_2) : (e_1, e_2) \in \mathcal{E}^{\rightarrow}\}$

end if

end while

return Messages $\{\chi_{e_0 \rightarrow e_1} [X_{e_0 \cap e_1}] : (e_0, e_1) \in \mathcal{E}^{\rightarrow}\}$

Theorem 9. All messages during constraint propagation are sound, meaning that for all $(e_0, e_1) \in \mathcal{E}^{\rightarrow}$ it holds that

$$\mathbb{I}_{\neq 0} \left(\langle \tau^{\mathcal{G}} \rangle_{[X_{e_0 \cap e_1}]} \right) \prec \chi_{e_0 \rightarrow e_1} [X_{e_0 \cap e_1}].$$

Proof. We show this theorem by induction over the while loop of Algorithm 3. At the first iteration, we have for all messages $\chi_{e_0 \rightarrow e_1} [X_{e_0 \cap e_1}] = \mathbb{I} [X_{e_0 \cap e_1}]$ and thus

$$\tau^{\mathcal{G}} = \langle \{\tau^{\mathcal{G}}\} \cup \{\chi_{e_0 \rightarrow e_1} [X_{e_0 \cap e_1}] : (e_0, e_1) \in \mathcal{E}^{\rightarrow}\} \rangle_{[X_{\mathcal{V}}]}. \quad (2)$$

By Thm. 7 we then have for the first message send along the pair (e_0, e_1) that

$$\begin{aligned} \mathbb{I}_{\neq 0} \left(\langle \tau^{\mathcal{G}} \rangle_{[X_{e_0 \cap e_1}]} \right) &\prec \mathbb{I}_{\neq 0} \left(\langle \{\tau^{e_0} [X_{e_0}]\} \cup \{\chi_{e_2 \rightarrow e_0} [X_{e_2 \cap e_0}] : (e_2, e_0) \in \mathcal{E}^{\rightarrow}, e_2 \neq e_1\} \rangle_{[X_{e_0 \cap e_1}]} \right) \\ &= \chi_{e_0 \rightarrow e_1} [X_{e_0 \cap e_1}]. \end{aligned}$$

We now assume that at an arbitrary state of the algorithm the inequality holds for all previously sent messages. By Thm. 8 we can contract the messages with the tensor network without changing it and (2) thus still holds. We then conclude with Thm. 7 that the claimed property also holds for the new message. $\square$

Example 22 (Message passing for the Sudoku instance of Example 19). We iteratively solve a Sudoku puzzle by determining a possible value based on neighboring cells, rows and squares (using Thm. 7) and adding to our knowledge (using Thm. 8). For example, consider the following $n = 2$ Sudoku puzzle, where a first entailment step uses only the knowledge of the rules and the blue cells to determine the value 3 in the first square:

$$\begin{array}{|c|c|c|c|} \hline 1 & & \textcolor{blue}{3} & 2 \\ \hline & \textcolor{blue}{2} & & \\ \hline & & 4 & \\ \hline 4 & 3 & & \\ \hline \end{array} = \begin{array}{|c|c|c|c|} \hline 1 & & 3 & 2 \\ \hline \textcolor{red}{3} & 2 & & \\ \hline & & 4 & \\ \hline 4 & 3 & & \\ \hline \end{array} = \dots = \begin{array}{|c|c|c|c|} \hline 1 & \textcolor{red}{4} & 3 & 2 \\ \hline \textcolor{red}{3} & 2 & \textcolor{red}{1} & \textcolor{red}{4} \\ \hline \textcolor{red}{2} & \textcolor{red}{1} & 4 & \textcolor{red}{3} \\ \hline 4 & 3 & \textcolor{red}{2} & \textcolor{red}{1} \\ \hline \end{array}$$

To illustrate the first reasoning step of assigning $X_{0,1,0,0,2}$ we make the following entailment steps applying Thm. 7. We also depict in Figure 13 the corresponding messages in the Constraint Propagation Algorithm on the hypergraph $\mathcal{G}_{\text{Sudoku}, n}$.

- From $X_{0,1,0,1,1}$ (i.e. the 2 in the cell $(0, 1, 0, 1)$) and the Sudoku rule that at the cell $(0, 1, 0, 1)$ exactly one number is assigned, we get

$$\left(\bigoplus_{i \in [n^2]}^{(1)} X_{0,1,0,1,i} \right) \wedge X_{0,1,0,1,1} \models \neg X_{0,1,0,1,2},$$

That is, that the number 3 is not in the cell $(0, 1, 0, 1)$. This entailment step is performed by three consecutive messages (see $\chi^{(0,[3])}$ in Figure 13) along the directions

$$(e_0, e_1) \in [(\{X_{0,1,0,1,1}\}, \{X_{0,1,0,1,1}, I_{0,1,0,1,\cdot}\}), (\{X_{0,1,0,1,1}, I_{0,1,0,1,\cdot}\}, \{X_{0,1,0,1,2}, I_{0,1,0,1,\cdot}\}), \\ (\{X_{0,1,0,1,2}, I_{0,1,0,1,\cdot}\}, \{X_{0,1,0,1,2}, I_{0,\cdot,0,\cdot,2}\})].$$

Intuitively, the messages communicate to the square constraint $I_{0,\cdot,0,\cdot,2}$, that by the position constraint $I_{0,1,0,1,\cdot}$ the variable 3 cannot be assigned at $(0, 1, 0, 1)$.

- From $X_{0,0,1,0,2}$ (i.e. the 3 in the cell $(0, 0, 1, 0)$) and the Sudoku rule that at the row $(0, 0)$ exactly one number is assigned, we get

$$\left(\bigoplus_{c0, c1 \in [n]}^{(1)} X_{0,0,c0,c1,2} \right) \wedge X_{0,0,1,0,2} \models \neg X_{0,0,0,0,2} \wedge \neg X_{0,0,0,1,2},$$

That is, that the number 3 is neither in the cell $(0, 0, 0, 0)$ nor in $(0, 0, 0, 1)$. This entailment step is performed by five consecutive messages (see $\chi^{(1,[5])}$ in Figure 13) along the directions

$$(e_0, e_1) \in [(\{X_{0,0,1,0,2}\}, \{X_{0,0,1,0,2}, I_{0,0,\cdot,\cdot,2}\}), (\{X_{0,0,1,0,2}, I_{0,0,\cdot,\cdot,2}\}, \{X_{0,0,0,0,2}, I_{0,0,\cdot,\cdot,2}\}), \\ (\{X_{0,0,1,0,2}, I_{0,0,\cdot,\cdot,2}\}, \{X_{0,0,0,1,2}, I_{0,0,\cdot,\cdot,2}\}), (\{X_{0,0,0,0,2}, I_{0,0,\cdot,\cdot,2}\}, \{X_{0,0,0,0,2}, I_{0,\cdot,0,\cdot,2}\}), \\ (\{X_{0,0,0,1,2}, I_{0,\cdot,0,\cdot,2}\}, \{X_{0,0,0,1,2}, I_{0,\cdot,0,\cdot,2}\})].$$

The messages communicate that based on the decomposition cores of the constraint to the number $i = 3$ in the first row $(r_0, r_1) = (0, 0)$, that the number 3 cannot be assigned at $(0, 0, 0, 0)$ and $(0, 0, 0, 1)$.

We add these formulas to our knowledge base (justified by Thm. 8) and use the rule, that 3 appears exactly once in the first square

$$\left(\bigoplus_{r1, c1 \in [n]}^{(1)} X_{0,r1,0,c1,2} \right) \wedge (\neg X_{0,1,0,1,2}) \wedge (\neg X_{0,0,0,0,2} \wedge \neg X_{0,0,0,1,2}) \models X_{0,1,0,0,2}.$$

That is, we conclude that the number 3 must be in the cell $(0, 1, 0, 0)$, which information is also included in the updated knowledge base for further reasoning steps. This last entailment step is performed by four consecutive messages (see $\chi^{(2,[4])}$ in Figure 13) along the directions

$$(e_0, e_1) \in [(\{X_{0,1,0,1,2}, I_{0,\cdot,0,\cdot,2}\}, \{X_{0,1,0,0,2}, I_{0,\cdot,0,\cdot,2}\}), (\{X_{0,1,0,0,2}, I_{0,\cdot,0,\cdot,2}\}, \{X_{0,0,0,1,2}, I_{0,\cdot,0,\cdot,2}\}), \\ (\{X_{0,0,0,1,2}, I_{0,\cdot,0,\cdot,2}\}, \{X_{0,1,0,0,2}, I_{0,\cdot,0,\cdot,2}\}), (\{X_{0,1,0,0,2}, I_{0,\cdot,0,\cdot,2}\}, \{X_{0,1,0,0,2}\})]$$

The first three messages communicate, that the 3 is not possible the positions $(0, 1, 0, 1)$, $(0, 0, 0, 1)$ and $(0, 0, 1, 0)$ and the fourth message concludes that the 3 then has to be at position $(0, 1, 0, 0)$.

We now iteratively apply similar reasoning steps and store the entailed variables in E^{entailed} , until we arrive at the right side of the above sketch. We conclude that

$$\mathcal{KB}^2 \wedge \left(\bigwedge_{(r_0, r_1, c_0, c_1, i) \in E^{\text{start}}} X_{r_0, r_1, c_0, c_1, i} \right) \models \left(\bigwedge_{(r_0, r_1, c_0, c_1, i) \in E^{\text{entailed}}} X_{r_0, r_1, c_0, c_1, i} \right).$$

Since all Sudoku rules are satisfied in the final assignment and to each cell (r_0, r_1, c_0, c_1) we found exactly one $i \in [n^2]$ such that $(r_0, r_1, c_0, c_1, i) \in E^{\text{start}} \cup E^{\text{entailed}}$, there is a unique solution of the puzzle and we conclude

$$\mathcal{KB}^2 \wedge \left(\bigwedge_{(r_0, r_1, c_0, c_1, i) \in E^{\text{start}}} X_{r_0, r_1, c_0, c_1, i} \right) \\ = \left(\bigwedge_{(r_0, r_1, c_0, c_1, i) \in E^{\text{start}}} X_{r_0, r_1, c_0, c_1, i} \right) \wedge \left(\bigwedge_{(r_0, r_1, c_0, c_1, i) \in E^{\text{entailed}}} X_{r_0, r_1, c_0, c_1, i} \right).$$

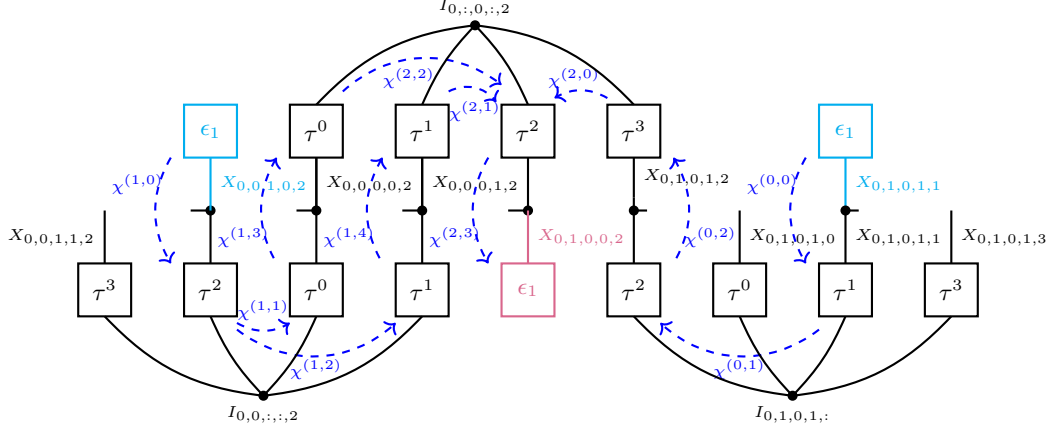


Figure 13: The tensor network decomposition of 3 out of $4 \cdot 2^2 = 64$ rules in the $2^2 \times 2^2$ Sudoku knowledge base (see Example 21), namely to the number 3 appearing once in the $(0,0)$ -square (top), the number 3 appearing once in the $(0,0)$ -row (bottom left) and a unique number appearing at the $(0,1,0,1)$ -position (bottom right). The evidence of the number 3 already being assigned at the position $(0,0,1,0)$ is sketched by a basis vector ϵ_1 on the left side, and the number 2 assigned at position $(0,1,0,1)$ analogously on the right side. During Constraint Propagation Algorithm 3 on the hypergraph of Sudoku rules and evidence (see Example 22), this evidence is in three epochs of messages propagated to the constraints by partial entailment steps and imply that $X_{0,1,0,0,2}$ is true, that is that at the position $(0,1,0,0)$ the number 3 needs to be assigned. We depict the messages between the cores by dashed lines labeled by $\chi^{(0,[3])}$, $\chi^{(1,[5])}$ and $\chi^{(2,[4])}$ and provide further interpretation in Example 22.

6 Hybrid Logic Networks

Let us now exploit the common formulation of logical formulas and probabilistic models in CompActNets to define hybrid models that combine both aspects. We call CompActNets Hybrid Logic Networks in the special case of Boolean statistics t and elementary activations.

6.1 Parametrization

We first introduce Hybrid Logic Networks, which can be regarded as a unification of logical and probabilistic models.

Definition 21 (Hybrid Logic Network (HLN)). *Given a Boolean statistic t , we call any element of $\Lambda^{t, \text{EL}}$ a Hybrid Logic Network. The extended canonical parameter set for t is the set*

$$\mathcal{P}_p := \{(\mathcal{U}, y_{\mathcal{U}}) : \mathcal{U} \subset [p], y_{\mathcal{U}} \in \prod_{\ell \in \mathcal{U}} [2]\} \times \mathbb{R}^p.$$

For each Hybrid Logic Network $\mathbb{P}^{t, (\mathcal{U}, y_{\mathcal{U}}, \theta)} [X_{[d]}]$, we can associate a tuple $(\mathcal{U}, y_{\mathcal{U}}, \theta)$ consisting of a subset $\mathcal{U} \subset [p]$, a tuple $y_{\mathcal{U}} \in \prod_{\ell \in \mathcal{U}} [2]$, and $\theta [L] \in \mathbb{R}^p$ such that

$$\mathbb{P}^{t, (\mathcal{U}, y_{\mathcal{U}}, \theta)} [X_{[d]}] = \left\langle \beta^t [Y_{[p]}, X_{[d]}], \xi^{(\mathcal{U}, y_{\mathcal{U}}, \theta)} [Y_{[p]}] \right\rangle_{[X_{[d]}] \emptyset}$$

where the activation core is

$$\xi^{(\mathcal{U}, y_{\mathcal{U}}, \theta)} [Y_{[p]}] = \left\langle \alpha^{\theta} [Y_{[p]}], \kappa^{(\mathcal{U}, y_{\mathcal{U}})} [Y_{[p]}] \right\rangle_{[Y_{[p]}]}.$$

We notice that the parametrization by $\mathcal{P}_p$ is one-to-one for any non-vanishing elementary activation tensor up to a scalar factor. Given an arbitrary elementary activation tensor $\bigotimes_{\ell \in [p]} \xi^{\ell} [Y_{\ell}]$, we can always find a corresponding tuple in $\mathcal{P}_p$ by choosing²

$$\mathcal{U} = \{\ell : \mathbb{I}_{\neq 0} (\xi^{\ell} [Y_{\ell}]) \neq \mathbb{I} [Y_{\ell}]\},$$

²Here $\mathbb{I}_{\neq 0} (\cdot)$ is the indicator of non-zero entries acting coordinatewise and $\mathbb{I} [Y_{\ell}]$ is the vector $[1, 1]^T$.

further for all $\ell \in \mathcal{U}$

$$y_\ell = \begin{cases} 0 & \text{if } \mathbb{I}_{\neq 0}(\xi^\ell[Y_\ell]) = \epsilon_0[Y_\ell] \\ 1 & \text{if } \mathbb{I}_{\neq 0}(\xi^\ell[Y_\ell]) = \epsilon_1[Y_\ell] \end{cases}$$

and a parameter vector $\theta[L] \in \mathbb{R}^p$ defined for all $\ell \in [p]$ as

$$\theta[L = \ell] = \begin{cases} 0 & \text{if } \ell \in \mathcal{U} \\ \ln \left[\frac{\xi^\ell[Y_\ell=1]}{\xi^\ell[Y_\ell=0]} \right] & \text{if } \ell \notin \mathcal{U}. \end{cases}$$

Then we have by construction that there is $\lambda > 0$ with

$$\bigotimes_{\ell \in [p]} \xi^\ell[Y_\ell] = \lambda \cdot \xi^{(\mathcal{U}, y_{\mathcal{U}}, \theta)}[Y_{[p]}].$$

Let us demonstrate the utility of Hybrid Logic Networks with an example from accounting.

Example 23 (Hybrid Logic Network for a toy accounting model). *We consider a system of three variables $A1$ Account 1 is booked, $A2$ Account 2 is booked, F a feature on an invoice. Assume the following two rules have to be respected:*

- *Exactly one account must be booked.*
- *If feature F is present on the invoice, the account $A1$ is typically booked.*

We formalize this with the statistic

$$t = (X_{A1} \oplus X_{A2}, X_F \Rightarrow X_{A1}).$$

While the first formula is a hard feature, the second is soft since prone to exceptions. We parameterize the first output of the statistic with the hard parameters by setting the set of indices to be initialized with hard logic $A = \{0\}$ and the corresponding initialization $y_0 = 1$, meaning that the first output of the statistic has to be true for the input to have positive probability. The “hard logic activation tensor” should be indifferent to the second part of the statistic and only impose rules on the first part, leading to

$$\kappa^{(A, y_A)}[Y_0, Y_1] = \epsilon_{y_0}[Y_0] \otimes \mathbb{I}[Y_1] = \begin{bmatrix} 0 \\ 1 \end{bmatrix} \otimes \begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

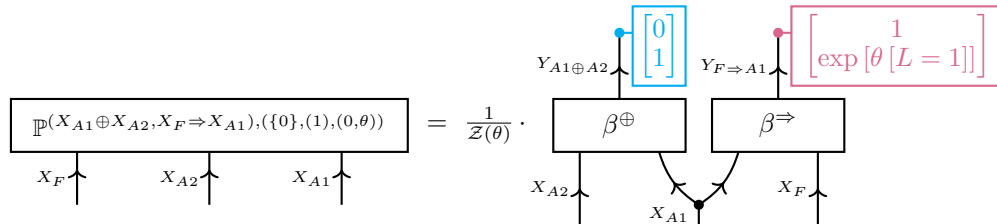
Since the first feature is hard, the “soft logic activation tensor” should be invariant under the first coordinate of the canonical parameter and we set $\theta[L = 0] = 0$. The soft parameters are chosen as $\theta[L] = [0, \theta[L = 1]]^\top$ to achieve

$$\alpha^\theta[Y_0, Y_1] = \alpha^{0,0}[Y_0] \otimes \alpha^{1, \theta[L=1]}[Y_1] = \begin{bmatrix} 1 \\ 1 \end{bmatrix} \otimes \begin{bmatrix} 1 \\ \exp[\theta[L = 1]] \end{bmatrix}.$$

The activation tensor of the hybrid network then has the form

$$\xi^{(\mathcal{U}, y_{\mathcal{U}}, \theta)}[Y_0, Y_1] = \begin{bmatrix} 0 \\ 1 \end{bmatrix} \otimes \begin{bmatrix} 1 \\ \exp[\theta[L = 1]] \end{bmatrix}.$$

We get the following tensor network representation of the Hybrid Logic Network representing the toy accounting example before normalization to a distribution



The resulting Hybrid Logic Network is a tensor $\mathbb{P}^{t, (\mathcal{U}, y_{\mathcal{U}}, \theta)}[X_{A1}, X_{A2}, X_F]$ of order 3. With $Y_{F \Rightarrow A1} = 1$ for $F = 0$ and any A_1 it has the coordinates

$$\mathbb{P}(X_{A1} \oplus X_{A2}, X_F \Rightarrow X_{A1}, (\{0\}, (1), (0, \theta)))[X_{A1}, X_{A2}, X_F] = \frac{1}{1 + 3 \cdot \exp[\theta]} \cdot X_{A1} \begin{array}{c} \begin{array}{cc} \begin{array}{c} 0 \\ \vdots \\ 1 \end{array} & \begin{array}{c} 0 \\ \vdots \\ 1 \end{array} \\ \begin{array}{c} 0 \\ \vdots \\ 1 \end{array} & \begin{array}{c} 0 \\ \vdots \\ 1 \end{array} \end{array} \begin{bmatrix} 0 & 1 \\ \exp[\theta] & 0 \end{bmatrix} \begin{array}{c} \begin{array}{c} 0 \\ \vdots \\ 1 \end{array} \\ \begin{array}{c} 0 \\ \vdots \\ 1 \end{array} \end{array}$$

6.2 Parameter estimation in Hybrid Logic Networks

Let us now briefly discuss how Hybrid Logic Networks can be trained on data based on likelihood maximization. Given a dataset $\left((x_0^j, \dots, x_{d-1}^j) : j \in [m] \right)$ consisting of m independent and identically distributed samples from an unknown distribution, we want to find a Hybrid Logic Network $\mathbb{P}^{t, (\mathcal{U}, y_{\mathcal{U}}, \theta)} [X_{[d]}]$ with a statistic $t = (f_0, \dots, f_{p-1})$ that minimizes the negative log likelihood

$$\mathcal{L}_D((\mathcal{U}, y_{\mathcal{U}}, \theta)) := -\frac{1}{m} \sum_{j \in [m]} \ln \left[\mathbb{P}^{t, (\mathcal{U}, y_{\mathcal{U}}, \theta)} [X_{[d]} = x_{[d]}^j] \right].$$

We can rewrite the loss using the empirical mean vector $\mu_D [L] \in \mathbb{R}^p$, which is defined for $\ell \in [p]$ as

$$\mu_D [L = \ell] = \frac{1}{m} \sum_{j \in [m]} f_{\ell} [X_{[d]} = x_{[d]}^j],$$

by

$$\mathcal{L}_D((\mathcal{U}, y_{\mathcal{U}}, \theta)) = \langle \mu_D [L], \theta [L] \rangle_{[\emptyset]} - \ln \left[\left\langle \xi^{(\mathcal{U}, y_{\mathcal{U}}, \theta)} [Y_{[p]}], \beta^t [Y_{[p]}, X_{[d]}] \right\rangle_{[\emptyset]} \right].$$

Since $(\mathcal{U}, y_{\mathcal{U}})$ influences only the second term, the best hard parameters can be found by

$$\mathcal{U} = \{\ell : \mu_D [L = \ell] \in \{0, 1\}\} \quad \text{and} \quad y_{\ell} = \mu_D [L = \ell] \quad \text{for} \quad \ell \in A.$$

We further optimize the coordinates $\ell \in [p] \setminus \mathcal{U}$ of $\theta [L] \in \mathbb{R}^p$ alternately by the coordinate descent steps

$$\frac{\partial \mathcal{L}_D((\mathcal{U}, y_{\mathcal{U}}, \theta))}{\partial \theta [L = \ell]} = 0 \Leftrightarrow \theta [L = \ell] = \ln \left[\frac{\mu [L = \ell]}{(1 - \mu [L = \ell])} \cdot \frac{\tau [Y_{\ell} = 0]}{\tau [Y_{\ell} = 1]} \right].$$

where

$$\tau [Y_{\ell}] = \left\langle \{ \beta^{f_{\tilde{\ell}}} : \tilde{\ell} \in [p] \} \cup \{ \alpha^{\tilde{\ell}, \theta} : \tilde{\ell} \in [p], \tilde{\ell} \neq \ell \} \cup \{ \nu \} \right\rangle_{[Y_{\ell}]}.$$

Based on an interpretation of the coordinate descent steps as matching steps for the mean parameters or moments to f_{ℓ} , we call this method *alternating moment matching* for Hybrid Logic Networks and provide pseudocode for it in Algorithm 4. We notice that, during the coordinate descent steps, computing the marginal probability of the variable Y_{ℓ} with respect to the current network parameters is required. This is the computational bottleneck of the algorithm and can be approached by various approximate inference methods, e.g., variational inference (see for example the CAMEL method Ganapathi et al. (2008)).

It can be shown that the algorithm converges if and only if there is a Hybrid Logic Network matching the empirical moments of the data. For more details we refer to (Goessmann, 2025, Chapter 9).

Example 24 (Continuation of Example 23). Recall the statistic of Example 23 and consider a dataset of $m = 20$ states summarized in the frequency table:

Frequency in Dataset	x_{A1}	x_{A2}	x_F
0	0	0	0
0	0	0	1
7	0	1	0
2	0	1	1
1	1	0	0
10	1	0	1
0	1	1	0
0	1	1	1

We then have for the satisfaction rates of $f_0 = X_{A1} \oplus X_{A2}$ and $f_1 = X_F \Rightarrow X_{A1}$ that

$$\mu_D [L = 0] = \frac{20}{20} = 1 \quad \text{and} \quad \mu_D [L = 1] = \frac{7 + 1 + 10}{20} = 0.9.$$

Algorithm 4 yields a reasonable convergence criterion choice (such as finite iterations or convergence of $\theta [L]$)

$$\mathcal{U} = \{0\} \quad , \quad y_{\mathcal{U}} = 1 \quad \text{and} \quad \theta [L] = \left[\ln \left[\left(\frac{0}{0.1} \right) \cdot \left(\frac{1}{3} \right) \right] \right] = \left[\ln [3] \right] \approx \left[\frac{0}{1.098612} \right].$$

Algorithm 4 Alternating Moment Matching for Hybrid Logic Networks

Require: Mean parameter $\mu_D [L]$ **Ensure:** Parameters $(\mathcal{U}, y_{\mathcal{U}}, \theta)$ for the approximating HLN $\mathbb{P}^{(t, \theta, \nu)}$

Set

$$\mathcal{U} = \left\{ \ell : \ell \in [p], \mu [L = \ell] \in \{0, 1\} \right\}$$

and a tuple $y_{\mathcal{U}}$ with $y_{\ell} = \mu [L = \ell]$ for $\ell \in \mathcal{U}$.Set $\theta [L] = 0 [L]$ **while** Convergence criterion is not met **do** **for all** $\ell \in [p] \setminus \mathcal{U}$ **do**

Compute

$$\tau [Y_{\ell}] = \left\langle \{ \beta^{f_{\tilde{\ell}}} : \tilde{\ell} \in [p] \} \cup \{ \alpha^{\tilde{\ell}, \theta} : \tilde{\ell} \in [p], \tilde{\ell} \neq \ell \} \cup \{ \nu \} \right\rangle_{[Y_{\ell}]}$$

Set

$$\theta [L = \ell] = \ln \left[\frac{\mu [L = \ell]}{(1 - \mu [L = \ell])} \cdot \frac{\tau [Y_{\ell} = 0]}{\tau [Y_{\ell} = 1]} \right]$$

end for**end while****return** $(\mathcal{U}, y_{\mathcal{U}}, \theta [L])$

To derive this, we notice that Algorithm 4 treats formula f_0 as a hard constraint and assigns $\mathcal{U} = \{0\}$ and $y_{\mathcal{U}} = 1$. In the while loop we then have for the formula f_1

$$\tau [Y_1] = \left\langle \epsilon_1 [Y_0], \beta^{f_0} [Y_0, X_F, X_{A1}, X_{A2}], \beta^{f_1} [Y_1, X_F, X_{A1}, X_{A2}] \right\rangle_{[Y_1]} = \begin{bmatrix} 1 \\ 3 \end{bmatrix}$$

since f_0 has 4 models, of which 3 are also models of f_1 and 1 is instead a model of $\neg f_1$. Notice, that the tensor $\tau [Y_1]$ will not change in any further iteration of the while and the parameter $\theta [L = 1]$ will therefore stay constant until the termination of the algorithm.

6.3 Entailment by Hybrid Logic Networks

Let us now demonstrate a further use of our unified treatment of probabilistic and logical models by investigating a generalized concept of entailment. Entailment can be generalized to probabilistic models by deciding whether a propositional formula is always satisfied given a probabilistic model.

Theorem 10. Let $\mathbb{P}^{t, (\mathcal{U}, y_{\mathcal{U}}, \theta)} [X_{[d]}]$ be a Hybrid Logic Network and $h [X_{[d]}]$ a propositional formula. Then $\mathbb{P}^{t, (\mathcal{U}, y_{\mathcal{U}}, \theta)}$ probabilistically entails h , that is,

$$\left\langle \mathbb{P}^{t, (\mathcal{U}, y_{\mathcal{U}}, \theta)} [X_{[d]}], h [X_{[d]}] \right\rangle_{[\emptyset]} = 1,$$

if and only if

$$f^{t, (\mathcal{U}, y_{\mathcal{U}})} \models h,$$

where

$$f^{t, (\mathcal{U}, y_{\mathcal{U}})} [X_{[d]}] = \left(\bigwedge_{\ell \in \mathcal{U} : y_{\ell} = 1} f_{\ell} [X_{[d]}] \right) \wedge \left(\bigwedge_{\ell \in \mathcal{U} : y_{\ell} = 0} \neg f_{\ell} [X_{[d]}] \right).$$

Proof. We have

$$\left\langle \mathbb{P}^{t, (\mathcal{U}, y_{\mathcal{U}}, \theta)} [X_{[d]}], h [X_{[d]}] \right\rangle_{[\emptyset]} = 1$$

if and only if

$$\left\langle \mathbb{P}^{t,(\mathcal{U},y_{\mathcal{U}},\theta)} [X_{[d]}], h [X_{[d]}] - \mathbb{I} [X_{[d]}] \right\rangle_{[\emptyset]} = 0$$

which is equal to

$$\left\langle \mathbb{P}^{t,(\mathcal{U},y_{\mathcal{U}},\theta)} [X_{[d]}], \neg h [X_{[d]}] \right\rangle_{[\emptyset]} = 0.$$

Since $\mathbb{P}^{t,(\mathcal{U},y_{\mathcal{U}},\theta)} [X_{[d]}]$ is non-negative this is equivalent to

$$\left\langle \mathbb{I}_{\neq 0} \left(\mathbb{P}^{t,(\mathcal{U},y_{\mathcal{U}},\theta)} [X_{[d]}] \right), \neg h [X_{[d]}] \right\rangle_{[\emptyset]} = 0.$$

We use that

$$\mathbb{I}_{\neq 0} \left(\mathbb{P}^{t,(\mathcal{U},y_{\mathcal{U}},\theta)} [X_{[d]}] \right) = f^{t,(\mathcal{U},y_{\mathcal{U}})} [X_{[d]}]$$

and get that this is further equivalent to

$$\left\langle f^{t,(\mathcal{U},y_{\mathcal{U}})} [X_{[d]}], \neg h [X_{[d]}] \right\rangle_{[\emptyset]} = 0,$$

which is by Def. 20 $f^{t,(\mathcal{U},y_{\mathcal{U}})} \models h$. □

Example 25 (Continuation of Example 24). *Consider again the Hybrid Logic Network*

$$\mathbb{P}^{(X_{A1} \oplus X_{A2}, X_F \Rightarrow X_{A1}), (\{0\}, (1), (0, \ln[3]))} [X_{A1}, X_{A2}, X_F]$$

from Example 24 and assume we want to decide the probabilistic entailment of the formula

$$h [X_{A1}, X_{A2}, X_F] = \neg X_{A1} \vee \neg X_{A2} \vee \neg X_F,$$

which has all states but $(1, 1, 1)$ as a model (and is therefore referred to as a maxterm). Using Thm. 10 we have that

$$\left\langle \mathbb{P}^{(X_{A1} \oplus X_{A2}, X_F \Rightarrow X_{A1}), (\{0\}, (1), (0, \ln[3]))} [X_{A1}, X_{A2}, X_F], h [X_{A1}, X_{A2}, X_F] \right\rangle_{[\emptyset]} = 1$$

if and only if $X_{A1} \oplus X_{A2} \models \neg X_{A1} \vee \neg X_{A2} \vee \neg X_F$. By Def. 20, this entailment holds since by the De-Morgan rule

$$\begin{aligned} \langle X_{A1} \oplus X_{A2}, \neg(\neg X_{A1} \vee \neg X_{A2} \vee \neg X_F) \rangle_{[\emptyset]} &= \langle X_{A1} \oplus X_{A2}, X_{A1}, X_{A2}, X_F \rangle_{[\emptyset]} \\ &= \langle X_F \rangle_{[\emptyset]} \cdot \langle X_{A1} \oplus X_{A2}, X_{A1}, X_{A2} \rangle_{[\emptyset]} \\ &= 0. \end{aligned}$$

We thus conclude, that h is probabilistically entailed by $\mathbb{P}^{(X_{A1} \oplus X_{A2}, X_F \Rightarrow X_{A1}), (\{0\}, (1), (0, \ln[3]))}$.

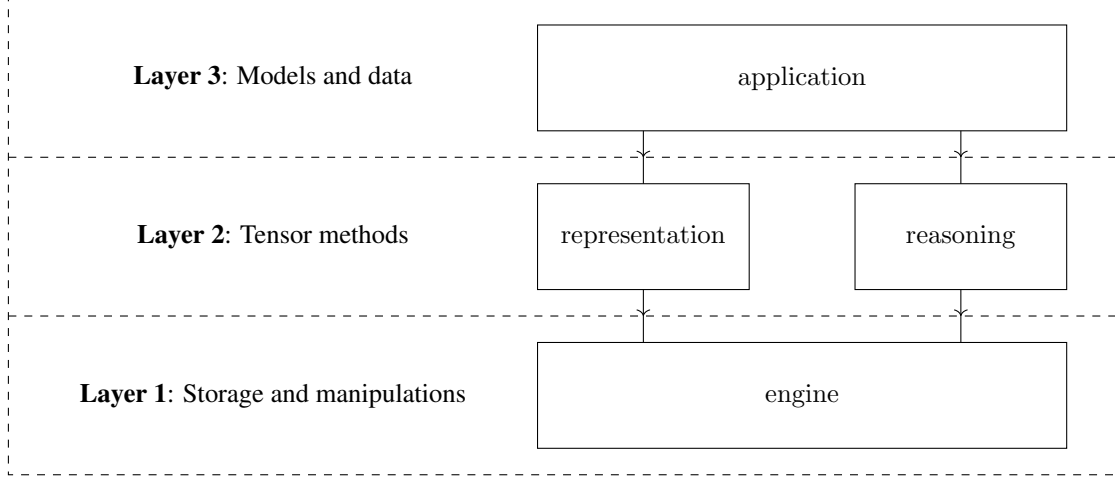
7 Implementation in the python library tnreason

The concepts presented in this paper have been implemented in the open source python library `tnreason`³. In this section, we explain the basic design and functionality of this library and draw close connections to the theoretical exposition in the previous sections. In particular, Appendix B provides detailed implementations of the algorithms and examples in this work.

7.1 Architecture

The package consists of four subpackages and three layers of abstraction:

³`tnreason` is available in version 2.0.0 at pypi.org/tnreason and maintained at github.com/tnreason/tnreason-py.



In the subpackage `tnreason.engine` we implement tensors, tensor networks, contractions, and normalizations. In the subpackage `tnreason.representation` the basic tensor encoding schemes such as basis encodings are available. In the subpackage `tnreason.reasoning` we implement reasoning algorithms, such as generalizations of the message passing algorithms presented in Algorithm 1, Algorithm 3, and Algorithm 4. In the subpackage `tnreason.application` one can construct tensor network encodings of propositional formulas and datasets.

7.2 Basic usage

We demonstrate the basic usage of the `tnreason` package with the implementation of Example 15. We first install the package (e.g. by `pip install tnreason == 2.0.0`) and import it by

```
from tnreason import engine, application
```

Keeping Def. 1 in mind, the tensor instances in `shape` and `colors` arguments are `list` instances specifying the `int` dimension m_k and a `str` identifier for X_k . The formula in Example 15 is a sum of the one-hot encodings of its three models (see Example 16) and is created by

```
formula = engine.create_from_slice_iterator(shape=[2,2,2],
→ colors=["X_0", "X_1", "X_2"], sliceIterator=[(1, {"X_0":0, "X_1":1, "X_2":0}),
→ (1, {"X_0":1, "X_1":0, "X_2":0}), (1, {"X_0":1, "X_1":1, "X_2":0})])
```

The slice iterator is an iterator over tuples $(val, posDict)$, which specifies elementary tensors to be summed. The `posDict` are `dict` instances, where the keys are the `str` tensor colors and the values are `int`. Each `posDict` collects leg vectors of the corresponding elementary tensor that are not trivial. These leg vectors are the basis vectors enumerated by the corresponding `int` value.

Single tensor coordinates can be retrieved by indexing with a `posDict`. We can, for example, check whether `{"X_0":0, "X_1":1, "X_2":0}` is a model:

```
assert formula[{"X_0":0, "X_1":1, "X_2":0}] == 1
```

By default the tensor is created as a `engine.NumpyCore` instance, where coordinates are stored as instances of `numpy.array`. Further core types exploiting different sparsity principles can be chosen by the argument `coreType`, see (Goessmann, 2025, Appendix A).

Following Def. 2, tensor networks are implemented as tensor valued `dict` instances with `str` keys. For example a tensor network is created from the propositional syntax of the above formula (see Example 20):

```
fDecomp = application.create_cores_to_expressionsDict({"f0":
→ ["and", ["or", "X_0", "X_1"], ["not", "X_2"]]])
```

Here we apply a nested-list description of decomposition hypergraphs (see Def. 17) with a specification of the logical connectives in the first position of the list (by `"and"`, `"or"`, `"not"` we refer to the connectives $\wedge$, $\vee$, $\neg$). Equivalently, we can exploit the $\wedge$ symmetry and create it by multiple formulas:

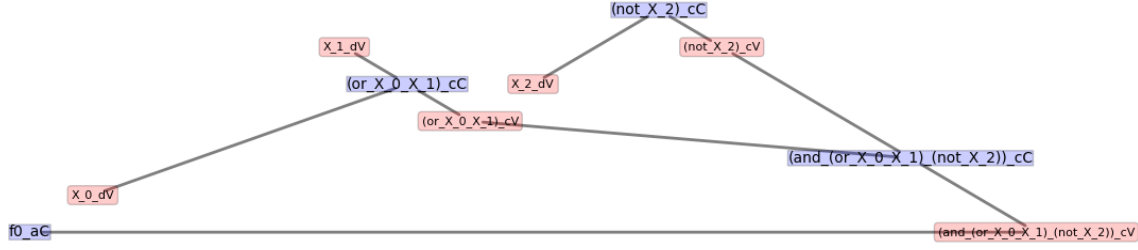


Figure 14: Factor graph highlighting a tensor network decomposition of the syntactic decomposition of the propositional formula of Example 20. Blue blocks highlight hyperedges carrying tensors and red blocks highlight variables. The tensor label suffixes "`_cC`" and "`_aC`" indicate whether the tensor is part of the computation network or the activation network. The variable label suffixes "`_dV`" and "`_cV`" indicate whether the variable is distributed or computed and therefore auxiliary. This graph has been generated with the method `tnreason.engine.draw_factor_graph` of `tnreason`.

```
fDecomp = application.create_cores_to_expressionsDict({"f0":
→ ["or", "X_0", "X_1"], "f1": ["not", "X_2"]})
```

A depiction of the underlying hypergraph as a factor graph, which highlights edges as blue blocks and nodes as red blocks, can be created with `engine.draw_factor_graph(fDecomp)` (see Figure 14). Single tensors can be obtained by contracting a tensor network while specifying the open variables (for an explanation of the suffixes, see Figure 14), for example:

```
contracted = engine.contract(fDecomp, openColors=["X_0_dV", "X_1_dV", "X_2_dV"])
```

By default the contractions are performed using `numpy.einsum` and further execution schemes can be selected with the argument `contractionMethod`, see (Goessmann, 2025, Appendix A).

8 Conclusion & outlook

This work developed a tensor network formalism to capture in a unifying way the main concepts of AI, which build the core of the probabilistic, neural and logical approaches. We introduced Computation-Activation Networks (CompActNets) as a generic architecture to represent classes of propositional knowledge bases, graphical models and more generic exponential families. Moreover, we demonstrate the representation and training of hybrid models combining logical and probabilistic aspects, illustrating that CompActNets represent a powerful, versatile and mathematically grounded framework for Neuro-Symbolic AI.

We have shown that model inference such as the calculating marginal distributions and deciding entailment correspond with tensor network contractions. To efficiently perform these inferences, we presented message passing schemes, which have been shown to be exact in specific cases. In general, however, the efficient computation of contractions is not possible, since they are related to the NP-hardness of probabilistic inferences in graphical models (see Koller and Friedman (2009)) and of logical reasoning (see Russell and Norvig (2021)). In cases where exact inference is not feasible, the derivation of error bounds for approximate inference schemes on CompActNets is an interesting direction for future research.

Further approximation schemes to overcome this bottleneck are summarized under the umbrella of variational inference (see Wainwright and Jordan (2008)), such as generic expectation-propagation methods or mean field methods. While these schemes are developed either for graphical models or more general exponential families, we plan to derive similar methods for more general CompActNets, such as Hybrid Logic Networks. Further frequently applied schemes are particle-based inference schemes such as Gibbs sampling.

The integration of symbolic and neural methods is an active research area (see Colelough and Regli (2024) for a systematic review). The CompActNets framework enables both, symbolic logical as well as probabilistic models, but enables also the representation of generic functions. CompActNets based on architectures combining symbolically verbalizable and more generic neural parts are thus a promising direction for Neuro-Symbolic AI.

The CompActNets framework offers an immediate practical application as a verifiable reasoning engine for AI agents in high-stakes domains such as regulatory compliance, clinical decision support, accounting, process planning and

security. By leveraging the framework’s inherent flexibility, Large Language Models (see Vaswani et al. (2017)) can be adapted to function as semantic translators that dynamically construct problem-specific tensor networks in the form of CompActNets from natural language descriptions, effectively treating the reasoning engine as an external tool. This approach mitigates the hallucination risks of probabilistic models by delegating complex logical execution to the exact linear algebra of the tensor network, ensuring that the inference process is both rigorous and reproducible. Consequently, this synergy enables the deployment of reliable AI systems where the intuitive power of the Large Language Model is grounded by the explainable, instance-adaptive topology of the CompActNets.

Acknowledgements

AG and ME acknowledge funding from the German Federal Ministry of Education and Research (BMBF), grant number FKZ 13N17160, "Verbundprojekt: Quantum Read-Once-Memory - Verwandlung von klassischen Daten zu Quantenzuständen - Teilvorhaben: Quantenschaltkreis-Optimierung von Quantenzuständen durch Tensor-Netzwerke (QOQ-tn)". JS and ME acknowledge funding from the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) under Germany’s Excellence Strategy – The Berlin Mathematics Research Center MATH+ (EXC-2046/2, project ID: 390685689, Project PaA-7). MF acknowledges funding from Einstein Research Unit on Quantum Devices.

References

- Ian Affleck, Tom Kennedy, Elliott H. Lieb, and Hal Tasaki. Rigorous results on valence-bond ground states in antiferromagnets. *Physical Review Letters*, 59(7):799–802, August 1987. doi: 10.1103/PhysRevLett.59.799. URL <https://link.aps.org/doi/10.1103/PhysRevLett.59.799>.
- Alejandro Mata Ali. Explicit Solution Equation for Every Combinatorial Problem via Tensor Networks: MeLoCoToN, February 2025. URL <http://arxiv.org/abs/2502.05981>.
- Artur S. Avila Garcez and Gerson Zaverucha. The Connectionist Inductive Learning and Logic Programming System. *Applied Intelligence*, 11(1):59–77, July 1999. ISSN 1573-7497. doi: 10.1023/A:1008328630915. URL <https://doi.org/10.1023/A:1008328630915>.
- Samy Badreddine, Artur d’Avila Garcez, Luciano Serafini, and Michael Spranger. Logic Tensor Networks. *Artificial Intelligence*, 303:103649, February 2022. ISSN 0004-3702. doi: 10.1016/j.artint.2021.103649. URL <https://www.sciencedirect.com/science/article/pii/S0004370221002009>.
- Alejandro Barredo Arrieta, Natalia Díaz-Rodríguez, Javier Del Ser, Adrien Bennetot, Siham Tabik, Alberto Barbado, Salvador Garcia, Sergio Gil-Lopez, Daniel Molina, Richard Benjamins, Raja Chatila, and Francisco Herrera. Explainable Artificial Intelligence (XAI): Concepts, taxonomies, opportunities and challenges toward responsible AI. *Information Fusion*, 58:82–115, June 2020. ISSN 1566-2535. doi: 10.1016/j.inffus.2019.12.012. URL <https://www.sciencedirect.com/science/article/pii/S1566253519308103>.
- A. V. Berezutskii, I. A. Luchnikov, and A. K. Fedorov. Simulating quantum circuits using the multi-scale entanglement renormalization ansatz. *Physical Review Research*, 7(1):013063, January 2025. doi: 10.1103/PhysRevResearch.7.013063. URL <https://link.aps.org/doi/10.1103/PhysRevResearch.7.013063>.
- Lawrence D. Brown. *Fundamentals of Statistical Exponential Families*. Institute of mathematical Statistics, Hayward, Calif, June 1987. ISBN 978-0-940600-10-2.
- George Casella and Roger Berger. *Statistical Inference*. Cengage Learning, Pacific Grove, Calif, June 2001. ISBN 978-0-534-24312-8.
- P. Clifford and J. M. Hammersley. Markov fields on finite graphs and lattices. *Unpublished*, 1971. URL <https://ora.ox.ac.uk/objects/uuid:4ea849da-1511-4578-bb88-6a8d02f457a6>.
- Brandon C Colelough and William Regli. Neuro-Symbolic AI in 2024: A Systematic Review. Jeju, South Korea, 2024.
- Thomas M. Cover and Joy A. Thomas. *Elements of Information Theory*. Wiley-Interscience, Hoboken, N.J, 2nd edition edition, September 2006. ISBN 978-0-471-24195-9.
- Tiangang Cui and Sergey Dolgov. Deep Composition of Tensor-Trains Using Squared Inverse Rosenblatt Transports. *Foundations of Computational Mathematics*, 22(6):1863–1922, December 2022. ISSN 1615-3383. doi: 10.1007/s10208-021-09537-5. URL <https://doi.org/10.1007/s10208-021-09537-5>.
- Sergey Dolgov and Robert Scheichl. A Hybrid Alternating Least Squares–TT-Cross Algorithm for Parametric PDEs. *SIAM/ASA Journal on Uncertainty Quantification*, March 2019. doi: 10.1137/17M1138881. URL <https://epubs.siam.org/doi/10.1137/17M1138881>.

- Sergey Dolgov, Boris N. Khoromskij, Alexander Litvinenko, and Hermann G. Matthies. Polynomial Chaos Expansion of Random Coefficients and the Solution of Stochastic Partial Differential Equations in the Tensor Train Format. *SIAM/ASA Journal on Uncertainty Quantification*, November 2015. doi: 10.1137/140972536. URL <https://epubs.siam.org/doi/10.1137/140972536>.
- Sergey Dolgov, Dante Kalise, and Luca Saluzzi. Data-Driven Tensor Train Gradient Cross Approximation for Hamilton–Jacobi–Bellman Equations. *SIAM Journal on Scientific Computing*, September 2023. doi: 10.1137/22M1498401. URL <https://epubs.siam.org/doi/10.1137/22M1498401>.
- Pedro Domingos. Tensor Logic: The Language of AI, October 2025. URL <http://arxiv.org/abs/2510.12269>. arXiv:2510.12269 [cs].
- Martin Eigel, Max Pfeffer, and Reinhold Schneider. Adaptive stochastic Galerkin FEM with hierarchical tensor representations. *Numerische Mathematik*, 136(3):765–803, July 2017. ISSN 0945-3245. doi: 10.1007/s00211-016-0850-x. URL <https://doi.org/10.1007/s00211-016-0850-x>.
- Martin Eigel, Reinhold Schneider, Philipp Trunschke, and Sebastian Wolf. Variational Monte Carlo—bridging concepts of machine learning and high-dimensional partial differential equations. *Advances in Computational Mathematics*, 45(5):2503–2532, December 2019. ISSN 1572-9044. doi: 10.1007/s10444-019-09723-8. URL <https://doi.org/10.1007/s10444-019-09723-8>.
- Martin Eigel, Manuel Marschall, Max Pfeffer, and Reinhold Schneider. Adaptive stochastic Galerkin FEM for lognormal coefficients in hierarchical tensor representations. *Numerische Mathematik*, 145(3):655–692, July 2020. ISSN 0945-3245. doi: 10.1007/s00211-020-01123-1. URL <https://doi.org/10.1007/s00211-020-01123-1>.
- Martin Eigel, Reinhold Schneider, and David Sommer. Dynamical low-rank approximations of solutions to the Hamilton–Jacobi–Bellman equation. *Numerical Linear Algebra with Applications*, 30(3):e2463, 2023. ISSN 1099-1506. doi: 10.1002/nla.2463. URL <https://onlinelibrary.wiley.com/doi/abs/10.1002/nla.2463>. _eprint: <https://onlinelibrary.wiley.com/doi/pdf/10.1002/nla.2463>.
- Varun Ganapathi, David Vickrey, John Duchi, and Daphne Koller. Constrained approximate maximum entropy learning of Markov random fields. In *Proceedings of the Twenty-Fourth Conference on Uncertainty in Artificial Intelligence*, UAI’08, pages 196–203, Arlington, Virginia, USA, July 2008. AUAI Press. ISBN 978-0-9749039-4-1.
- Lise Getoor and Ben Taskar. *Introduction to Statistical Relational Learning*. MIT Press, September 2019. ISBN 978-0-262-53868-8.
- Ivan Glasser, Ryan Sweke, Nicola Pancotti, Jens Eisert, and Ignacio Cirac. Expressive power of tensor-network factorizations for probabilistic modeling. *Advances in Neural Information Processing Systems*, 32, 2019.
- Alex Goessmann. *Uniform Concentration of Tensor and Neural Networks: An Approach towards Recovery Guarantees*. PhD Thesis, Technische Universität Berlin, Berlin, 2021. URL <https://depositonce.tu-berlin.de/handle/11303/15990>.
- Alex Goessmann. The Tensor-Network Approach to Efficient and Explainable AI, 2025. URL <https://github.com/EnexaProject/enexa-tensor-reasoning-documentation/>.
- Alex Goessmann, Ingo Roth, Gitta Kutyniok, Michael Götte, Ryan Sweke, and Jens Eisert. Tensor network approaches for data-driven identification of non-linear dynamical laws. In *Advances in Neural Information Processing Systems - First Workshop on Quantum Tensor Networks in Machine Learning*, page 21, 2020.
- Nikita Gourianov, Peyman Givi, Dieter Jaksch, and Stephen B. Pope. Tensor networks enable the calculation of turbulence probability distributions. *Science Advances*, 11(5):eads5990, January 2025. doi: 10.1126/sciadv.ads5990. URL <https://www.science.org/doi/10.1126/sciadv.ads5990>.
- Robert Gruhlke, David Sommer, Max Kirstein, Martin Eigel, and Claudia Schillings. Reverse Diffusion Sampling with Tensor Train Approximations of Hamilton–Jacobi–Bellman Equations. *SIAM Journal on Scientific Computing*, January 2026. doi: 10.1137/24M1637210. URL <https://epubs.siam.org/doi/10.1137/24M1637210>.
- W. Hackbusch and S. Kühn. A New Scheme for the Tensor Representation. *Journal of Fourier Analysis and Applications*, 15(5):706–722, October 2009. ISSN 1531-5851.
- Wolfgang Hackbusch. *Tensor Spaces and Numerical Tensor Calculus*. Springer Series in Computational Mathematics. Springer-Verlag, Berlin Heidelberg, 2012. ISBN 978-3-642-28026-9. doi: 10.1007/978-3-642-28027-6.
- Paul Hagemann, Janina Schütte, David Sommer, Martin Eigel, and Gabriele Steidl. Sampling from Boltzmann Densities with Physics Informed Low-Rank Formats. In Tatiana A. Bubba, Romina Gaburro, Silvia Gazzola, Kostas Papafitsoros, Marcelo Pereyra, and Carola-Bibiane Schönlieb, editors, *Scale Space and Variational Methods in Computer Vision*, pages 374–386. Springer Nature Switzerland, Cham, 2025. ISBN 978-3-031-92366-1.

- Frank L. Hitchcock. The Expression of a Tensor or a Polyadic as a Sum of Products. *Journal of Mathematics and Physics*, 6(1-4):164–189, 1927. ISSN 1467-9590. doi: <https://doi.org/10.1002/sapm192761164>.
- Sepp Hochreiter. Toward a broad AI. *Communications of the ACM*, 65(4):56–57, January 2022. ISSN 0001-0782, 1557-7317. doi: 10.1145/3512715. URL <https://dl.acm.org/doi/10.1145/3512715>.
- Daphne Koller and Nir Friedman. *Probabilistic Graphical Models: Principles and Techniques*. The MIT Press, Cambridge, Mass., 1. edition edition, July 2009. ISBN 978-0-262-01319-2.
- Zachary C. Lipton. The mythos of model interpretability. *Commun. ACM*, 61(10):36–43, September 2018. ISSN 0001-0782. doi: 10.1145/3233231. URL <https://dl.acm.org/doi/10.1145/3233231>.
- Christian Lubich, Thorsten Rohwedder, Reinhold Schneider, and Bart Vandereycken. Dynamical Approximation by Hierarchical Tucker and Tensor-Train Tensors. *SIAM Journal on Matrix Analysis and Applications*, May 2013. doi: 10.1137/120885723. URL <https://epubs.siam.org/doi/10.1137/120885723>.
- Alan K. Mackworth. Consistency in networks of relations. *Artificial Intelligence*, 8(1):99–118, February 1977. ISSN 0004-3702. doi: 10.1016/0004-3702(77)90007-8. URL <https://www.sciencedirect.com/science/article/pii/0004370277900078>.
- Giuseppe Marra, Sebastijan Dumančić, Robin Manhaeve, and Luc De Raedt. From statistical relational to neurosymbolic artificial intelligence: A survey. *Artificial Intelligence*, 328:104062, March 2024. ISSN 0004-3702. doi: 10.1016/j.artint.2023.104062. URL <https://www.sciencedirect.com/science/article/pii/S0004370223002084>.
- John McCarthy. Programs with Common Sense. In *Proceedings of the Teddington Conference on the Mechanization of Thought Processes*, pages 75–91. Her Majesty’s Stationary Office, London, 1959. URL <http://www-formal.stanford.edu/jmc/mcc59.html>.
- Kevin P. Murphy. *Probabilistic Machine Learning: An Introduction*. The MIT Press, Cambridge, Massachusetts London, England, March 2022. ISBN 978-0-262-04682-4.
- Marc Mézard. *Information, Physics, and Computation*. ACADEMIC, Oxford ; New York, March 2009. ISBN 978-0-19-857083-7.
- Maximilian Nickel, Kevin Murphy, Volker Tresp, and Evgeniy Gabrilovich. A Review of Relational Machine Learning for Knowledge Graphs. *Proceedings of the IEEE*, 104(1):11–33, January 2016. ISSN 0018-9219, 1558-2256. doi: 10.1109/JPROC.2015.2483592. URL <https://ieeexplore.ieee.org/document/7358050/>.
- Román Orús. Tensor networks for complex quantum systems. *Nature Reviews Physics*, 1(9):538–550, September 2019. ISSN 2522-5820. doi: 10.1038/s42254-019-0086-7.
- I. V. Oseledets. Tensor-Train Decomposition. *SIAM Journal on Scientific Computing*, 33(5):2295–2317, January 2011. ISSN 1064-8275. doi: 10.1137/090752286. URL <https://epubs.siam.org/doi/10.1137/090752286>.
- Judea Pearl. *Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference*. Morgan Kaufmann, s.l., September 1988. ISBN 978-1-55860-479-7.
- Judea Pearl. *Causality: Models, Reasoning and Inference*. Ausgezeichnet: ACM Turing Award for Transforming Artificial Intelligence 2011. Cambridge University Press, Cambridge New York, NY Port Melbourne New Delhi Singapore, 2 edition, November 2009. ISBN 978-0-521-89560-6.
- Roger Penrose. *Spinors and Space-Time: Volume 1, Two-Spinor Calculus and Relativistic Fields*. Cambridge University Press, Cambridge, February 1987. ISBN 978-0-521-33707-6.
- E. J. G. Pitman. Sufficient statistics and intrinsic accuracy. *Mathematical Proceedings of the Cambridge Philosophical Society*, 32(4):567–579, December 1936. ISSN 1469-8064, 0305-0041. doi: 10.1017/S0305004100019307. URL <https://www.cambridge.org/core/journals/mathematical-proceedings-of-the-cambridge-philosophical-society/article/abs/sufficient-statistics-and-intrinsic-accuracy/6A3E45FB1C423F3F684308F8910D6919>.
- Matthew Richardson and Pedro Domingos. Markov logic networks. *Machine Learning*, 62(1-2):107–136, February 2006. ISSN 0885-6125, 1573-0565. doi: 10.1007/s10994-006-5833-1. URL <http://link.springer.com/10.1007/s10994-006-5833-1>.
- Elina Robeva and Anna Seigal. Duality of graphical models and tensor networks. *Information and Inference: A Journal of the IMA*, 8(2):273–288, June 2019. ISSN 2049-8772. doi: 10.1093/imaiai/iay009.
- Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach, Global Edition: A Modern Approach, Global Edition*. Pearson, Boston, 4 edition, May 2021. ISBN 978-1-292-40113-3.

- Aaron Sander, Maximilian Fröhlich, Mazen Ali, Martin Eigel, Jens Eisert, Michael Hintermüller, Christian B. Mendl, Richard M. Milbradt, and Robert Wille. Quantum circuit simulation with a local time-dependent variational principle, August 2025a. URL <http://arxiv.org/abs/2508.10096>. arXiv:2508.10096 [quant-ph].
- Aaron Sander, Maximilian Fröhlich, Martin Eigel, Jens Eisert, Patrick Gelß, Michael Hintermüller, Richard M. Milbradt, Robert Wille, and Christian B. Mendl. Large-scale stochastic simulation of open quantum systems. *Nature Communications*, 16(1):11074, December 2025b. ISSN 2041-1723. doi: 10.1038/s41467-025-66846-x. URL <http://arxiv.org/abs/2501.17913>. arXiv:2501.17913 [quant-ph].
- Md Kamruzzaman Sarker, Lu Zhou, Aaron Eberhart, and Pascal Hitzler. Neuro-symbolic artificial intelligence: Current trends. *AI Communications*, 34(3):197–209, March 2022. ISSN 18758452, 09217126. doi: 10.3233/AIC-210084. URL <https://www.medra.org/servlet/aliasResolver?alias=iospress&doi=10.3233/AIC-210084>.
- Geoffrey G. Towell and Jude W. Shavlik. Knowledge-based artificial neural networks. *Artificial Intelligence*, 70(1):119–165, October 1994. ISSN 0004-3702. doi: 10.1016/0004-3702(94)90105-8. URL <https://www.sciencedirect.com/science/article/pii/0004370294901058>.
- Philipp Trunschke, Martin Eigel, and Anthony Nouy. Weighted sparsity and sparse tensor networks for least squares approximation. *The SMAI Journal of computational mathematics*, 11:289–333, 2025. ISSN 2426-8399. doi: 10.5802/smai-jcm.126. URL <https://smai-jcm.centre-mersenne.org/articles/10.5802/smai-jcm.126/>.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is All you Need. In *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017. URL https://proceedings.neurips.cc/paper_files/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html.
- Martin J. Wainwright and Michael Irwin Jordan. *Graphical Models, Exponential Families, and Variational Inference*. Now Publishers Inc, 2008. ISBN 978-1-60198-184-4.
- Steven R. White. Density-matrix algorithms for quantum renormalization groups. *Physical Review B*, 48(14):10345–10356, October 1993. doi: 10.1103/PhysRevB.48.10345. URL <https://link.aps.org/doi/10.1103/PhysRevB.48.10345>.

A Proof of the Factorization Theorems

We now provide proofs for the factorization theorems stated in Sect. 3. These proofs are classically known (see e.g. Koller and Friedman (2009) for Hammersley-Clifford and Casella and Berger (2001) for Fisher-Neyman). They are here provided in our tensor networks notation and for hypergraphs for completeness.

A.1 Hammersley-Clifford

Different to the original statement (see Clifford and Hammersley (1971)), we here proof the analogous statement for hypergraphs, where we have to demand the property of clique-capturing defined in Def. 14. We start with showing the following Lemmata to be exploited in the proof.

Lemma 4. *Let $\tau[X_{\mathcal{V}}]$ be a positive tensor. Then we have for any index $y_{\mathcal{V}}$*

$$\tau[X_{\mathcal{V}}] = \left\langle \left(\langle \tau \rangle_{[X_{\mathcal{V} \setminus \mathcal{W}}, X_{\mathcal{W}} = y_{\mathcal{W}}]} \right)^{(-1)^{|\mathcal{U}| - |\mathcal{W}|}} : \mathcal{W} \subset \mathcal{U} \subset \mathcal{V} \right\rangle_{[X_{\mathcal{V}}]},$$

where the exponentiation is performed coordinatewise and positivity of τ ensures the well-definedness.

Proof. It suffices to show, that for an arbitrary index $x_{\mathcal{V}}$ we have

$$\tau[X_{\mathcal{V}} = x_{\mathcal{V}}] = \prod_{\mathcal{U} \subset \mathcal{V}} \prod_{\mathcal{W} \subset \mathcal{U}} \left(\langle \tau \rangle_{[X_{\mathcal{V} \setminus \mathcal{W}} = x_{\mathcal{V} \setminus \mathcal{W}}, X_{\mathcal{W}} = y_{\mathcal{W}}]} \right)^{(-1)^{|\mathcal{U}| - |\mathcal{W}|}}.$$

We do this by applying a logarithm on the right hand side and grouping the terms by $\mathcal{W}$ as

$$\begin{aligned} & \ln \left[\prod_{\mathcal{U} \subset \mathcal{V}} \prod_{\mathcal{W} \subset \mathcal{U}} \langle \tau \rangle_{[X_{\mathcal{V} \setminus \mathcal{W}} = x_{\mathcal{V} \setminus \mathcal{W}}, X_{\mathcal{W}} = y_{\mathcal{W}}]} \right]^{(-1)^{|\mathcal{U}| - |\mathcal{W}|}} \\ &= \sum_{\mathcal{W} \subset \mathcal{V}} \ln \left[\langle \tau \rangle_{[X_{\mathcal{V} \setminus \mathcal{W}} = x_{\mathcal{V} \setminus \mathcal{W}}, X_{\mathcal{W}} = y_{\mathcal{W}}]} \right] \left(\sum_{\mathcal{U} \subset \mathcal{V} : \mathcal{W} \subset \mathcal{U}} (-1)^{|\mathcal{U}| - |\mathcal{W}|} \right) \\ &= \sum_{\mathcal{W} \subset \mathcal{V}} \ln \left[\langle \tau \rangle_{[X_{\mathcal{V} \setminus \mathcal{W}} = x_{\mathcal{V} \setminus \mathcal{W}}, X_{\mathcal{W}} = y_{\mathcal{W}}]} \right] \left(\sum_{i \in [|\mathcal{V}| - |\mathcal{W}|]} (-1)^i \binom{|\mathcal{V}| - |\mathcal{W}|}{i} \right) \end{aligned}$$

Now, by the generic binomial theorem we have that for $n \in \mathbb{N}, n \neq 0$

$$0 = (1 - 1)^n = \sum_{i \in [n]} (-1)^i \binom{n}{i}.$$

Therefore, the summands for $\mathcal{W} \neq \mathcal{V}$ vanish and we have

$$\begin{aligned} & \ln \left[\prod_{\mathcal{U} \subset \mathcal{V}} \prod_{\mathcal{W} \subset \mathcal{U}} \langle \tau \rangle_{[X_{\mathcal{V} \setminus \mathcal{W}} = x_{\mathcal{V} \setminus \mathcal{W}}, X_{\mathcal{W}} = y_{\mathcal{W}}]} \right]^{(-1)^{|\mathcal{U}| - |\mathcal{W}|}} \\ &= \ln [\tau [X_{\mathcal{V}} = x_{\mathcal{V}}]] \left(\sum_{i \in [0]} (-1)^i \binom{0}{i} \right) \\ &= \ln [\tau [X_{\mathcal{V}} = x_{\mathcal{V}}]]. \end{aligned}$$

Applying the exponential function on both sides establishes the claim. $\square$

Lemma 5. Let τ be a positive tensor and $\mathcal{U} \subset \mathcal{V}$ an arbitrary subset. When there are $a, b \in \mathcal{U}$ such that

$$\langle \tau \rangle_{[X_a, X_b | X_{\mathcal{V} \setminus \{a, b\}}]} = \left\langle \langle \tau \rangle_{[X_a | X_{\mathcal{V} \setminus \{a, b\}}]}, \langle \tau \rangle_{[X_b | X_{\mathcal{V} \setminus \{a, b\}}]} \right\rangle_{[X_{\mathcal{U}}]},$$

then for any indices $y_{\mathcal{U}}$ and $x_{\mathcal{U}}$

$$\prod_{\mathcal{W} \subset \mathcal{U}} \left(\langle \tau \rangle_{[X_{\mathcal{V} \setminus \mathcal{W}} = x_{\mathcal{V} \setminus \mathcal{W}}, X_{\mathcal{W}} = y_{\mathcal{W}}]} \right)^{(-1)^{|\mathcal{U}| - |\mathcal{W}|}} = 1.$$

Proof. We abbreviate

$$Z_{\mathcal{W}} = \langle \tau \rangle_{[X_{\mathcal{V} \setminus \mathcal{W}} = x_{\mathcal{V} \setminus \mathcal{W}}, X_{\mathcal{W}} = y_{\mathcal{W}}]}.$$

By reorganizing the sum over $\mathcal{W} \subset \mathcal{U}$ into $\mathcal{W} \subset \mathcal{U} \setminus \{a \cup b\}$ we have

$$\prod_{\mathcal{W} \subset \mathcal{U}} (Z_{\mathcal{W}})^{(-1)^{|\mathcal{U}| - |\mathcal{W}|}} = \prod_{\mathcal{W} \subset \mathcal{U} \setminus \{a, b\}} \left(\frac{Z_{\mathcal{W}} \cdot Z_{\mathcal{W} \cup \{a, b\}}}{Z_{\mathcal{W} \cup \{a\}} \cdot Z_{\mathcal{W} \cup \{b\}}} \right)^{(-1)^{|\mathcal{U}| - |\mathcal{W}|}}. \quad (3)$$

From the independence assumption it follows that for any index x

$$\begin{aligned} & \langle \tau \rangle_{[X_a = x_a | X_{\mathcal{V} \setminus \{\mathcal{W} \cup \{a, b\}\}} = x_{\mathcal{V} \setminus \{\mathcal{W} \cup \{a, b\}\}}, X_{\mathcal{W}} = y_{\mathcal{W}}, X_b = x_b]} \\ &= \langle \tau \rangle_{[X_a = x_a | X_{\mathcal{V} \setminus \{\mathcal{W} \cup \{a, b\}\}} = x_{\mathcal{V} \setminus \{\mathcal{W} \cup \{a, b\}\}}, X_{\mathcal{W}} = y_{\mathcal{W}}]} \\ &= \langle \tau \rangle_{[X_a = x_a | X_{\mathcal{V} \setminus \{\mathcal{W} \cup \{a, b\}\}} = x_{\mathcal{V} \setminus \{\mathcal{W} \cup \{a, b\}\}}, X_{\mathcal{W}} = y_{\mathcal{W}}, X_b = y_b]}. \end{aligned}$$

Applying this in each bracket term of (3) we get

$$\begin{aligned} \frac{Z_{\mathcal{W}}}{Z_{\mathcal{W} \cup \{a\}}} &= \frac{\langle \tau \rangle_{[X_a = x_a | X_{\mathcal{V} \setminus \{\mathcal{W} \cup \{a, b\}\}} = x_{\mathcal{V} \setminus \{\mathcal{W} \cup \{a, b\}\}}, X_{\mathcal{W}} = y_{\mathcal{W}}, X_b = x_b]}}{\langle \tau \rangle_{[X_a = y_a | X_{\mathcal{V} \setminus \{\mathcal{W} \cup \{a, b\}\}} = x_{\mathcal{V} \setminus \{\mathcal{W} \cup \{a, b\}\}}, X_{\mathcal{W}} = y_{\mathcal{W}}, X_b = x_b]}} \\ &= \frac{\langle \tau \rangle_{[X_a = x_a | X_{\mathcal{V} \setminus \{\mathcal{W} \cup \{a, b\}\}} = x_{\mathcal{V} \setminus \{\mathcal{W} \cup \{a, b\}\}}, X_{\mathcal{W}} = y_{\mathcal{W}}, X_b = y_b]}}{\langle \tau \rangle_{[X_a = y_a | X_{\mathcal{V} \setminus \{\mathcal{W} \cup \{a, b\}\}} = x_{\mathcal{V} \setminus \{\mathcal{W} \cup \{a, b\}\}}, X_{\mathcal{W}} = y_{\mathcal{W}}, X_b = y_b]}} \\ &= \frac{Z_{\mathcal{W} \cup \{b\}}}{Z_{\mathcal{W} \cup \{a, b\}}}. \end{aligned}$$

Thus, each factor in (3) is trivial, which establishes the claim. $\square$

We are finally ready to prove the Hammersley-Clifford Thm. 1 based on the Lemmata above.

Proof of Thm. 1. ii) $\Rightarrow$ i) By Lem. 4 we have for any indices $x_{\mathcal{V}}$ and $y_{\mathcal{V}}$

$$\mathbb{P}[X_{\mathcal{V}} = x_{\mathcal{V}}] = \prod_{\mathcal{U} \subset \mathcal{V}} \prod_{\mathcal{W} \subset \mathcal{U}} (\mathbb{P}[X_{\mathcal{W}} = x_{\mathcal{W}}, X_{\mathcal{V} \setminus \mathcal{W}} = y_{\mathcal{V} \setminus \mathcal{W}}])^{(-1)^{|\mathcal{U}| - |\mathcal{W}|}}.$$

Using the clique-capturing assumption of Thm. 1, we find for any subset $\mathcal{U} \subset \mathcal{V}$, which is not contained in a hyperedge $a, b \in \mathcal{U}$ such that X_a is independent of X_b conditioned on $X_{\mathcal{U} \setminus \{a, b\}}$. If no such nodes $a, b \in \mathcal{U}$ exists, $\mathcal{U}$ would be contained in a hyperedge since the hypergraph is assumed to be clique-capturing. By Lem. 5 we then have

$$\prod_{\mathcal{W} \subset \mathcal{U}} (\mathbb{P}[X_{\mathcal{W}} = x_{\mathcal{W}}, X_{\mathcal{V} \setminus \mathcal{W}} = y_{\mathcal{V} \setminus \mathcal{W}}])^{(-1)^{|\mathcal{U}| - |\mathcal{W}|}} = 1.$$

Using the function

$$\alpha : \{\mathcal{U} : \exists e \in \mathcal{E} : \mathcal{U} \subset e\} \rightarrow \mathcal{E},$$

we label the remaining node subsets by a hyperedge containing the subset. For each $e \in \mathcal{E}$, we build the tensor

$$\tau^e[X_e] = \prod_{\mathcal{U} : \alpha(\mathcal{U}) = e} \prod_{\mathcal{W} \subset \mathcal{U}} (\mathbb{P}[X_{\mathcal{W}} = x_{\mathcal{W}}, X_{\mathcal{V} \setminus \mathcal{W}} = y_{\mathcal{V} \setminus \mathcal{W}}])^{(-1)^{|\mathcal{U}| - |\mathcal{W}|}}.$$

and get that

$$\begin{aligned} \mathbb{P}[X_{\mathcal{V}}] &= \langle \{\tau^e[X_e] : e \in \mathcal{E}\} \rangle_{[X_{\mathcal{V}}]} \\ &= \langle \{\tau^e[X_e] : e \in \mathcal{E}\} \rangle_{[X_{\mathcal{V}} | \emptyset]}. \end{aligned}$$

We have thus constructed a Markov network with trivial partition function, whose contraction coincides with the probability distribution.

i) $\Rightarrow$ ii): To show the converse statement, assume that there is a Markov network representing the distribution $\mathbb{P}[X_{\mathcal{V}}]$ and choose subsets $A, B, C \subset \mathcal{V}$ such that C separates A from B . Denote by $\mathcal{V}_0$ the nodes with paths to A , which do not contain a node in C and by $\mathcal{V}_1$ the nodes with paths to B , which do not contain a node in C . Furthermore, we denote by $\mathcal{E}_0$ the hyperedges which contain a node in $\mathcal{V}_0$ and by $\mathcal{E}_1$ the hyperedges which contain a node in $\mathcal{V}_1$. By assumption of separability, both sets $\mathcal{E}_0$ and $\mathcal{E}_1$ are disjoint and no node in A is in a hyperedge in $\mathcal{E}_1$ and respectively no node in B is in a hyperedge in $\mathcal{E}_0$. We then have

$$\begin{aligned} \langle \{\tau^e[X_e] : e \in \mathcal{E}\} \rangle_{[X_A, X_B | X_C = x_C]} &= \langle \{\tau^e[X_e] : e \in \mathcal{E}\} \cup \{\epsilon_{x_C}\} \rangle_{[X_A, X_B | \emptyset]} \\ &= \langle \{\tau^e : e \in \mathcal{E}_0\} \cup \{\epsilon_{x_C}\} \rangle_{[X_A | \emptyset]} \\ &\quad \otimes \langle \{\tau^e : e \in \mathcal{E}_1\} \cup \{\epsilon_{x_C}\} \rangle_{[X_B | \emptyset]}. \end{aligned}$$

By Def. 11, this is the independence of X_A and X_B conditioned on X_C . □

A.2 Fisher-Neyman

Since sufficient statistics are sometimes introduced based on the data processing inequality (see e.g. Cover and Thomas (2006)), we also show that also that definition is equivalent to the factorization of the family. Here, $I(X; Y)$ denotes the mutual information of two random variables X, Y .

Theorem 11 (Fisher-Neyman factorization theorem). *Let $\mathbb{P}[X, Z]$ be a joint distribution of variables X, Z with values $\text{val}(X), \text{val}(Z)$ and let t be a statistic, which maps $\text{val}(X)$ to $\text{val}(Y_t)$. We introduce a variable Y_t and define a joint distribution by*

$$\mathbb{P}[X, Y_t, Z] = \langle \mathbb{P}[X, Z], \beta^t[Y_t, X] \rangle_{[X, Y_t, Z]}.$$

The following are equivalent:

i) *The Data Processing Inequality holds straight, that is*

$$I(Z; X) = I(Z; Y_t)$$

ii) *$Z \rightarrow Y_t \rightarrow X$ is a Markov Chain, that is*

$$(Z \perp X) | Y_t$$

iii) There are tensors $\xi[Y_t, Z]$ and $\nu[X]$ such that for any $x \in \text{val}(X)$ and $z \in \text{val}(Z)$

$$\mathbb{P}[Z = z, X = x] = \xi[Y_t = t(x), Z = z] \cdot \nu[X = x].$$

Proof. $i) \Leftrightarrow ii)$: We always have

$$I(Z; X) = I(Z; (X, Y_t)) = I(Z; Y_t) + I(Z; X|Y_t)$$

and thus $i)$ is equivalent to

$$I(Z; X|Y_t) = 0.$$

Using the KL-divergence characterization of the mutual information, this is equivalent to

$$\mathbb{P}[Z, X|Y_t] = \langle \mathbb{P}[Z|Y_t], \mathbb{P}[X|Y_t] \rangle_{[Z, X, Y_t]}.$$

This is equivalent to the conditional independence statement $ii)$.

$ii) \Rightarrow iii)$: Let us assume $ii)$. For all $z \in \text{val}(Z)$ and $x \in \text{val}(X)$ we then have

$$\begin{aligned} \mathbb{P}[Z = z|X = x] &= \mathbb{P}[Z = z|X = x, Y_t = t(x)] \\ &= \mathbb{P}[Z = z|Y_t = t(x)] \end{aligned}$$

Here we used that Y_t has a deterministic dependence on X . Therefore, there is a tensor ξ such that for all $z \in \text{val}(Z)$ and $x \in \text{val}(X)$

$$\xi[Y_t = t(x), Z = z] = \mathbb{P}[Z = z|X = x].$$

We further define a tensor $\nu[X] = \mathbb{P}[X]$ and get

$$\begin{aligned} \mathbb{P}[Z = z, X = x] &= \mathbb{P}[X = x] \cdot \mathbb{P}[Z = z|X = x] \\ &= \xi[Y_t = t(x), Z = z] \cdot \nu[X = x]. \end{aligned}$$

$iii) \Rightarrow ii)$: When assuming $iii)$ we have for all $(x, z) \in \text{val}(Z) \times \text{val}(X)$

$$\begin{aligned} \mathbb{P}[Z = z|X = x] &= \langle \xi[Y_t, Z], \beta^t[Y_t, X], \nu[X] \rangle_{[Z=z|X=x]} \\ &= \langle \xi[Y_t, Z], \beta^t[Y_t, X = x], \nu[X = x] \rangle_{[Z=z|\emptyset]} \\ &= \langle \xi[Y_t, Z], \epsilon_{t(x)}[Y_t] \rangle_{[Z=z|\emptyset]} \\ &= \mathbb{P}[Z = z|Y_t = t(x)]. \end{aligned}$$

We further have for almost all $y_t \in \text{val}(Y_t)$, $z \in \text{val}(Z)$ and $x \in \text{val}(X)$ that $y_t = t(x)$ and

$$\mathbb{P}[Z = z|X = x, Y_t = y_t] = \mathbb{P}[Z = z|X = x]$$

and with the above at thus at almost all such pairs

$$\mathbb{P}[Z = z|X = x, Y_t = y_t] = \mathbb{P}[Z = z|Y_t = y_t].$$

This is equivalent to $ii)$. □

Thm. 2 follows from Thm. 11 by the equivalence of $ii)$ and $iii)$.

B Implementation of the algorithms and examples

The implementations of the algorithms and concepts are available at <https://github.com/tnreason/nesy-demonstration> and implemented with `tnreason` in the version 2.0.0.

B.1 Algorithm 1, 2 and 3 (Tree, Directed Belief and Constraint Propagation)

The three message passing algorithms are implemented as functions in one class `ContractionPropagation`, since they share common structure.

```
1  from tnreason.engine import contract
2  from tnreason.engine import create_from_slice_iterator as create
3
4
5  class ContractionPropagation:
6      """
7      Summary Class for the Tree Belief, Directed Belief and Constraint Propagation Algorithms
8      """
9      def __init__(self, cores):
10         self.cores = cores
11         self.directions = {send: [receive for receive in cores if
12                                 set(cores[send].colors) & set(
13                                     cores[receive].colors) and receive != send]
14                             for send in cores}
15         self.messages = {receive: {} for receive in self.cores}
16
17     def trivial_message(self, send, receive):
18         """
19         Prepares trivial message from the send to the receive hyperedge
20         """
21         commonColors = list(set(self.cores[send].colors) & set(self.cores[receive].colors))
22         shape = [self.cores[send].shape[i]
23                 for i, c in enumerate(self.cores[send].colors) if c in commonColors]
24         return create(shape=shape, colors=commonColors, sliceIterator=[(1, {})])
25
26     def calculate_message(self, send, receive):
27         """
28         Contract received messages with hypercore to send new
29         """
30         return contract({send: self.cores[send],
31                          **{preSend: self.messages[send][preSend] for preSend in
32                             self.messages[send]
33                             if preSend != receive}},
34                        openColors=list(set(self.cores[send].colors) &
35                                       set(self.cores[receive].colors)))
36
37     def tree_propagation(self):
38         """
39         Implementation of the Directed Belief Propagation Algorithm:
40         Messages are sent starting at the leafs and scheduled if all others received at a
41         core
42         """
43         schedule = [(send, receive) for send in self.cores for receive in
44                     self.directions[send] if len(self.directions[send]) == 1]
45         while len(schedule) > 0:
46             send, receive = schedule.pop()
47             self.messages[receive][send] = self.calculate_message(send, receive)
48             for next in self.directions[receive]:
49                 if (not receive in self.messages[next] and
50                     all([(otherSendKey in self.messages[receive] or otherSendKey == next
51                           for otherSendKey in self.directions[receive] if
52                             otherSendKey != receive)])):
53                     schedule.append((receive, next))
54
55     def directed_propagation(self, edgeDirections):
56         """
57         Implementation of the Directed Belief Propagation Algorithm:
58         Messages are sent in direction of the hypergraph
59         """
60         filteredDirections = {
61             send: [
62                 receive for receive in self.directions[send]
63                 if (common := set(self.cores[send].colors) &
64                    set(self.cores[receive].colors))
65                    and common.issubset(set(edgeDirections[send][1]))
66                    and common.issubset(set(edgeDirections[receive][0]))]
67         }
```

```

64         for send in self.directions
65     }
66
67     schedule = [(send, receive) for send in filteredDirections
68                 for receive in filteredDirections[send] if len(edgeDirections[send][0])
69                 ↪ == 0]
70
71     while len(schedule) > 0:
72         send, receive = schedule.pop()
73         self.messages[receive][send] = self.calculate_message(send, receive)
74         for x in set(edgeDirections[send][1]) & set(edgeDirections[receive][0]):
75             edgeDirections[receive][0].remove(x)
76         if len(edgeDirections[receive][0]) == 0:
77             schedule = schedule + [(receive, next) for next in
78                                     ↪ filteredDirections[receive]
79                                     if (receive, next) not in schedule]
80
81     def constraint_propagation(self, startSendKeys):
82         """
83         Implementation of the Constraint Propagation Algorithm:
84         Messages are resent, when the support of a received message has changed
85         """
86         schedule = [(send, receive) for send in startSendKeys for receive in
87                     self.directions[send]]
88         while len(schedule) > 0:
89             send, receive = schedule.pop()
90             message = (self.messages[receive][send].clone() if send in
91                       ↪ self.messages[receive]
92                       else self.trivial_message(send, receive))
93             cont = self.calculate_message(send, receive)
94
95             messageChanged = False
96             for val, pos in message:
97                 if message[pos] != 0 and cont[pos] == 0:
98                     message[pos] = - message[pos]
99                     messageChanged = True
100             self.messages[receive][send] = message
101
102             for next in self.directions[receive]:
103                 if messageChanged and next != receive and (receive, next) not in schedule:
104                     schedule.append((receive, next))

```

B.1.1 Example 13 and 14 (Integer Summation in m -adic Representation)

Following the decomposition of m -adic summations into local summations, the function `get_sum_tn` produces a corresponding tensor network of basis encodings. We test by coordinate retrieval operations, whether the summation is performed correctly.

```

1  from tnreason import engine
2  import math
3
4  from copy import deepcopy
5
6
7  def get_sum_tn(m, d):
8      return {"b_0": engine.create_from_slice_iterator(
9          shape=[m, 2, m, m],
10         colors=[f"Y_{0}", f"Z_{0}", f"X_{0}", f"TX_{0}"],
11         sliceIterator=[(1, {f"Y_{0}": (x + tx) % m, f"Z_{0}": math.floor((x + tx) / m),
12                           f"X_{0}": x, f"TX_{0}": tx}) for x in range(m) for tx in
13                           ↪ range(m)]),
14         **{f"middleBlock{k}": engine.create_from_slice_iterator(
15             shape=[m, 2, m, m, 2],
16             colors=[f"Y_{k}", f"Z_{k}", f"X_{k}", f"TX_{k}", f"Z_{k-1}"],
17             sliceIterator=[
18                 (1, {f"Y_{k}": (x + tx + z0) % m,
19                       f"Z_{k}": math.floor((x + tx + z0) / m),
20                       f"X_{k}": x, f"TX_{k}": tx, f"Z_{k-1}": z0}) for x
21                       in range(m) for tx in range(m) for z0 in range(2)]
22             ) for k in range(1, d - 1)},
23         **{f"b_{d-1}": engine.create_from_slice_iterator(
24             shape=[m, 2, m, m, 2],

```

```

24         colors=[f"Y_{d - 1}", f"Y_{d}", f"X_{d - 1}", f"TX_{d - 1}", f"Z_{d - 2}"],
25         sliceIterator=[
26             (1, {f"Y_{d - 1}": (x + tx + z0) % m, f"Y_{d}": math.floor((x + tx + z0) /
27                 ↪ m),
28                 f"X_{d - 1}": x, f"TX_{d - 1}": tx, f"Z_{d - 2}": z0}) for x
29             in range(m) for tx in range(m) for z0 in range(2)]
30     }}}
31
32 def encode_digits(num0, num1, m):
33     return {**{f"X_{len(num0) - 1 - i}_eC": engine.create_from_slice_iterator(shape=[m],
34         ↪ colors=[
35             f"X_{len(num0) - 1 - i}"], sliceIterator=[(1, {f"X_{len(num0) - 1 - i}":
36                 ↪ int(digit)}))] for
37             i, digit in enumerate(num0)},
38         **{f"TX_{len(num1) - 1 - i}_eC": engine.create_from_slice_iterator(shape=[m],
39             ↪ colors=[
40                 f"TX_{len(num1) - 1 - i}"], sliceIterator=[
41                 (1, {f"TX_{len(num0) - 1 - i}": int(digit)}))] for i, digit in
42                 enumerate(num1)}}
43
44 assert 1 == encode_digits("0001", "0000", 10)[f"X_0_eC"][f"X_0": 1]}
45 assert 0 == encode_digits("0001", "0000", 10)[f"X_0_eC"][f"X_0": 0]}
46
47 ## Example: 08+12=020 in basis 10
48 m = 10
49 catorder = 2
50 assert 1 == int(engine.contract(coreDict={**get_sum_tn(m, catorder), **encode_digits("08",
51     ↪ "12", m)}),
52     openColors=[f"Y_{k}" for k in range(catorder + 1)])[
53     {f"Y_2": 0, f"Y_1": 2, f"Y_0": 0}])
54
55 assert 1 == int(engine.contract(coreDict={**get_sum_tn(m, catorder), **encode_digits("00",
56     ↪ "00", m)}),
57     openColors=[][:])
58
59 ## Example: 10+11=101 in basis 2
60 m = 2
61 catorder = 2
62 assert 1 == int(engine.contract(coreDict={**get_sum_tn(m, catorder), **encode_digits("10",
63     ↪ "11", m)}),
64     openColors=[f"Y_{k}" for k in range(catorder + 1)])[
65     {f"Y_2": 1, f"Y_1": 0, f"Y_0": 1}])
66
67 assert 1 == int(engine.contract(coreDict={**get_sum_tn(m, catorder), **encode_digits("10",
68     ↪ "11", m)}),
69     openColors=[][:])
70
71 from demonstrations.comp_act_nets.algorithms import propagation as cp
72
73 edgeDirections = {
74     **{f"X_{i}_eC": [[], [f"X_{i}"]] for i in range(catorder)},
75     **{f"TX_{i}_eC": [[], [f"TX_{i}"]] for i in range(catorder)},
76     "b_0": [[f"X_0", "TX_0"], [f"Y_0", "Z_0"]],
77     **{f"b_{i}": [[f"X_{i}", f"TX_{i}", f"Z_{i - 1}"], [f"Y_{i}", f"Z_{i}"]]
78         for i in range(1, catorder - 1)},
79     f"b_{catorder - 1}": [[f"X_{catorder - 1}", f"TX_{catorder - 1}", f"Z_{catorder - 2}"],
80         [f"Y_{catorder - 1}", f"Y_{catorder}"]],
81 }
82
83 propagator = cp.ContractionPropagation(**get_sum_tn(m, catorder), **encode_digits("01",
84     ↪ "01", m))
85 propagator.directed_propagation(edgeDirections=deepcopy(edgeDirections))
86
87 ## Check whether the message arrived at b_1 states that the carry bit is 1
88 assert propagator.messages["b_1"]["b_0"][f"Z_0": 0] == 0
89 assert propagator.messages["b_1"]["b_0"][f"Z_0": 1] == 1
90
91 propagator = cp.ContractionPropagation(**get_sum_tn(m, catorder),
92     **encode_digits("10", "10", m))
93 propagator.directed_propagation(edgeDirections=deepcopy(edgeDirections))
94
95 ## Check whether the message arrived at b_1 states that the carry bit is 1
96 assert propagator.messages["b_1"]["b_0"][f"Z_0": 0] == 1
97 assert propagator.messages["b_1"]["b_0"][f"Z_0": 1] == 0

```

B.1.2 Example 7 and 12 (Student Markov network)

We here implement the Markov network on the hypergraph of Example 7, with tensors having independent random coordinates drawn from the uniform distribution on $[0, 1]$. We test in a final `assert` statement, whether the messages resulting from Algorithm 1 in a tree implementation contract to the marginal distribution, which we directly compute for comparison.

```

1  from tnreason.engine import create_random_core, contract
2
3  studentTensorNetwork = {
4      "t0": create_random_core(name="t0", colors=["G", "D", "I"], shape=[6, 3, 2]),
5      "t1": create_random_core(name="t1", colors=["L", "G"], shape=[2, 6]),
6      "t2": create_random_core(name="t2", colors=["I", "S"], shape=[2, 10]),
7  }
8
9  ## Execute the contraction propagation algorithm in the tree-based implementation
10
11 from demonstrations.comp_act_nets.algorithms import propagation as cp
12
13 propagator = cp.ContractionPropagation(studentTensorNetwork)
14 propagator.tree_propagation()
15
16 ## Test on the marginals of the variables "L", "G" (core "t1")
17
18 testContraction = contract(studentTensorNetwork, openColors=["L", "G"])
19 propContraction = contract({"mes_t0_t1": propagator.messages["t1"]["t0"],
20                             "t1": studentTensorNetwork["t1"]}, openColors=["L", "G"])
21
22 tolerance = 1e-6
23 for posDict in [{"L": 0, "G": 1}, {"L": 1, "G": 5}]:
24     assert abs(testContraction[posDict] - propContraction[posDict]) < tolerance

```

B.1.3 Example 19, 21 and 22 (Sudoku Game)

We implement the $n^2 \times n^2$ Sudoku with the start assignment given in Example 19 and apply the Constraint Propagation Algorithm 3 to deduce the full assignment. We then test whether the correct board assignment (given in Example 22) has been found.

```

1  from tnreason.engine import contract
2  from tnreason.engine import create_from_slice_iterator as create
3  import numpy as np
4
5
6  def create_sudoku_rule_tensor_network(n):
7      """
8      Creates a tensor network of  $n^2 \times n^2$   $\tau$  matrices to each Sudoku constraint
9      """
10     rulesSpecDict = {
11         ## Column Constraints
12         **{f"I_{r0}_{r1}_{c0}_{c1}_{i}": [f"X_{r0}_{r1}_{c0}_{c1}_{i}" for r0 in range(n) for r1
13         ↪ in range(n) for c0 in range(n) for c1 in range(n)
14         for i in range(n ** 2)],
15         ## Row Constraints
16         **{f"I_{r0}_{r1}_{c0}_{c1}_{i}": [f"X_{r0}_{r1}_{c0}_{c1}_{i}" for c0 in range(n) for c1
17         ↪ in range(n) for r0 in range(n) for r1 in range(n)
18         for i in range(n ** 2)],
19         ## Squares Constraints
20         **{f"I_{r0}_{r1}_{c0}_{c1}_{i}": [f"X_{r0}_{r1}_{c0}_{c1}_{i}" for r1 in range(n) for c1
21         ↪ in range(n) for r0 in range(n) for c0 in range(n)
22         for i in range(n ** 2)],
23         ## Position Constraints
24         **{f"I_{r0}_{r1}_{c0}_{c1}_{i}": [f"X_{r0}_{r1}_{c0}_{c1}_{i}" for i in range(n ** 2)]
25         for r0 in range(n) for r1 in range(n) for c0 in range(n) for c1 in range(n)}
26     }
27     cores = {}
28     for decomKey in rulesSpecDict:
29         cores.update({
30             decomKey + "_" + atomVar: create(

```

```

31         shape=[2, len(rulesSpecDict[decomKey])],
32         colors=[atomVar, decomKey],
33         sliceIterator=[(1, {atomVar: 0}),
34                        (-1, {atomVar: 0, decomKey: i}),
35                        (1, {atomVar: 1, decomKey: i})])
36         for i, atomVar in enumerate(rulesSpecDict[decomKey])
37     })
38     return cores
39
40
41 def encode_trivial_extended_evidence(E, n):
42     """
43     Prepares e1 basis vectors for known variables and trivial vectors for others
44     """
45     return {**{f"{r0}_{r1}_{c0}_{c1}_{i}_eC":
46                create(shape=[2], colors=[f"X_{r0}_{r1}_{c0}_{c1}_{i}"],
47                      sliceIterator=[(1, {f"X_{r0}_{r1}_{c0}_{c1}_{i}": 1})])
48                for r0, r1, c0, c1, i in E},
49             **{f"{r0}_{r1}_{c0}_{c1}_{i}_eC":
50                create(shape=[2], colors=[f"X_{r0}_{r1}_{c0}_{c1}_{i}"],
51                      sliceIterator=[(1, {})])
52                for r0 in range(n) for r1 in range(n) for c0 in range(n)
53                for c1 in range(n) for i in range(n ** 2) if (r0, r1, c0, c1, i) not in E}}
54
55
56 def extract_resulting_evidence(propagator, n):
57     """
58     Returns the evidence given a ContractionPropagation instance
59     """
60     return [(r0, r1, c0, c1, i) for r0 in range(n) for r1 in range(n)
61            for c0 in range(n) for c1 in range(n) for i in range(n ** 2)
62            if contract({
63                "eC": propagator.cores[f"{r0}_{r1}_{c0}_{c1}_{i}_eC"],
64                **propagator.messages[f"{r0}_{r1}_{c0}_{c1}_{i}_eC"],
65                openColors=[f"X_{r0}_{r1}_{c0}_{c1}_{i}"])[f"X_{r0}_{r1}_{c0}_{c1}_{i}": 0]] ==
66            ↪ 0]
67
68 def tuples_to_array(evidence, n=2):
69     """
70     Arranges the variables in an array
71     """
72     array = np.zeros(shape=(n ** 2, n ** 2))
73     for (r0, r1, c0, c1, i) in evidence:
74         array[r0 * n + r1, c0 * n + c1] = i + 1
75     return array
76
77
78 from demonstrations.comp_act_nets.algorithms import propagation as cp
79
80 n = 2
81 evidence = [(0, 0, 0, 0, 0), (0, 0, 1, 0, 2), (0, 0, 1, 1, 1),
82            (0, 1, 0, 1, 1), (1, 0, 1, 0, 3), (1, 1, 0, 0, 3),
83            (1, 1, 0, 1, 2)]
84 propagator = cp.ContractionPropagation(
85     cores={**create_sudoku_rule_tensor_network(n=n),
86            **encode_trivial_extended_evidence(evidence, n=n)})
87 propagator.constraint_propagation([f"{r0}_{r1}_{c0}_{c1}_{i}_eC" for (r0, r1, c0, c1, i) in
88     ↪ evidence])
89 solutionArray = tuples_to_array(extract_resulting_evidence(propagator, n=2))
90 assert np.all(solutionArray == np.array([[1, 4, 3, 2], [3, 2, 1, 4], [2, 1, 4, 3], [4, 3, 2,
91     ↪ 1]]))

```

B.2 Algorithm 4 (Alternating Moment Matching)

We implement the Alternating Moment Matching algorithm, which estimates the parameters of Hybrid Logic Networks, as a class `MomentMatcher`.

```

1 from tnreason.engine import contract
2 from tnreason.engine import create_from_slice_iterator as create
3 import math
4

```

```

5
6 class MomentMatcher:
7     def __init__(self, cores, hCols, satRates):
8         self.cores = cores
9         self.hCols = hCols
10        self.satRates = satRates
11
12        self.hardParams = {hCol: int(satRates[hCol]) for hCol in self.hCols if
13                           satRates[hCol] in [0, 1]}
14        self.softParams = {hCol: 0 for hCol in self.hCols if hCol not in self.hardParams}
15
16    def update_canonical_parameter(self, uCol):
17        con = contract(**self.cores,
18                      **{hCol: create(shape=[2], colors=[hCol],
19                                     sliceIterator=[(1, {hCol: self.hardParams[hCol]})])
20                      for hCol in self.hardParams},
21                      **{hCol: create(shape=[2], colors=[hCol],
22                                     sliceIterator=[(1, {hCol: 0}),
23                                                       (math.exp(self.softParams[hCol]),
24                                                         {hCol: 1})])
25                      for hCol in self.softParams if hCol != uCol}
26                      }, openColors=[uCol])
27        self.softParams[uCol] = math.log(self.satRates[uCol] * con[{uCol: 0}] / (
28            (1 - self.satRates[uCol]) * con[{uCol: 1}]))
29
30    def alternate(self, iterations=1):
31        for _ in range(iterations):
32            for hCol in self.softParams:
33                self.update_canonical_parameter(hCol)

```

Let us now show the usage of the algorithm on the toy accounting model presented in Example 23. To this end we train the parameters based on a the dataset described in Example 24, and assert that the learned parameters are close to the true parameters. Note that a single iterations suffices for convergence in this simple example.

```

1 import pandas as pd
2
3 samples = pd.DataFrame({
4     "A1": [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1],
5     "A2": [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
6     "F": [0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1],
7 })
8
9 formulaExpressions = {
10     "(xor_A1_A2)": ["xor", "A1", "A2", 0],
11     "(imp_F_A1)": ["imp", "F", "A1", 0],
12 }
13
14 from tnreason.engine import normalize
15 from tnreason.application import data_to_cores as dtc
16 from tnreason.application import create_cores_to_expressionsDict as cte
17 from demonstrations.comp_act_nets.algorithms import moment_matching as mm
18
19 satRates = {
20     formulaKey + "_cV":
21         normalize(**dtc.create_data_cores(samples),
22                  **cte({formulaKey: formulaExpressions[formulaKey]}),
23                  outColors=[formulaKey + "_cV"], inColors=[])[{formulaKey + "_cV": 1}]
24     for formulaKey in formulaExpressions
25 }
26
27 matcher = mm.MomentMatcher(cores=cte(formulaExpressions),
28                             satRates=satRates, hCols=["(xor_A1_A2)_cV", "(imp_F_A1)_cV"])
29 matcher.alternate(iterations=1)
30 assert abs(matcher.softParams["(imp_F_A1)_cV"] - 1.09861228866811) < 1e-8

```